Zyphor

[Zai-Fer]

Timothy Ka

Contents

Analysis

Introduction:

Often physics is dismissed as a boring academic field for geniuses. It is not hard to understand why, after all, the first image that usually pops up is either Einstein or some random images of differential equations on dynamical systems. It is even more unfortunate for some who regard it as destroying beauty, dejecting life from nature with numbers and symbols. This however cannot be further from the truth, physics is the study of nature, to explain the mysteries. At least in my perspective, the explanation provided by physics allows for even greater marvelling of the world. It is not only physical beauty which we can observe, but an additional layer of mathematical elegance.

In the modern world, most teenagers spend most of their days on electronic devices. In fact, in the UK, 13-15 year olds spend 11 hours and 40 minutes gaming per week. The 16-18s also rank quite high. It is unfathomable how much more advanced physics and society as a whole would be if that time could be converted for learning. Of course, it is natural for teenagers to have fun, the solution would thus have to lie in making learning fun. There are existing learning apps which incorporate gaming features such as rank up, earn gems, etc after completing lessons or tests. But these apps were never as attractive as a first person shooter game for me. The only solution would therefore be creating a game which encompasses knowledge in itself, that the learning curve of the game would be the physics of it.

Initial Objective:

1. Attract the attention of people between 13-18 (End users)
2. Brings out physics knowledge while gaming

Initial Interview:

This is conducted to research popular games which my target users play, as well as the most important features of their favourite games to gain information about how to attract the target users. There are a total of four interviews for people of varying ages to gain a more complete insight.

Ollie:
How long do you spend gaming per week? 5 hours in school, 14 hours off school.
What types of games do you enjoy and play the most? FPS, Rainbow 6 siege, forza horizon 3, fortnite, sea of thieves.
Why do you play it and what is your main motivation? My friends play them, if friends play any other game then I will still play it. Forza Horizon 3, quite relaxing.
Do you gain anything from gaming? Teamwork, communication, military tactics and knowledge, reflex time.
What are your other hobbies? Photography, Guitar, Rowing
What would attract you to a newly released game? Good graphic design, good storyline, good communication.
What is the most important feature of a game in your view? Playable, easy to play.
Is there a preference for an open ended adventure type game to a quest driven game? A bit of both.

Joey:
How long do you spend on gaming per week? 10 - 15 hours
What types of games do you enjoy and play the most? FPS games that I played are CSGO, WoW, Forza Horizon 5.
Why do you play it and what is your main motivation? My friends play them, if friends play any other game then I will still play it
Do you gain anything from gaming? I don't gain skills from any games
What are your other hobbies? Badminton, music - explore on varying genres, beatbox
What would attract you to a newly released game? Good background music, original soundtrack, how it blends into the game. Unique, stands out from the playing style of other games. Also, games that can play with your friends
What is the most important feature of a game in your view? Can start quickly and end quickly, if a set time then less than an hour.
Is there a preference for an open ended adventure type game to a quest driven game? Don't mind either if there is a predefined quest of timeline.

Thomas:
How long do you spend on gaming per week? 8 hours (in school) off school dates 10
What types of games do you enjoy and play the most? FPS, car racing, Forza, Insurgency SandStorm, battlefield
Why do you play it and what is your main motivation? Insurgency SandStorm is brutal, like gore, pressure releasing; car-racing, just like motorsport enjoyment
Do you gain anything from gaming? No skill gains
What are your other hobbies? Fishing, Photography, Swimming
What would attract you to a newly released game? Realistic,both graphical capabilities and mechanics
What is the most important feature of a game in your view? Graphics rendering.
Is there a preference for an open ended adventure type game to a quest driven game? Prefer an open world game

Marcus:
How long do you spend gaming per week? 30-40 hours if not in school.
What types of games do you enjoy and play the most? FPS, escape from Tarkov.
Why do you play it and what is your main motivation? Complex systems, different injuries require different treatments and complex weapon modding. Steep learning curve, make me want to continue to play on until I become better.
Do you gain anything from gaming? I have learnt teamwork and preparation.
What would attract you to a newly released game? Complex games and varying ways to accomplish the same task, allows for creativity.
What is the most important feature of a game in your view? Complex and have a really steep learning curve.
Is there a preference for an open ended adventure type game to a quest driven game? Quest driven, have specific guidelines for gaming.

One general conclusion from 4 interviews from across the age group is that FPS remains the most popular, which is unsurprising as it generates the most adrenaline rush. Another basic feature that must be present is realism of the game, the graphical capabilities must be good (perhaps doesn't need to be absolutely top notch but still needs to be at least smooth and playing it feels natural). Marcus has

an anomaly with his love of complex systems, but that could be due to his extra maturity as a year 13 student in contrast to a year 9-11 student.

Further Interviews

I identified my main 'feedback client' to be Tommy Walker, a year 9 physics enthusiast. This allows for a greater understanding of the current situation for 13 year olds as they are heading towards the 'gaming peak' as shown from the research. Such important questions include their lifestyle and main interests, but more critically how a current student can manage to avoid falling into the trap but instead spends his time learning. The former could give invaluable feedback towards the style of game to be developed and the latter for the 'main attractive point' towards physics, such as one area in black hole.

I will also add on my comment on each aspect as I assumed the role when I was back at 13-14 years old. To provide the reader with some background context, I had only migrated to the UK from Hong Kong at year 10. My interest in physics however wasn't really developed until year 11. Another critical point which I need to discuss is the clashing working environment from my old school to my current one. Although my previous education is based upon the Canadian curriculum (at an international school), the location in Hong Kong gives it slightly more academic focus. I was able to gather a bunch of friends who would just hang out at break relaxing with discussion of academic journals or solving maths, which is unheard of here. This leads to me wasting 2 years (and continually culminating) on various games and I hope to help others break away from the trap which I am stuck in.

To aid for easier identification, I have labelled the questions in black, Tommy's response in blue, and my comments in red.

What is your first exposure to physics?

Reading books and YouTube when I was younger, along with science at school.

I had always watched YouTube on physics, ie Veritasium, SmartEveryDay, etc. But my first book and serious 'physics' came in year 11 after the reading of Hawking's 'A Brief History of Time'.

What captures your interest in physics?

I like really enjoy maths, especially algebra, but I also like understanding things, both conceptually and mathematically.

It is about the encapsulation of natural beauty into mathematical elegance that really grabbed my attention.

What do you do most in your free time?

I either read if I have the time, watch YouTube (both educational or recreational) scroll on Instagram or YouTube shorts, I watch and follow a lot of maths and physics, so I like learning even if it's in minute bites. Gaming too.

(Me at year 9) Watching YouTube on physics, maths and football as well as arguing with other people over futile matters (which I still enjoy today, although the gaming bit has increased exponentially)

Do you prefer to learn physics by hardcore (textbook, etc) or learn it while doing something else that ignites an interest in a particular field. (For example, you come across the newest next gen electric cars and wonder how it works)?

Both, I really like the hardcore practise style because I find it enjoyable really ingraining information. I also however have learnt a lot of physics by studying black holes in depth. I can hyper study one subject and then change when I've learnt everything. I learn a lot from this about physics.

I had personally never 'properly' learned physics, it is always just the interesting things that captures my attention before I delve further into the topic.

Are good resources for learning physics in a non-formal way (think of when you could relax and enjoy your time while learning – one example for me is Veritasium the YT guy) hard to find?

There are loads, Kurzgesagt is amazing. People don't realise how important visual learning is. Nearly all great discoveries came from someone imagining something, pictures details questions etc. There are loads of content that excels at this excellently, like on YouTube.

There is loads of content on YouTube and I do regret not discovering some absolute hidden gems earlier such as in year 9 or 10. However, there isn't really an easy systematic approach to it which is also why my friend and I are working on a website aiming to provide a more structured learning style and hopefully be able to connect to this project.

If you have to identify one main challenge to learning physics, what would that be?

It's hard to say. All my mistakes are mostly from silly errors or fatigue. I would say pleasing the mark scheme is harder than the actual subject.

From year 10 to now, it is the focus to learn that is the hardest for me.

If you do game, how much do you spend gaming per week (school and holiday)?

Maybe an hour a day, can get to 4 hours a day in the holidays.

(Year 9) Maybe up to an hour per day, but usually limits myself to below 15-30 minutes.

What are the common games(types/genres) that are played between your peers?

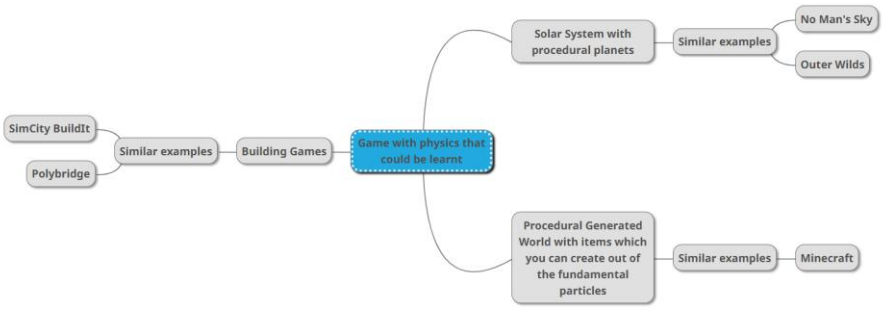
I love open world games, but games like Pokémon (turned based) which is also ironically now open world are great too. I like solving puzzles but also 'twitch games' like BoTW or TotK were there's lots fighting, but intriguing story also.

(Year 9) I played games like World of Tanks mostly with some occasional football manager-ish games such as Top Eleven, bethemanager, etc. The former is due to the fact that my church friends love to play it and the latter is of personal interest. Commonly though, FPS games are quite common.
What are the topics of interest which your peers talk about?

Annoyingly, my peers don't often go into enough depth, or they understand what I'm talking about. I have a friend who talks about politics a lot. It's not particularly amazing but it's interesting to see what he (mostly wrongly) thinks about world affairs, migration etc.

(Year 9) As stated earlier, we were more into doing maths and solving puzzles or discussing random academic related stuff, but I do know that is not the current reality.

Brainstorm of Ideas:



Each focuses on a different aspect of physics: building games enables players to learn classical mechanics, procedural planets allow for astrophysics, and procedural worlds can give light for understanding the fundamental particles.

Thinking back to the first physics ideas which both me and Tommy got dragged into as black holes, it makes sense to explore the astrophysics part further; as Tommy said, it is about imagination, which I will further into elegance of beauty. I believe that mechanics is too ‘mathsy’ for the average person, fundamental particles are just straight up very confusing for the majority (even some of my peers doing A-level physics struggles to grasp hold of it). Therefore, the only suitable option is procedural planets where real life planets and celestial phenomena such as black holes, neutron stars, etc could be explored with learning celestial mechanics on the way. It is also necessary to create something that gives the player ‘excitement’ which they crave in their FPS games, as well as ‘tasks’ to complete. For now, an idea for the former is to implement some sort of Aliens to combat and space warfare as an extension. One extension is to implement the Kardashev Scale, where the aim of the game is to reach the final level of civilization.

Existing systems

No Man’s Sky

“No Man’s Sky” is an action-adventure survival game developed by Hello Games. I have chosen to start with this game as it's the most popular procedural planet software available. The game is set in an infinite, procedurally generated universe and allows players to engage in five principal activities: exploration, survival, combat, trading, and base building. In total, it has 18 quintillion planets, each with unique biomes. The main storyline, also known as the Artemis Path, involves the player character uncovering the mysteries of Artemis and the Fourth Race, as well as the true nature of the Atlas, the Portals, and the universe of the game as a whole. However, these aren’t the sole purpose of

the game but rather as side missions that allow for unlocking different features, such as resetting the universe or ability to identify black holes.

The procedural generation system in “No Man’s Sky” starts with a base model for each entity type (like creatures or spaceships). This model is a collection of potential body parts and geometries. These descriptors can then be mashed together and assembled through code to make a new entity, their texture generation and animations. In an interview by its co-founder, it was stated that the game is built with a galactic mouth. This enables creation of galaxies of stars, with each star having its own solar system with planets filled with life and ecology. This is done via ‘colouring’ random squares on an infinite 3D scrolling graph paper. The planets are then generated around it, as well as everything on it. These are also ‘planet-sized’, where you could walk for literal weeks and weeks around the planet before eventually coming back to the same point. The rendering generation is only done when the player goes there, done via inputting the location of the player. If you fly away then it would be thrown away but if coming back again would give the same exact generation as previous. Two people on the same planet will also always see the same thing. The creatures are first created as silhouettes by artists, before applying code to give variations. More mutated characters are given extra aggressive characteristics, also adapting their animations. It implements a level of detail system where flying towards would cause it to be generated in higher and higher details. Throughout the video, he also reiterates the importance of maths such as the usage of fourier transform, differential equations. More importantly, all of the terrain is based on procedural algorithms and with one click of regeneration the whole terrain would look completely different than that of before.

A critical difference for No Man Sky’s orbital mechanics is that there are no orbital mechanics, the planets don’t actually move or rotate. Instead, the effects of sunrise and sunset are generated by the rotation of a skybox around the planet, with stars being textures in the skybox so it could never be reached. This means that every planet would experience the same day-night cycle as the sun and stars are painted to be infinitely far away. Natural phenomena such as storms are also not scientifically accurate but rather follow the path of the user.

Finally, although it simulates the whole universe, the distances between universes were not accurate representations of real-world astronomical data. It uses units of u, ks and Light Years for various distances. However, even though there is light year in the real world, NMS implemented a very scaled down inaccurate version. (A light year is supposed to be about 9.46 trillion kilometres but the maximum jump range for a freighter in the game is about 6341 LY, approximately 60 trillion kilometres in one jump). In terms of terrain generation, it uses a pseudo-random algorithm which represents a ‘map’ of all the celestial bodies including its properties but which is not rendered procedurally until you travel near enough the bodies. Disappointingly, it uses uniform gravity on all bodies with the exception of airless planets. It also implements ‘warp drive’ as a means of travelling between star systems, which is a sci-fi exaggerated version of the real life Alcubierre drive that allows for ‘faster than light’ travel. All in all, it can be said to prioritise gameplay over scientific accuracy.

Outer Wilds

“Outer Wilds" is an action-adventure game developed by Mobius Digital and published by Annapurna Interactive. The game is much harder to fully quantify compared to ‘No Man’s Sky’ due to the delicacy of its puzzle-like nature, as the game objective itself is to fuel the player’s curiosity in discovering ‘clues’ to why things are as they are: especially why the planetary system is stuck at a 22 minute loop. This is focused as the main aim in contrast to No Man’s Sky, with the storyline as the

only objective. If equipped with all knowledge of the game, the actual gameplay duration is rather short, with speedrunners able to complete the game in 10 minutes.

As a brief overview of the game, the player is set to explore the system on foot and in a small spacecraft, investigating the alien ruins of the Nomai. One particular feature which I found fascinating is the implementation of Quantum Shards and Quantum Moons. These implemented the basis of quantum mechanics into macroscopic level objects. For example, the shards can be found by Signalscrope across four planets which each having 3-4 locations that they can be found at once the line of sight has been broken. These shards guide the players to learn the 3 rules of quantum objects: Quantum Imaging, Quantum Entanglement and Rule of Sixth Location. The first two being real life phenomena and are very innovative for their method in educating gamers on these topics, albeit its unrealism.

However, “Outer Wilds” does not use procedural generation in the same way as “No Man’s Sky”. Instead of generating its universe on the fly, the worlds and explorable locations in “Outer Wilds” are hand-crafted, manually creating each planet, moon, and other celestial bodies in the game. This approach allows for a high level of detail and control over the game environment, ensuring that each location is unique and carefully designed.

Yet, the game does incorporate some elements of procedural generation. For example, the behaviour of certain entities in the game, such as the celestial bodies and their orbits, is determined by physics simulations. These simulations take into account factors like gravity and momentum, resulting in dynamic and unpredictable movements. (However, it uses inverse linear rather than inverse quadratic such as Newton’s law.) This adds a layer of realism to the game and ensures that each playthrough is unique.

SpaceEngine

SpaceEngine is a software of a realistic virtual Universe which the player can explore at his own pace. The Universe is generated procedurally based on real scientific data and real celestial objects such as planets and moons of our Solar System and stars nearby with all of the newly discovered exoplanets, as well as thousands of galaxies that are known. As it is more of a simulation and education tool, it lacks a game feature but it does have a Space Flight School that offers a hands-on guide for SpaceEngine ship piloting. The ship is also very detailed, with various ship controls, HUD all following spacecraft flying concepts and allow the user to attempt space navigation such as how to enter a stable orbit. One extra feature is the ability for time control where the speed of time can be altered to observe any celestial phenomena the user wishes to.

For the gritty nitty of its process, the developers start by collecting real world data from various astronomical catalogues such as Hipparcos, NGC and IC. The planets are then placed into the engine with these parameters. It then uses scientifically accurate procedural generation algorithms, meaning that it is created with confinement with such examples being surfaces of real planets being generated based on their properties.

Universe Sandbox

This is more of a game version of SpaceEngine. However, unlike the first two, the appeal is to visualise, create and destroy on an unimaginable scale rather than being objective based. Similarly, it

has features of time control, allowing users to witness evolution of planetary systems in sped up frames. In turn, historical events such as total solar eclipse or supernova can be witnessed. However, the most conspicuous feature is its ability to ‘create your own system’. Here you could either start with an existing star system such as the solar system and throw in a random star to see what would happen or just create a new system from scratch and you can witness its evolution.

One unique implementation is its extensive modelling of planetary climates using the energy balance equation and global average energy balance. It goes even further for Earth and Mars with consideration of latitudinal variations to redistribute energy between the equator and poles. This is to add on greenhouse effects, atmosphere and oceans to ensure realism with the warmth of the planet. It also uses a material system where each planet is assigned a composition of different materials. The composition then affects the procedural generation for unique celestial bodies. Finally, for orbital simulation, it uses N-body simulation with Newton’s law of universal gravitation.

Other Systems

There are many other systems of planetary exploration such as Spore, Kerbal Space Program 2, ASTRONEER, Starfield, Stellaris, Star Citizen and Space Engineers. These each allow for different aims, mainly being terraforming/base building and role playing. Although these are in their own respect great games, it deviates from the aim of my project. One major criticism of such role playing games is that it will either be extremely complicated, more complicated than Starfield or Star Citizen (both been criticised for its lack of actual freedom in role play), or a poor man’s version of an already unlikable game. Building rockets or space bases or the ability to terraform planets would be pretty cool, but that is not my objective of the game and therefore could be overlooked for now.

General Aim:

With topic research finished, I conclude that the game is to be a free exploratory game of the solar system. This is a game where players could control a spaceship to fly about in free space and land without cut scenes. The player should be able to move freely on the planet once landed. Each planet should be distinct and be reflective of real-life planets both in terms of size and shape. All the planets should follow Newton’s law of gravitation to portray realistic physics. This is so that players could explore and learn about the solar system interactively while having fun. I would very much like to have a storyline similar to outer wilds if given enough time, or maybe even atmospheric simulations.

Engine Choice:

As the game would require 3D graphics, this limits to mainly two options, unreal engine and Unity. I have an initial preference for unreal simply due to that it uses C++, which is a language I am planning to learn anyways compared to C# of Unity. There is godot as well, but the graphical capability of the engine is too low for my preference. (There is also no point learning GDscript as it is not a transferable skill, although it claims to be similar to python syntax, it is quite different in reality.)

	Unity	Unreal
Language	C#	C++ with Blueprints
Graphical Capabilities +	Decent, but not as good as	Very advanced, even games

performance	unreal, though 2D games are better supported	like fortnite are made with Unreal (supports distributed execution)
Learning Curve	Commonly argued as an ‘easier’ introductory due to C# having less complexity.	‘Harder’ with C++ but it allows for Blueprints which could make low complex coding very quick
Extra	Has great AR and VR support + a larger community, meaning more questions are resolved and could find help easier.	Includes multiplayer support and AI support. Although the community is slightly smaller, it still stands at 162k member

After speaking to my computer science teacher, although my initial idea is to go with Unreal Engine, Unity offers a better OOP learning environment as it does not include Blueprints which helps to streamline your process automatically. This would give better demonstration of understanding in OOP principles and form a solid foundation for the future, despite that C# is not a language that I would prefer to learn.

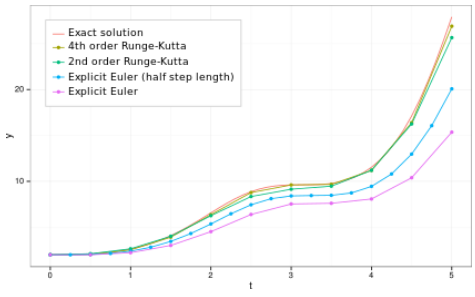
I have then found Sebastian Lague’s tutorial series on Unity which guided me through the basics of the engine.

Research + Prototype

Planetary Physics

Gravitational interaction between bodies are described by $F = GmM/r^2$, force = gravitational constant * mass of itself(m) * mass of the other object(M) / squared distance between the two. In order to calculate the resultant force, it is therefore necessary to complete vector addition of all forces on the body. This force is then applied into $F = ma$, or rearranged into $a = F/m$. As the self mass cancels out, the acceleration of the object is calculated by $a = GM/r^2$. Finally, the planets velocity is calculated by $v = u + at$, thus velocity += a*time, with position += velocity*time. As each time step between updating frames cannot be infinitesimally small (as exact solution from integration would be), it means we can only use numerical methods for updating its positions. The simplest is to directly update the position from velocity*time_step. This is known as the Euler’s method, the most straightforward numerical technique for approximating solutions for ordinary differential equations. However, over the long run it becomes rather inaccurate.

A better alternative is the Runge-Kutta 4th order method (RK4). Runge Kutta methods work by calculating slopes at multiple steps between the time steps, taking a Taylor series approximation with the weighted averages. (Euler’s method is technically a first order RK method, with full accuracy on the linear term.) Therefore the error term of Euler’s method is O(h^2) and for RK4 is O(h^5), with h being the



time step. RK4 – written as $y_{n+1} = y_n + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4)$ – is the most common RK method as computational cost increases significantly beyond the fourth order. (Due to RK4 being the highest order explicit Runge-Kutta method before requiring more steps than the order of accuracy. Ie to compute RK4, 4 steps in total are needed but for RK5 6 steps are needed.) The greatest accuracy advantage is therefore when h is very small, order of convergence becomes much steeper (due to higher order). (for any $h < 0$, $h^n < h$, with $h^{n+1} < h^n$ as well as $mh^n > h^n$ for $m > 1$.) However, that also means the error is much greater if h is large using the RK4 method, meaning a tradeoff for accuracy and speed of simulation. A solution to this is to use fix updates for behind the scene calculation of planetary positions with greater intervals. However, the practicality of such would be to be tested first, before refinements for the solution could be made.

NB: Numerical methods such as Euler or RK4 are only possible to solve first order ODEs, so a state vector is needed for calculation of the acceleration and velocity instead of directly calculating acceleration like a second order differential equation.

Procedural Generation

Terrain Generation:

Noise Function:

The most common and easiest approach is coherent noise function, which allows for continuous noise values and therefore smooth terrain. By far the most widely adopted system is **Perlin Noise**, developed by Ken Perlin in the 1980s. As an overview, it first defines a grid of random gradient vectors (depending on the dimension required). It then computes the dot product between gradient vectors and their offset to obtain a scalar value for each point. Finally, the scalar values are interpolated to generate the point's final noise values. This process ensures similar values between nearby points and therefore a continuous stream of noise.

There is an alternative of Simplex Noise, which is an updated variation produced by Ken Perlin to address the directional artefacts in Perlin Noise.

	Perlin Noise	Simplex Noise
Grid Structure	Square grid in 2D	Equilateral triangle grid in 2D
Value Calculation	Calculates the value of a point from the each corner of the square it is in	Calculates the value of a point from the each of the three corners of the triangle it is in
Artefacts	Has noticeable right-angle artefacts	Has less noticeable triangular 60-degree artefacts
Performance	Generally slower	Generally faster

Visual Properties	More mediated, less chaotic	More evenly spread, more chaotic
Usage	More widely used due to its simplicity and familiarity	Less widely used despite its efficiency and fewer artefacts
Time Complexity	O(2^n)	O(n^2)

There is also OpenSimplex Noise, which is implemented due to Simplex Noise's patent (which expired in 2022). This uses larger contribution kernels than Simplex noise, therefore smoother output and has reasonable visual isotropy due to the lack of directional bias. However, the larger kennels mean a drop in performance (more vertices to be evaluated) as well as a non-optimal gradient set. There is OpenSimplex2, a refined version that reduces non-optimal gradient sets. But there is not much point of implementing this larger version if Perlin Noise is available for usage (especially as a function of Mathf library in C#).

As I was planning to implement 3D voxel (which is an extension now), I had also found a very nice Noise Shader Library for Unity written by keijiro on both simplex and perlin noises of 2D and 3D variants for HLSL, (which is itself translated from the original webgl-noise library written by Stefan Gustavson and Ahima Art for referencing.) This could be used later if I am to use 3D noise.

Fractal Brownian motion (fBm):

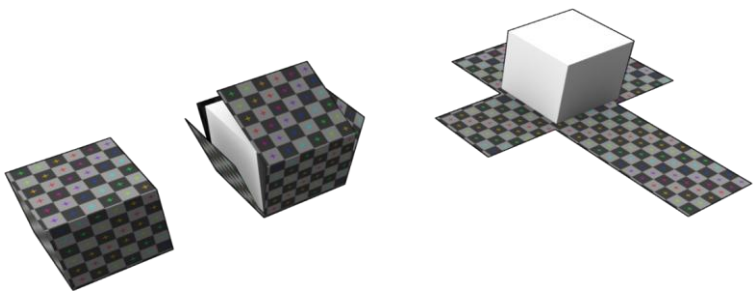
To make noise more interesting rather than just a bunch of identical mountains is to overlay levels of noise together, or implementing Fractal Brownian Motion. The two keywords are 'lacuarinty' and 'persistence', or just how much do you want to scale the frequency and amplitude (respectively) for each subsequent layer of noise.

Height-Based Terrain Sphere Generation:

The first step is to generate procedural planets, requiring some form of terrain applied onto a sphere-like mesh. Despite the simplicity of the target, it had proven to be much harder to even generate the sphere in the first place.

UV Map:

This generates a **2D** representation of a 3D object, the UV map can be texturized and is usually a predefined image before applying to GameObjects. However, as I am to procedurally generate the terrain and use its height to generate the terrain surface details, a predefined UV map here does not make too much sense.



Terrain Heightbased Map:

Unity offers Mesh class that takes inputs of vertices, triangles, and uv coordinates. The order of the triangles which you entered in would be the order of the output. (Hence direction of order matters as Unity only renders one side to save memory). It will be much more sensible to generate the terrain based off a noise then use that height to generate the colour based off a gradient. As Unity provides a gradient class, it speeds up this approach as well as opposed to a height based texture terrain.

Generation Methods:

One approach is using a subdivided icosahedron, a polygon of 42 vertices, with five triangles sharing each vertex. However, the main issue comes with the inability to fine twist the detailing of the surfaces, with each subdivision creating 4 new surfaces of each surface. This quickly blows up after 4-5 subdivisions and makes it very hard to fine tune the detailing of the sphere.

One alternate approach is to make use of the built-in primitive UV sphere, where a rectangular grid is mapped to a sphere. One common approach is to map it along the latitude and longitude. However, as the longitudinal lines are closer at the poles and furthest apart at the equator, there is an imbalance of details, with the poles being more geometrically detailed, leading to uneven balance of rendering.

A better approach is to use the Fibonacci sphere. This is due to the fact that the golden ratio (phi) is deemed to be the ‘most’ irrational number, meaning points generated on the sphere should be random and even spaced out. However, although it is very simple to generate the points on a sphere, difficulty arises with the generation of quads as it requires stereographic projection and Delaunay triangulation, which are both quite complex to achieve. This approach is also met with obstruction for implementation of a continuous level of detail system.

Another approach is to use an ‘inflated’ cube. This is done by making a cube and then mapping each vertex of the cube to change the points’ distance from centre to an equal radius. This is a much easier method to generate a sphere. However, the difficulty arises when attempting to implement CLOD, which it could do fantastically but the seams of the edges where there is a greater unequal size of triangles are quite hard to resolve.

The best solution it seems is to go back to the original icosahedron model, but this time rather than vertices on the midpoint of edge of each surface, it could be possible to generate more than a point on each edge for fining granulation. However, upon further research, it has proven to be painstakingly annoying to find the exact distances of the edges which are needed to model the sphere. Therefore, alternative approaches are needed.

A Voxel Approach:

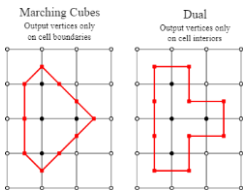
Luckily, Sebastian Lague strikes again with his coding adventure of Marching Cubes. Although this is meant to be more advanced as it allows for features like terraforming, caves and overhangs, I would say the concept is easier to understand than the method of multiple vertex per edge icosahedron. This is to add on there are virtually no tutorials on the latter method except a remark from Sebastian Lague as a very messy implementation from what he had done after figuring it on paper. This is therefore the best as it has a bonus of additional functionalities.

A brief explanation of the method would be as follows:

1. Some sort of signed distance function (SDF) that returns a value for any point in space – f(x,y,z)
2. Based on these values to determine where the isosurface is (i.e. a list of points where it equals to the same value, for simplicity 0 is taken – or else the function could be modified to get 0 anyways). This is done by detecting for changes of signs of points (one 'cube' – 8 points which form vertices of a cube – at each time, therefore its name). The value is then interpolated on the edges to determine where the isosurface should intersect the edge. The quads of meshes are then formed using a lookup table where although there are 256 cases of configurations of quads in a cube, there are only 15 unique ones.
3. This process is then repeated for all cubes until the mesh is finished.

I therefore tried to first just understand how Seb Lague implemented it, and it means further research into compute shaders which I had never heard of before and a new language of HLSL to pick up – there is not too much to the language. This method is very useful (combined with compute buffers) as it can offload the CPU's work onto the GPU and when doing procedurally generated objects (especially with 3D noise), this becomes much more efficient as each of the kernels can run in parallel and create a multithreaded system.

To avoid myself (unconsciously) copying him, I have decided to implement dual contouring instead. Another advantage is to avoid the complexity of the look-up tables for configuration, as well as better quality meshes (especially for sharp edges). See right for image to explain.



Since I have decided to use dual contouring, I should probably explain how it works, since I have chosen this and did it with all other methods that I have rejected. The first two steps are identical to marching cubes, so I will avoid a repeat.

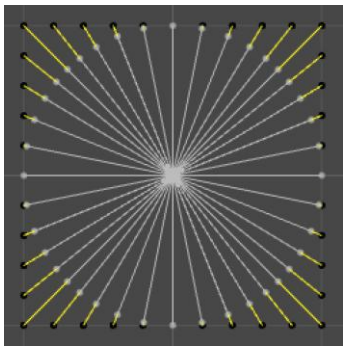
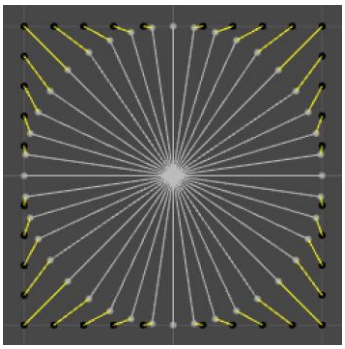
3. After putting a vertex on the edge, we need extra information of a gradient of the function (ie. just differentiate the function). This gradient is then used to find the normal at the vertex on edge. As there must be two vertices at least on each cube – other than the rare case of when the vertex is on the cube vertex – there must be two normals and where these normals touch is the vertex. (Notice as from the image that the vertex is within the cube).
4. Build the quads by connecting the vertices.

I have then read through Boris the Brave's python implementation and understand a bit better of how to implement it, but of which is rather tedious. Nevertheless, I tried to make a prototype of it in unity, but such an attempt was proven to be futile as I couldn't manage to generate the right type of meshes to give the desired effect. I have investigated some other examples of voxel continuous level of detail and realised it was more complicated than I first imagined. Therefore, I would first sidestep from implementing the voxel approach back into height-based map generation as I am running out of time for my analysis. This could be the first extension which I could do. However, the extra skills learnt especially for compute shaders could be transferable if I were to implement corrosion systems or generate an infinite terrain that requires simultaneous processing. So, it is not all just time wasted.

I have looked back at the types of sphere generations and decided that the 'inflated' cube is the best approach as it avoids the extra complexity which the Fibonacci mapping and the 'edged' icosahedron required. This however means a continuous LOD system would be harder to implement, meaning I need another method to support the landing. Upon further research, I discovered that there are two

types of implementations, normalised cube or sphere cube. The latter is a better version of the former that gives more uniform points and therefore more uniform detail.

Below offers a detailed comparison between the two methods:

	Normalised Cube	Sphereised Cube
2D Visual Differences (of a slice)		
Formulae difference	Take the normal of vectors of each point on the cube, (using the same vectors as the projection to the cube but changing their lengths to be equal)	Uses a clever mathematical derivation of $1-(1-x^2)(1-y^2)(1-z^2)$. This has a result of a mapping of the cube onto a sphere but with greater even distribution of points

However, it would be better to use the normalise function first before switching to the more even formulae to simplify development initially.

I had then also followed CatLikeCoding's tutorial series on spherical generation and noise generation, learning a bit about job+burst optimisation as well. This code could be seen in the appendix as it is pretty much a direct copy of his work. You can check his work here: CatLikeCoding

Here's a brief prototype for generating a 2D procedural terrain

Main script

```
using System.Collections;
using System.Collections.Generic;
using System.Linq;
using UnityEngine;

[RequireComponent(typeof(MeshFilter), typeof(MeshRenderer))]
public class MeshGen : MonoBehaviour
{
    Mesh mesh;
    Vector3[] vertices;
    int[] triangles;
    Color[] colors;

    int res = 256;

```

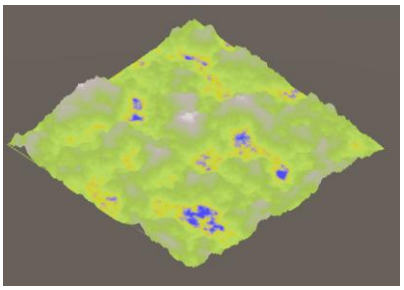
```
16.     public float scale = 50;
17.     public int seed;
18.     public float lacunarity = 2.0f;
19.     public Vector2 offset;
20.     public int octave = 6;
21.     public float persistance = 0.6f;
22.     public float amplitude = 20;
23.     public int frequency = 1;
24.     public Gradient gradient;
25.
26.     private float maxPossibleHeight = 0;
27.     private Vector2[] octaveOffsets;
28.     private float[,] heightMap;
29.     private float[] inverseMap;
30.
31.     public AnimationCurve heightCurve;
32.
33.     private void Awake()
34.     {
35.         mesh = new Mesh();
36.         transform.GetComponent<MeshFilter>().mesh = mesh;
37.         GenerateMesh();
38.     }
39.
40.     private void OnValidate()
41.     {
42.         GenerateMesh();
43.     }
44.
45.     private void GenerateMesh()
46.     {
47.         (maxPossibleHeight, octaveOffsets) = MapGen.GenSeed(seed, octave, offset,
persistance);
48.         ///Debug.Log("Octave Offsets: " + string.Join(", ", octaveOffsets.Select(o =>
o.ToString()).ToArray()));
49.         heightMap = MapGen.GenHeightMap(res + 1, res + 1, seed, octaveOffsets, scale,
octave, persistance, lacunarity, frequency);
50.         inverseMap = MapGen.GenInverseMap(maxPossibleHeight, heightMap);
51.         colors = MapGen.GenColorMap(inverseMap, gradient);
52.         CreateVertices();
53.         CreateTriangles();
54.         UpdateMesh();
55.     }
56.
57.     private void CreateVertices()
58.     {
59.         vertices = new Vector3[(res + 1) * (res + 1)];
60.         for (int i = 0; i < vertices.Length; i++)
61.         {
62.             int x = i % (res + 1);
63.             int y = i / (res + 1);
64.             vertices[i] = new Vector3(x, amplitude*heightCurve.Evaluate(inverseMap[i]), y);
65.         }
66.     }
67.
68.     private void CreateTriangles()
69.     {
70.         triangles = new int[res * res * 6];
71.
72.         int vert = 0;
73.         int tris = 0;
74.
75.         for (int y = 0; y < res; y++)
76.         {
77.             for (int x = 0; x < res; x++)
78.             {
79.                 triangles[tris] = vert;
80.                 triangles[tris + 1] = vert + res + 1;
81.                 triangles[tris + 2] = vert + 1;
82.                 triangles[tris + 3] = vert + 1;
```

```
                triangles[tris + 4] = vert + res + 1;
                triangles[tris + 5] = vert + res + 2;
                vert++;
                tris += 6;
            }
            vert++;
        }
    }

    private void UpdateMesh()
    {
        mesh.Clear();
        mesh.vertices = vertices;
        mesh.triangles = triangles;
        mesh.colors = colors;
        mesh.RecalculateNormals();
    }
}

```

Static functional class:

```
using System.Collections;
using System.Collections.Generic;
using System.Runtime.CompilerServices;
using System.Runtime.ExceptionServices;
using Unity.Burst;
using UnityEditor.UI;
using UnityEngine;

public static class MapGen
{
    public static (float, Vector2[]) GenSeed(int seed, int octave, Vector2 offset, float persistance)
    {
        float maxPossibleHeight = 0;
        float testAmplitude = 1f;

        System.Random prng = new System.Random(seed);
        Vector2[] octaveOffsets = new Vector2[octave];

        for (int i = 0; i < octave; i++)
        {
            float offsetX = prng.Next(-100000, 100000) + offset.x;
            float offsetY = prng.Next(-100000, 100000) + offset.y;
            octaveOffsets[i] = new Vector2(offsetX, offsetY);

            maxPossibleHeight += testAmplitude;
            testAmplitude *= persistance;

        }
        return (maxPossibleHeight, octaveOffsets);
    }

    public static float[,] GenHeightMap(int mapWidth, int mapHeight, int seed, Vector2[] octaveOffsets, float scale, int octave, float persistance, float lacunarity, float frequency = 1f, float noiseHeight = 0f)
    {
        float amplitude = 1f;
        float[,] heightMap = new float[mapWidth, mapHeight];

        // Centre the map
        float halfWidth = mapWidth / 2f;
        float halfHeight = mapHeight / 2f;

        for (int x = 0; x < mapWidth; x++)
        {
            for (int y = 0; y < mapHeight; y++)
```

```
44.             {
45.                 float localAmplitude = amplitude;
46.                 float localFrequency = frequency;
47.                 float localNoiseHeight = noiseHeight;
48.
49.                 for (int i = 0; i < octave; i++)
50.                 {
51.                     float sampleX = ((x - halfWidth) / scale + octaveOffsets[i].x) *
localFrequency;
52.                     float sampleY = ((y - halfHeight) / scale + octaveOffsets[i].y) *
localFrequency;
53.
54.                     //Flat floor level
55.                     float perlinValue = Mathf.PerlinNoise(sampleX, sampleY) * 2 - 1;
56.                     localNoiseHeight += perlinValue * localAmplitude;
57.                     localAmplitude *= persistance;
58.                     localFrequency *= lacunarity;
59.                 }
60.                 heightMap[x, y] = localNoiseHeight;
61.             }
62.         }
63.         return heightMap;
64.     }
65.
66.     public static float[] GenInverseMap(float maxPossibleHeight, float[,] heightMap)
67.     {
68.         int width = heightMap.GetLength(0);
69.         int height = heightMap.GetLength(1);
70.
71.         float[] inverseMap = new float[width * height];
72.
73.         for (int y = 0; y < height; y++)
74.         {
75.             for (int x = 0; x < width; x++)
76.             {
77.                 float inverseHeight = Mathf.InverseLerp(-maxPossibleHeight*0.7f,
maxPossibleHeight*0.7f, heightMap[x, y]);
78.                 inverseMap[y*height+x] = inverseHeight;
79.             }
80.         }
81.
82.         return inverseMap;
83.     }
84.
85.     public static Color[] GenColorMap(float[] inverseMap, Gradient gradient)
86.     {
87.
88.         Color[] colorMapColors = new Color[inverseMap.Length];
89.         for (int i = 0; i < inverseMap.Length; i++)
90.         {
91.             Color heightValue = gradient.Evaluate(inverseMap[i]);
92.             colorMapColors[i] = heightValue;
93.         }
94.         return colorMapColors;
95.     }
96. }
97.
```



Brief Explanation:
This prototype generates one chunk of terrain using Perlin noise and fbm for overlay. I have split into a main class which deals mesh handling and generation and a helper static class which contains the algorithms such as offset generation from seed, fbm, colour mapping from an inverse map (needs to be in the interval [0,1] in order for colour gradient.Evaluate to work – for a height based colour sampling which I determine using the gradient).

End User Requirements

After finishing the research and prototyping, I went back to conduct a final interview with Tommy to check on the updated requirements due to feasibility of production. As per before, Tommy's answer is labelled in red.

Would you prefer to have a largely scaled down version of the solar system or a more realistic version that fulfils the boundaries which unity could have?
I think realism would really help as it gives the accurate reflection of the scale of our universe and that is really one major thing I would say is quite amazing about it.

In order to make it a game, I was thinking of a storyline, one similar to the game Outer Wilds to avoid the extra complexity of implementing a shooter system. What are your thoughts on it, would you think it will be attractive to play?
Certainly, it will give more of an action/RPG type of game rather than a game of Fortnite, which is still one of the most popular genres that me and my peers play.

Limitations

1. Limited to the scope of 1 solar system, ie our own
2. Scaled down version of the solar system to fulfil mass restrictions on RigidBody for unity.
3. No biomes or animals existing on planets, it wouldn't be habitable, only some natural terrain which is traversable.
4. Wouldn't be able to simulate the effects of exposure to harmful radiation in outer space.
5. Wouldn't be able to mimic the sun accurately to demonstrate its brightness and hotness
6. Players would not die in the game.
7. Critically, I only have a year to develop such a game, with other subjects in mind, time will be really tight, so I need to be efficient and limit the scope of expansion in the project

Agreement with End User

After the finalised research, Tommy and I have finalised on the following objectives:

1. Procedural generation of noise mimicking planets of solar system

1.1. Use technique of Fractional Brownian Motion to generate 3D terrain
    1.1.1. Add upon the built-in Unity.Mathematics.noise.snoise function for noise variants
    1.1.2. Use seed to generate the base layer of noise to ensure that it remains constant every time it is being generated (pseudo-random)
    1.1.3. Tweak values for each of the 8 planets in solar system to match its looks
        1.1.3.1. [Optional] Use a base mesh filter for each planet for enhanced realism
        1.1.3.2. [Optional] Implement ridged noise for sharp features such as mountains or valleys, falls into experimenting section with other mathematical functions that could be used to deform terrain to generate desired result
    1.1.4. Use offset to ensure that each layer of noise comes from a different part of the simplex noise function for distinctiveness
    1.1.5. Implement a gradient through lerp of colours for various heights to generate height-based colours reflective of real terrain of planets
        1.1.5.1. [Optional] Implement shaders with techniques such as Triplanar mapping
2. Mesh generation for procedural real sized planets
    2.1. Generate by the sphere with technique of spherised cube
        2.1.1. [Optional] Optimise with more even formulae
    2.2. Model with real data (ie their velocity, position, mass, etc all scaled down)
        2.2.1. Each planet has a unique mesh to reflect off real life structures
    2.3. Implement surface gravity on the bodies which is varied proportionally to the mass of the planets
        2.3.1. Bounciness of Player On Terrain
    2.4. [Optional] Implement rotation on planets
    2.5. Friction on planets to avoid players slipping around due to the planet's rotation (if implemented) and movement
3. Implement continuous Level of Detail system using quadtrees
    3.1. Implement chunked terrain that allows for endless non-repeating terrain reflective of movement on a spherical planet
    3.2. Allows maximum detail of mesh without a significant drop in fps
4. [Extension] Optimise the generation of meshes with multithreading and scheduling
    4.1. Use Unity's job and burst system for best performance possible
5. Create a spaceship to fly around in, this would also be affected by gravitational forces of other bodies.
    5.1. Spaceship model could be taken from unity asset store
    5.2. When landed players can leave the spaceship and transfer into surface gravity and walk around the planet reflecting realistic physics interactions
6. Realistic gravitational attraction between bodies
    6.1. Implement gravity systems between celestial bodies with Newtonian law of universal gravitation.
    6.2. Find a way to balance out realism but fitting the constraint of Unity, its physics and rigidBody usages
    6.3. Use RK4 for greater accuracy over varying time steps
        6.3.1. Allow the user to control how fast or slow the simulation should be running
7. Create a 2D map for the solar system that you can view when flying

7.1. Have markers around planets to indicate the spaceship's velocity and distance away from planet
7.2. [Optional] Embed the minimap into the UI of the spaceship giving it a realistic look
7.3. [Extension 1] Build an auto-pilot system which could trace a path towards a planet after selecting on the 2D map to guide you there
7.4. [Extension 2] Include moons and satellites for planets in the solar system.
7.4.1. [Extension 4] Include other star systems with greater than one star. This could be unstable, but it should be fine as long as time is running rather slowly.
7.4.2. [Extension 5] Create a map of the universe for (include in game map of the universe, like a gaming map where if you could travel through astronomical concepts such as wormholes – ie where the player directly teleports to another universe)
7.4.2.1. [Extension 7] Implement cosmological phenomena such as neutron stars and black holes with realistic gravitational pull such that players would get sucked into and die.

8. Have a camera which follows the player/spaceship that stays away from them in a fixed distance.
8.1. Rendering of planets using Camera as the origin for optimisation such as occlusion culling to reduce vertices and triangles rendered
8.2. Allows swapping between the cams of player and spaceship seamlessly

9. [Extension] Create atmosphere of planets
9.1. [Extension] each planet should have its own unique atmosphere which matches their signature atmosphere in real life
9.2. [Extension ^{2}] such gaseous celestial bodies should only have the atmosphere and no internal core (such as sun/Jupiter, etc). Use the varying density of such atmospheres to calculate the distance which the spaceship can travel before getting flattened

10. [Extension] Implement using voxel technique such as dual contouring which allows for intricate structures and level of details
10.1. [Extension 3b] Implement an octree based continuous level of detail system to allow for players landing on the planets
10.1.1. [Extension 6] Allows for players to terraform the terrain after landing. This would make it possible for the player to 'colonise' the celestial bodies and building bases.

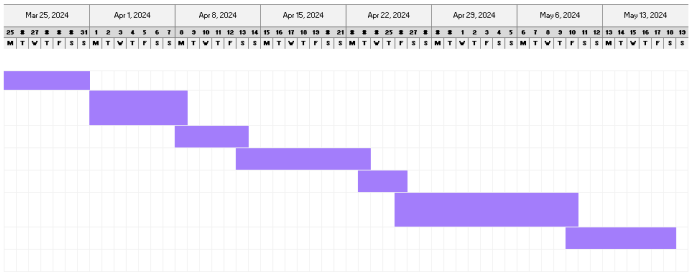
11. [Extension] Create a storyline like that of outer wilds but with facts of physics and questions which physicists had answered to provoke curiosity. This would give a scenario which begins on Earth with a 'telescope' ability to view the solar system from Earth but then able to explore them and answer the questions which had been posed to solve with building a complete image of the universe.

Critical Path

Planet Sim NEA

Project start: Mon, 4/1/2024
Display week: 0

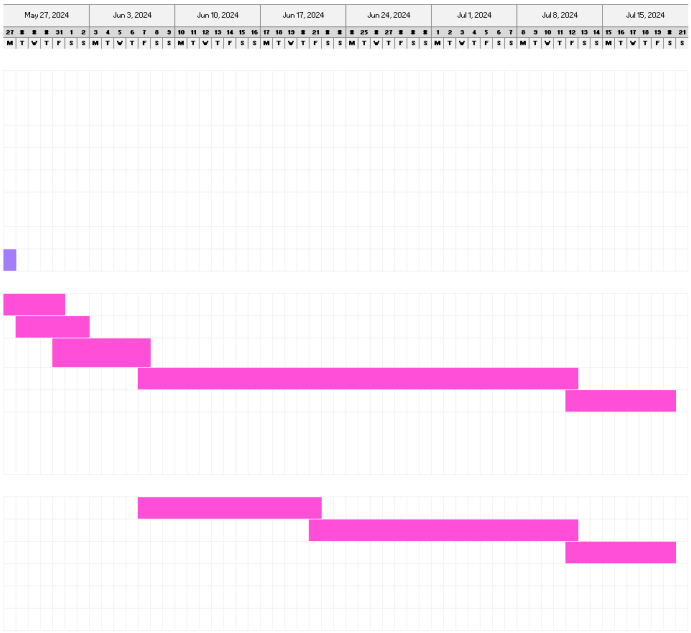
TASK	START	END
Analysis		
Decide Initial Objectives	25/03/2024	3/31/24
Interview For Project Definition + Find Suitable Client	4/1/24	4/8/24
Brainstorm Ideas	4/8/24	4/13/24
Research Existing System	4/13/24	4/23/24
Research Engine Choice	4/23/24	4/26/24
Research Algorithms Needed (ie gravitation, procedural gen, etc)	4/26/24	5/10/24
Prototyping	5/10/24	5/24/24
End User Requirements	5/24/24	5/27/24



Planet Sim NEA

Project start: Mon, 4/1/2024
Display week: 9

TASK	START	END
Analysis		
Decide Initial Objectives	25/03/2024	3/31/24
Interview For Project Definition + Find Suitable Client	4/1/24	4/8/24
Brainstorm Ideas	4/8/24	4/13/24
Research Existing System	4/13/24	4/23/24
Research Engine Choice	4/23/24	4/26/24
Research Algorithms Needed (ie gravitation, procedural gen, etc)	4/26/24	5/10/24
Prototyping	5/10/24	5/24/24
End User Requirements	5/24/24	5/27/24
Design		
Data Structures	5/27/24	5/31/24
UML Diagram	5/28/24	6/2/24
Further diagrams such as structure chart/ flow chart	5/31/24	6/7/24
Pseudocode/ First iteration of planet gen	6/7/24	7/12/24
Explain gravitation and RK4	7/12/24	8/2/24
Explain how I combine both systems together for the game + reasoning for values I choose and compatibility with built in Unity system	8/2/24	8/5/24
Code		
2D Quadtree and LOD	6/7/24	6/21/24
3D Planet with Quadtree and LOD + Noise	6/21/24	7/12/24
RK4 Gravitation	7/12/24	8/2/24
Spaceship control + landing	8/2/24	8/16/24
Player control	8/16/24	8/23/24
Optimisation + Job + Burst	8/23/24	9/13/24



Planet Sim NEA

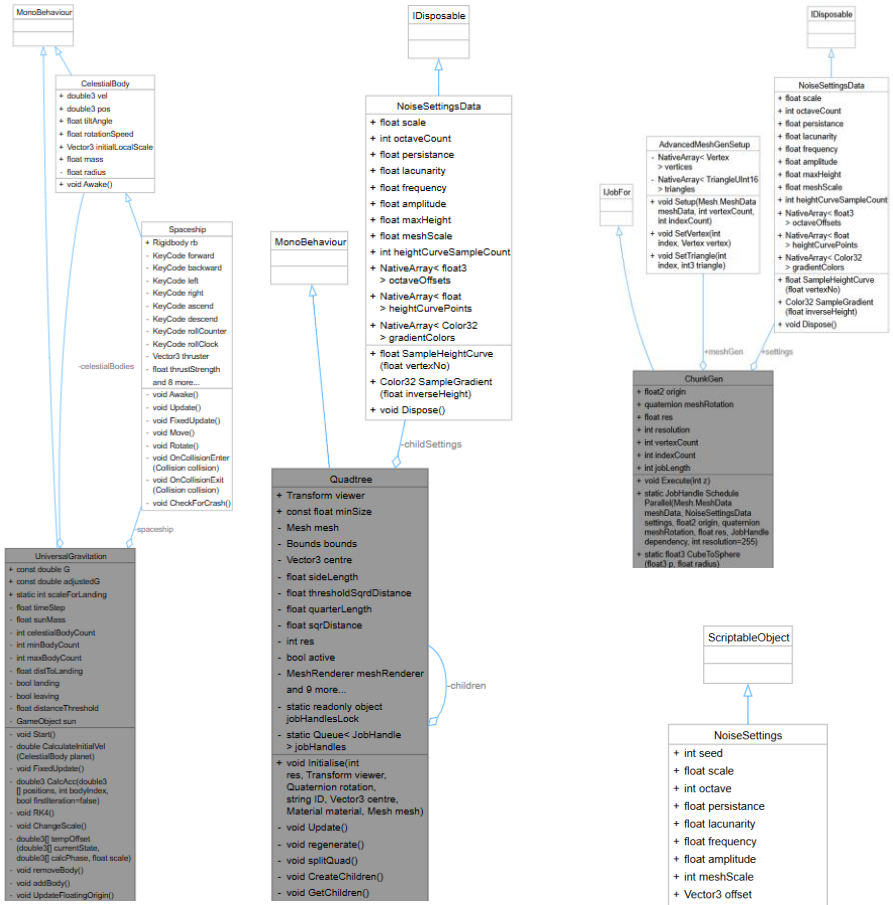
Project start: Mon, 4/1/2024

Display week: 17

TASK	START	END
Analysis		
Decide Initial Objectives	25/03/2024	3/31/24
Interview For Project Definition + Find Suitable Client	4/1/24	4/8/24
Brainstorm Ideas	4/8/24	4/13/24
Research Existing System	4/13/24	4/23/24
Research Engine Choice	4/23/24	4/26/24
Research Algorithms Needed (ie gravitation, procedural gen, etc)	4/26/24	5/10/24
Prototyping	5/10/24	5/24/24
End User Requirements	5/24/24	5/27/24
Design		
Data Structures	5/27/24	5/31/24
UML Diagram	5/28/24	6/2/24
Further diagrams such as structure chart/ flowchart	5/31/24	6/7/24
Pseudocode/ First iteration of planet gen	6/7/24	7/12/24
Explain gravitation and RK4	7/12/24	8/2/24
Explain how I combine both systems together for the game + reasoning for values I choose and compatibility with built in Unity system	8/2/24	8/5/24
Code		
2D Quadtree and LOD	6/7/24	6/21/24
3D Planet with Quadtree and LOD + Noise	6/21/24	7/12/24
RK4 Gravitation	7/12/24	8/2/24
Spaceship control + landing	8/2/24	8/16/24
Player control	8/16/24	8/23/24
Optmisation + Job + Burst	8/23/24	9/13/24

Design

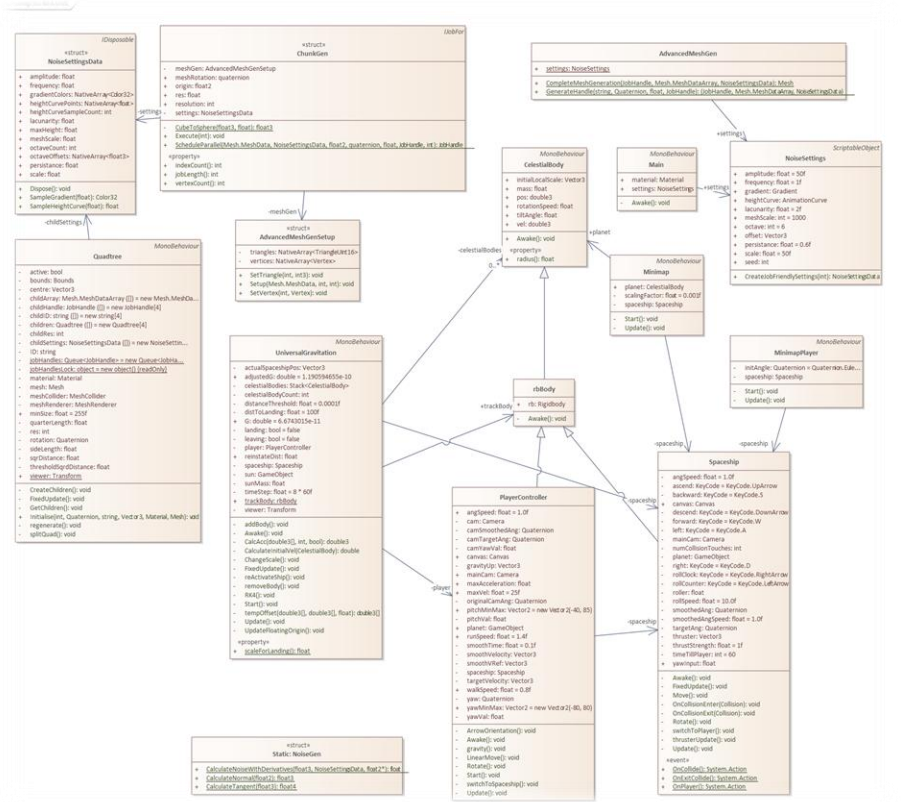
UML (Here is a UML for what I believe will be the critical classes for functionalities.)



Here the distinction between NoiseSettingsData and NoiseSettings should be clarified. NoiseSetting is a class inherited from Unity's scriptable object, which allows me to tweak the values of the terrain in the Unity editor. NoiseSettingsData however is a struct that is created based on the class. This is mainly to comply with Unity's job + burst system where jobs is inherently a struct and therefore parameters passed into it must also be of struct data type. The main reason for this is due to parallel multithreading, if a class type is used, which referenced to the same memory location, there could (and will) be clashes for accessing. Instead, struct by definition creates a copy of the class in a different location of the memory. Hence, it is important to always dispose the NoiseSettingsData after it is used to avoid clogging up memory!

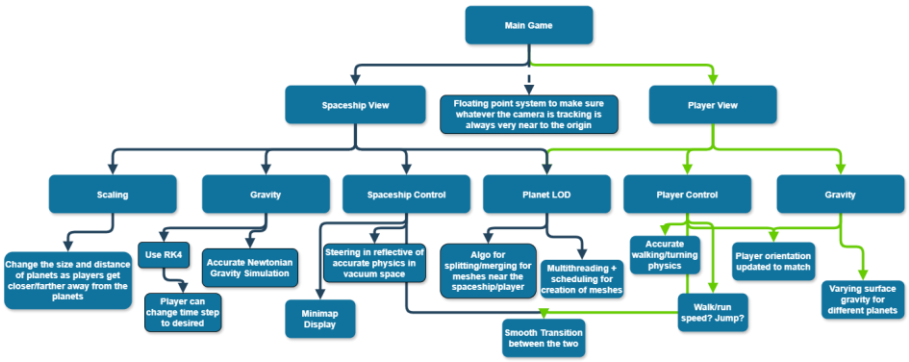
UML

Here is the final UML of the game after finished development.



NB: Arrows with white shaded triangle head represent inheritance whereas the normal is composition

Structure/Hierarchical Diagram:



(Initial) Brief Data Dictionary

Name	Type/Stuctures	Purpose
baseMap	float[,]	The 'smooth noise texture' that is used as the base layer of the terrain on planets
inverseMap	float[,]	Use Unity's Animation Curve to change the baseMap's height to the range [0,1] where I can tweak how each height is mapped to which values in [0,1]
gradientColor	Color/Color32[]	Sampling the colour base on the height (which is inverseLerped to put into the range [0,1] for sampling)
velocity	Vector3	Velocity of planets/spaceship
acceleration	Vector3	Acceleration of planets/spaceship
position	Vector3	Position of planets/spaceship
surfaceGravity	float	After landing on different planets, it is calculated based on the mass and density of the planet. This is to avoid real time simulation of other planet's gravitational pull
mass	float	Mass of planets

radius	float	Radius of planets
quadSplitThreshold	float	Check whether there should be a higher detail mesh or not
meshToGen	queue	Queuing up the meshes for generation with the closest ones to user having priority
Lacunarity/persistence/amplitude/frequency/scale/seed/octave	float/int	Variables to control the noise generation in fbm
Offset	Vector3	Noise offset to control which part of the noise texture to sample from
meshTag	JobHandle	Sent back for queuing the meshes which use the schedule function in job system

Libraries With No Books

With the game being developed in Unity, I extensively used their libraries and the .NET API System namespace to 1. Extra functionality for ease of use 2. Optimisation and compatibility with its engine. Here is a brief general overview of these libraries, the individual usages are covered later during the explanation of individual algorithms.

System Libraries:

1. Linq
    a. Language Integrated Query that extends the C# functionalities
        i. Used for array functionalities such as orderBy and LastOrDefault to sort or get data
2. Collections.Generic
    a. Type-safe collection in .NET, providing optimized data structures
        i. Queues<T>/ Stack<T>
3. Runtime.InteropServices
    a. Only used once with the struct definitions as a header.
    [StructLayout(LayoutKind.Sequential)] force a sequential layout of the data in struct
    → when multithreading the order won't be messed up

Unity (and its libraries):

1. UnityEngine
    a. Core library of Unity – fundamental classes and functionality
        i. Base system to work with GameObjects, Mesh, physics (rigidBody), Mathf, etc
    b. Everything is inherited from the base class Object
        i. Each GameObject has script(s) inherited from MonoBehaviour

        ii. NoiseSettings inherited from ScriptableObject, objects that I create as an asset to control the noise generation rather than as a gameObject
2. UnityEngine.Rendering
    a. This is to allow access to 'VertexAttributeDescriptor' and its corresponding functions which allows advanced mesh generation (directly manipulating the mesh data)
3. Mathematics
    a. A much better optimised version of Mathf for multithreading especially. (Even if it's not better in performance it is needed as I need the immutable struct type instead of the reference type which Mathf is based upon).
        i. Used its static noise namespace for snoise (simplex) as well
        ii. Used other normal maths operations for code within multithreading
4. Collections
    a. Provides unmanaged data structures to use in jobs and Burst-compiled code.
        i. NativeArray<T> (Allocator, NativeArrayOptions, etc)
5. Collections.LowLevel.Unsafe
    a. Only used once as headers to vertices and triangles in the mesh gen setup - [NativeDisableContainerSafetyRestriction] avoids the built-in constraints for handling struct in data types hence why the [StructLayout(LayoutKind.Sequential)] was needed for declaring those fields
6. Job
    a. System for writing multi-threaded code in Unity that utilizes available CPU cores
        i. Scheduling for dependency and thread safety
7. Burst
    a. Compiler technology that translates .NET bytecode into highly optimized native code
        i. SIMD vectorization support
        ii. Low level support for optimisation
    b. (Job + Burst integrate together for multithreading)

A Brief History of Time (Algorithms Used)

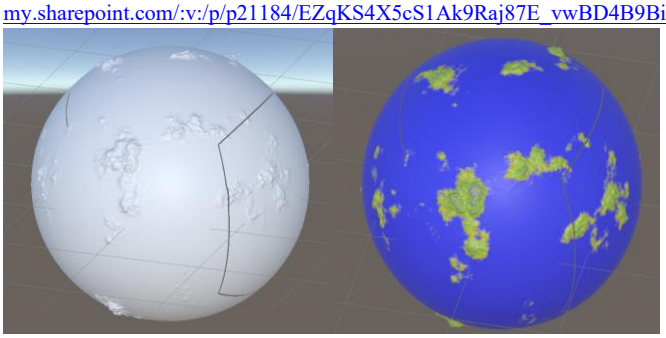
Planet LOD System

As explained in analysis, the first step is to generate a planet.

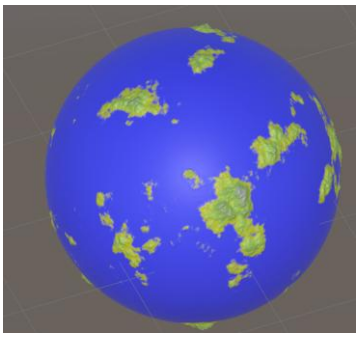
1. 6 square quadtree faces forming a cube → normalised to form a sphere
2. Each of the quadtree faces follows a recursive algorithm that splits up the face base on the distance from a viewer object (ie either the spaceship or the player)
    a. If it's below the distance it 'splits' and create 4 children that is half of its width and height until the quadtree mesh is of minimum
        i. These children are placed matching each corner of the initial parent, essentially subdividing the parent in its centre into 4 standard chunks
    b. If it's above the threshold then it 'reverts' back to its parent, disabling the children to save rendering power and maximise efficiency of nearby areas which the player can see most clearly
3. Generating 3D noise using simplex noise and fbm

    a. In the previous prototype I had selected Perlin, but I just thought simplex looks better in 3D with less artifact, technically it is the only dimension that is slightly slower as it is O(n^2) rather than O(2^n) but it really doesn't matter that much
    b. Incorporate the technique used for fbm (octaves, offsets, etc) to create more interesting and realistic terrain
    c. Use inverseLerp and Color to generate the colours from a colour gradient
4. Map the noise onto the sphere
    a. This bit is the trickiest and it depends on how I will exactly generate the 6 faces of a cube, and I will work from there

Here's a link to the first iteration of the planet:
https://canford-my.sharepoint.com/:v:/p/p21184/EZqKS4X5cS1Ak9Raj87E_vwBD4B9Biuo9NBmbYwx99HhVQ



The gaps here which is present is due to that I initially accidentally set the vertices as 257*257 instead of the max default of 256*256 of a default 16-bit mesh. But since it quite nicely shows the seams between the 6 faces, I left it in during development for clarity in patching up the noise generation of each of the chunk to make it match.



With the dimensions reworked, the result is as such. However, as seen from the video, the process of generation is very slow and freezes up the screen. So, in the future I would need to optimise it using the job + burst system for multithreading and scheduling to avoid freezes like this! However, as part of the critical path, it would be wiser to complete the gravitational system first before I optimise it.

Here is the code for it!

Main.cs

```
using System.Collections;
using System.Collections.Generic;
using System.Linq;
using System.Security.Cryptography;
using Unity.VisualScripting;
using UnityEditor;
using UnityEngine;

public class Main : MonoBehaviour
{
    public Material material;
    public NoiseSettings settings;
    private void Awake()
    {
        settings =
AssetDatabase.LoadAssetAtPath<NoiseSettings>("Assets/NoiseSettings.asset");
        MeshGen.settings = settings;
        Vector3[] rotations = new Vector3[6]
        {
            Vector3.zero,
            Vector3.forward,
            Vector3.forward*2,
            Vector3.back,
            Vector3.right,
            Vector3.left
        };

        Vector3[] origins = new Vector3[6]
        {
            Vector3.up,
            Vector3.left,
            Vector3.down,
            Vector3.right,
            Vector3.forward,
            Vector3.back
        };

        MeshGen.origins = origins;
        MeshGen.rotations = rotations;

        for (int i = 0; i < 6; i++)
        {
            GameObject childObj = new GameObject("Quadtree" + 0 + i.ToString());
            Quadtree quadtree = childObj.AddComponent<Quadtree>();
            quadtree.enabled = false;
            Vector3 origin = origins[i] * 128 * settings.meshScale;
            quadtree.Initialise(1, GameObject.FindGameObjectWithTag("Viewer").transform,
origin, Quaternion.Euler(rotations[i]*90), i.ToString(), origin, material);
            quadtree.enabled = true;
        }
    }

    private void OnValidate()
    {
        MeshGen.settings = settings;
    }
}

```

NoiseSettings.cs

```
using UnityEngine;

[CreateAssetMenu(fileName = "NoiseSettings", menuName = "Scriptable Objects/NoiseSettings")]
public class NoiseSettings : UpdatableData
{
    public int res = 256;
```

```
 7.     public float scale = 50;
 8.     public int seed;
 9.     public float lacunarity = 2.0f;
10.     public Vector3 offset;
11.     public int octave = 6;
12.     public float persistance = 0.6f;
13.     public float amplitude = 20;
14.     public int frequency = 1;
15.     public Gradient gradient;
16.     public AnimationCurve heightCurve;
17.     public int meshScale = 100;
18. }
19.
```

Here I used the Unity ScriptableObject class so I could tweak the values in Unity interface for different planet looks.

MapGen.cs

```
 1. using System.Collections;
 2. using System.Collections.Generic;
 3. using System.Runtime.CompilerServices;
 4. using System.Runtime.ExceptionServices;
 5. using Unity.Burst;
 6. using UnityEditor.UI;
 7. using UnityEngine;
 8. using Unity.Mathematics;
 9. using static Unity.Mathematics.math;
10.
11. public static class MapGen
12. {
13.     public static (float, Vector3[]) GenSeed(int seed, int octave, Vector3 offset, float
persistance)
14.     {
15.         float maxPossibleHeight = 0;
16.         float testAmplitude = 1f;
17.
18.         System.Random prng = new System.Random(seed);
19.         Vector3[] octaveOffsets = new Vector3[octave];
20.
21.         for (int i = 0; i < octave; i++)
22.         {
23.             float offsetX = prng.Next(-100000, 100000) + offset.x;
24.             float offsetY = prng.Next(-100000, 100000) + offset.y;
25.             float offsetZ = prng.Next(-100000, 100000) + offset.z;
26.             octaveOffsets[i] = new Vector3(offsetX, offsetY, offsetZ);
27.
28.             maxPossibleHeight += testAmplitude;
29.             testAmplitude *= persistance;
30.
31.         }
32.         return (maxPossibleHeight, octaveOffsets);
33.     }
34.
35.     // FINALLY IT WORKS PROPERLY!!!
36.     public static float[,] GenHeightMap(float res, int seed, Vector3[] octaveOffsets, float
scale, int octave, float persistance, float lacunarity, Vector2 bottomLeftCorner, Vector3
rotation, float frequency = 1f, float noiseHeight = 0f)
37.     {
38.         int length = 257;
39.         float amplitude = 1f;
40.         float[,] heightMap = new float[length, length];
41.         // Centre the map
42.         float halfLength = (length-1) /2f;
43.         Quaternion rotated = Quaternion.Euler(rotation*90);
44.         //Debug.Log("Rotation: " + rotated.eulerAngles);
```

```
 45.
 46.         for (float x = 0; x < length; x++)
 47.         {
 48.             for (float z = 0; z < length; z++)
 49.             {
 50.                 float localAmplitude = amplitude;
 51.                 float localFrequency = frequency;
 52.                 float localNoiseHeight = noiseHeight;
 53.
 54.                 for (int i = 0; i < octave; i++)
 55.                 {
 56.                     float sampleX = x/res + bottomLeftCorner.x;
 57.                     //float sampleX = ((x / res - threeQuarterLength + centre.x * (length -
1)) / scale) * localFrequency;
 58.                     float sampleZ = z / res + bottomLeftCorner.y ;
 59.                     //float sampleZ = ((z / res - threeQuarterLength + centre.y * (length -
1)) / scale) * localFrequency;
 60.                     Vector3 vertex = new Vector3(sampleX, 128, sampleZ);
 61.                     if (x == 0 && z == 0) Debug.Log("Vertex: " + vertex);
 62.                     vertex = vertex.normalized * 128;
 63.
 64.
 65.                     Vector3 rotatedVertex = rotated * vertex/scale * localFrequency;
 66.                     if (x == 0 && z == 0) Debug.Log("Rotated Vertex: " + rotatedVertex);
 67.
 68.                     //Flat floor level
 69.                     float simplexValue = noise.snoise((float3)(rotatedVertex +
octaveOffsets[i]*localFrequency)) * 2 - 1;
 70.                     localNoiseHeight += simplexValue * localAmplitude;
 71.                     localAmplitude *= persistance;
 72.                     localFrequency *= lacunarity;
 73.                 }
 74.                 heightMap[(int)x, (int)z] = localNoiseHeight;
 75.                 //Debug.Log("Height: " + localNoiseHeight);
 76.             }
 77.         }
 78.         return heightMap;
 79.     }
 80.
 81.     public static float[] GenInverseMap(float maxPossibleHeight, float[,] heightMap,
AnimationCurve heightCurve)
 82.     {
 83.         int width = heightMap.GetLength(0);
 84.         int height = heightMap.GetLength(1);
 85.
 86.         float[] inverseMap = new float[width * height];
 87.
 88.         for (int y = 0; y < height; y++)
 89.         {
 90.             for (int x = 0; x < width; x++)
 91.             {
 92.                 float inverseHeight = Mathf.InverseLerp(-maxPossibleHeight*0.7f,
maxPossibleHeight*0.7f, heightMap[x, y]);
 93.                 inverseMap[y*height+x] = heightCurve.Evaluate(inverseHeight);
 94.             }
 95.         }
 96.
 97.         return inverseMap;
 98.     }
 99.
100.     public static Color[] GenColorMap(float[] inverseMap, Gradient gradient)
101.     {
102.
103.         Color[] colorMapColors = new Color[inverseMap.Length];
104.         for (int i = 0; i < inverseMap.Length; i++)
105.         {
106.             Color heightValue = gradient.Evaluate(inverseMap[i]);
107.             colorMapColors[i] = heightValue;
108.         }
109.         return colorMapColors;
```

```
    }
}

```

MeshGen.cs

```
using System.Security.Cryptography;
using UnityEngine;
using UnityEngine.UIElements;
using Unity.Mathematics;
using static Unity.Mathematics.math;
using static UnityEngine.UI.Image;
using Unity.VisualScripting;

public static class MeshGen
{
    public static NoiseSettings settings;
    public static Vector3[] origins;
    public static Vector3[] rotations;

    public static Mesh GenerateMesh(int res, string ID, Vector3 position)
    {
        Mesh mesh = new Mesh();
        (float maxPossibleHeight, Vector3[] octaveOffsets) = MapGen.GenSeed(settings.seed, settings.octave, settings.offset, settings.persistance);
        //Debug.Log("max possible height: " + maxPossibleHeight);
        //Debug.Log("Octave Offsets: " + octaveOffsets[0] + octaveOffsets[1]);
        Debug.Log("Position" + position);
        // the line below is faulty! (no longer -- it's removed)
        Vector3 rotation = Vector3.zero;
        for (int i = 0; i < 6; i++)
        {
            if (position.normalized == origins[i])
            {
                rotation = rotations[i];
                break;
            }
        }

        string verticesID = ID.Substring(1);
        float scale = 0.5f;
        Vector2 origin = new Vector2(0, 0);
        int quarterLength = 6400;
        for (int i = 0; i < verticesID.Length; i++)
        {
            scale = exp2(i);
            char c = verticesID[i];
            switch (c)
            {
                case '0':
                    origin += Vector2.one * quarterLength / scale;
                    break;
                case '1':
                    origin += new Vector2(-1, 1) * quarterLength / scale;
                    break;
                case '2':
                    origin -= Vector2.one * quarterLength / scale;
                    break;
                case '3':
                    origin += new Vector2(1, -1) * quarterLength / scale;
                    break;
            }
        }
        Debug.Log("scale: " + scale);
        Debug.Log("Origin: " + origin);
        Vector2 bottomLeftCorner = origin - Vector2.one * quarterLength / scale;
        Debug.Log("bottomLeftCorner : "  + bottomLeftCorner);

```

```
        //Debug.Log(bottomLeftCorner);
        ///Debug.Log("Octave Offsets: " + string.Join(", ", octaveOffsets.Select(o => o.ToString()).ToArray()));
        float[,] heightMap = MapGen.GenHeightMap(res, settings.seed, octaveOffsets, settings.scale, settings.octave, settings.persistance, settings.lacunarity, bottomLeftCorner/100, rotation, settings.frequency);
        float[] inverseMap = MapGen.GenInverseMap(maxPossibleHeight, heightMap, settings.heightCurve);
        Color[] colors = MapGen.GenColorMap(inverseMap, settings.gradient);
        Vector3[] vertices = CreateVertices(res, settings.amplitude, inverseMap, settings.meshScale, origin);
        int[] triangles = CreateTriangles();
        UpdateMesh(mesh, vertices, triangles, colors);
        return mesh;
    }

    // issue 1 the create vertices function for normalisation gives changing normalisation origin instead of matching the origin of the mesh
    // issue is because I have gen the vertices assuming centre before shifting the position, I need to gen directly with its position of mesh!
    // fixed issue 1, but the positions of the meshes are out of place (shifted away from origin all by same scale factor)
    public static Vector3[] CreateVertices(int res, float amplitude, float[] inverseMap, int meshScale, Vector2 origin)
    {
        Vector3[] vertices = new Vector3[257 * 257];
        // changing the origin to match the sub quadtree mesh position relative to its max parent (disregarding orirentation)


        for (int i = 0; i < vertices.Length; i++)
        {
            float x = (i % 257 - 128) * meshScale / res + origin.x;
            float y = (i / 257 - 128) * meshScale / res + origin.y;
            Vector3 cubeVertex = new Vector3(x, 0, y);
            Vector3 worldCubeVertex = cubeVertex + Vector3.up*12800;
            //Debug.Log(cubeVertex.normalized);
            vertices[i] = worldCubeVertex.normalized*(12800+ amplitude * inverseMap[i]) - Vector3.up*12800;
            //vertices[i] = cubeVertex;
        }
        return vertices;
    }

    static float3 CubeToSphere (float3 p) => p * sqrt(
                        1f - ((p * p).yxx + (p * p).zzy) / 2f + (p * p).yxx * (p * p).zzy / 3f
            );

    public static int[] CreateTriangles()
    {
        int[] triangles = new int[256 * 256 * 6];

        int vert = 0;
        int tris = 0;

        for (int y = 0; y < 256; y++)
        {
            for (int x = 0; x < 256; x++)
            {
                triangles[tris] = vert;
                triangles[tris + 1] = vert + 257;
                triangles[tris + 2] = vert + 1;
                triangles[tris + 3] = vert + 1;
                triangles[tris + 4] = vert + 257;
                triangles[tris + 5] = vert + 257 + 1;
                vert++;
                tris += 6;
            }
            if(y == 255)
```

```
            {
                Debug.Log("Vert: " + vert);
            }
            vert++;
        }
        return triangles;
    }

    public static void UpdateMesh(Mesh mesh, Vector3[] vertices, int[] triangles, Color[]
colors)
    {
        mesh.Clear();
        // I accidentally use 257*257 vertices instead of 256*256, need to change later
        mesh.vertices = vertices;
        mesh.triangles = triangles;
        mesh.colors = colors;
        mesh.RecalculateNormals();
        mesh.RecalculateTangents();
    }
}

```

Quadtree.cs

```
using UnityEngine;
using System.Collections.Generic;
using System.Collections;
using Unity.Mathematics;
using static Unity.Mathematics.math;
using Unity.VisualScripting;
using System.Security.Cryptography;
using Palmmedia.ReportGenerator.Core;
using UnityEditor;

[RequireComponent(typeof(MeshFilter), typeof(MeshRenderer))]
public class Quadtree: MonoBehaviour
{
    public const float minSize = 256f;
    public Transform viewer;
    Mesh mesh;
    Bounds bounds;
    Quadtree[] children = new Quadtree[4];
    Vector3 centre;
    Vector3 size;
    Vector3 quarterSize;
    int res;
    bool active;
    MeshRenderer meshRenderer;
    string ID;
    Vector3 normalisedCentre;
    Quaternion rotation;
    Material material;

    public void Initialise(int res, Transform viewer, Vector3 position, Quaternion
rotation, string thisID, Vector3 centre, Material material)
    {
        mesh = MeshGen.GenerateMesh(res, thisID, position);
        GetComponent<MeshFilter>().mesh = mesh;
        meshRenderer = GetComponent<MeshRenderer>();
        meshRenderer.material = material;
        bounds = meshRenderer.bounds; //world space coordinates
        //center needs to be changed it is not accounting for the transform's position but
assuming it is at 0,0,0 (reason was because I set the position of all of them to bottom left
corner)
        //centre = new Vector3(bounds.center.x, 0, bounds.center.z) + position;
        Debug.Log("Centre: " + centre);
        this.centre = centre;
        // normalisedCentre is wrong need to change
```

```
        // issue resolved, forgot to *12800
        normalisedCentre = centre.normalized*12800;
        Debug.Log("Normalised Centre: " + normalisedCentre);
        transform.position = position;
        //Debug.Log("Position: " + position);
        // issue 2 the position of the mesh is incorrect -- issue resolved!
        transform.rotation = rotation;
        size = new Vector3(bounds.size.x*Mathf.Sqrt(2), 0, bounds.size.z*Mathf.Sqrt(2)); //
account for the normalised distance difference of mesh on 2d projection
        quarterSize = size / 4;
        this.viewer = viewer;
        this.res = res;
        this.ID = thisID;
        active = true;
        this.rotation = rotation;
        this.material = material;
    }
    private void Update()
    {
        if (active)
        {
            splitQuad();
        }
        else
        {
            regenerate();
        }
    }

    // Update called to check for when to regenerate itself if distance is too far
    // called for disabling the children / setactieve(false) to save on performance
    void regenerate()
    {
        float distance = (viewer.position - normalisedCentre).magnitude;
        if (distance > (size.x))
        {
            meshRenderer.enabled = true;
            active = true;
            for (int i = 0; i < 4; i++)
            {
                children[i].gameObject.SetActive(false);
            }
        }
        // called for disabling the children / setactieve(false) to save on performance
    }


    void splitQuad()
    {
        float distance = (viewer.position - normalisedCentre).magnitude;
        if (distance < (size.x / 4) && size.x > minSize)
        {
            enabled = false;
            if (children[0] is null)
            {
                CreateChildren();
            }
            else
            {
                GetChildren();
            }
            enabled = true;
            active = false;
            meshRenderer.enabled = false;
        }
    }

    void CreateChildren()
    {
        // using standard quadrant labelling to avoid confusion
```

```
111.         Vector3[] centres = new Vector3[]
112.         {
113.             centre + rotation * new Vector3(quarterSize.x, 0, quarterSize.z),
114.             centre + rotation * new Vector3(-quarterSize.x, 0, quarterSize.z),
115.             centre + rotation * new Vector3(-quarterSize.x, 0, -quarterSize.z),
116.             centre + rotation * new Vector3(quarterSize.x, 0, -quarterSize.z),
117.         };
118.
119.         for (int i = 0; i < centres.Length; i++)
120.         {
121.             GameObject childObj = new GameObject("Quadtree" + res.ToString() +
i.ToString());
122.             Quadtree child = childObj.AddComponent<Quadtree>();
123.             child.enabled = false;
124.             child.Initialise(res*2, viewer, transform.position, transform.rotation, ID +
i.ToString(), centres[i], material);
125.             child.enabled = true;
126.             children[i] = child;
127.         }
128.     }
129.
130.     void GetChildren()
131.     {
132.         for (int i = 0; i < 4; i++)
133.         {
134.             children[i].gameObject.SetActive(true);
135.         }
136.     }
137. }
138.
```

Below are some interesting details of the function of the code:

1. I decided to use rotation to generate the cube from 6 faces
2. I used a non-standard recursion technique
    a. Instead of calling a function within the same function in one script, what I did is each ‘quadtree’ mesh is an independent gameObject with the quadtree script attached
        i. Each of it check for itself the threshold distance (which I’ve set as the length of its mesh), if it’s below then it splits to create new children if there’s none existing already.
            1. If there are children, then simply call SetActive(true) for the children.
            2. The idea of saving the reference scripts for each child in an array saves time of regeneration without any substantial tradeoff in performance during its inactive state - SetActive(true) disables the update function for the scripts attached to the gameObject and removes it from rendering, effectively cutting down all cost.
        ii. After the children has been activated, the object disables its meshRenderer but keeps itself active to call regeneration each update to check whether the distance go above the threshold, which then calls the parent to be regenerated and SetActive(false) for its children
    b. The position of the children is calculated with an offset of the parent’s centre to 4 ‘corner centres’ – halfway between corner and centre – then normalised for the right curvature
3. I’ve used a slightly subpar but nevertheless interesting algorithm for mapping noise
    a. As I rotate the 6 faces, the original 6 faces could all face the same direction, hence the vertices calculation is constant
        i. Main feature is to use the origins for each of the quadtrees to track its 2D offset from the centre

ii. The system to find the origin is to introduce a new field which tracks each split and which quadrant the split mesh has belong to, then multiplying by the diminishing factor in scaling for each operation to access its origin location

b. However, the tricky bit as expected was the noise sampling
   i. Basing upon the prototype code, I sampled 6 'maps' basing on the 6 faces, each for their normalised and rotated coordinates. Each of these maps are then projected to their respective quadtree
   ii. Using similar height-based colour generation, using inverseLerp to change the heights into the range of [0,1], to match for the format needed for the gradient sampling.
      1. This uses scriptable object for the values so I can tweak them much easier. One of the pseudo-Earth like planet is shown above.

I can conclude two major improvements needed (and delivered – check latter pages) for this algorithm

1. Using the job system of Unity for both multithreading in each mesh creation
   a. jobHandle tag to pass in as dependencies for a sequential operation so that the freezes on the screen won't happen. This is done as explained in analysis with worker threads so it is offloaded from the main thread where other calculations could take place once the meshCollider is attached. I expect it to be rather easy as I had implemented during prototyping, so I know what to do and common errors to look out for. However, sequentially as laid out in my critical path, it is still development time, and I first need to implement the physics interaction between bodies which I need to further explain in design. The explanation for such system would be more detail below! (Future me editing: I was quite wrong, although it wasn't 'hard' as in conceptually difficult, I am still unfamiliar with how job+burst work. Took me 2 days to realise one line was wrong – Color32 needs a specific data type which I didn't realise).
2. Optimise the noise sampling algorithm which matches with the job + burst system

Gravitation

```
1. using UnityEngine;
 2.
 3. public class UniversallGravitation : MonoBehaviour
 4. {
 5.     CelestialBody[] planets;
 6.     void Start()
 7.     {
 8.         planets = FindObjectsByType<CelestialBody>(FindObjectsSortMode.None);
 9.         Debug.Log("Planets: " + planets.Length);
10.     }
11.
12.     private void FixedUpdate()
13.     {
14.         CalcForce();
15.     }
16.
17.     void CalcForce()
18.     {
19.         foreach (CelestialBody planet in planets)
20.         {
21.             if (planet.CompareTag("Sun"))
22.                 continue;
23.             Vector3 acceleration = Vector3.zero;
24.             foreach (CelestialBody other in planets)
25.             {
```

```
                if (planet == other)
                    continue;
                Vector3 r = other.transform.position - planet.transform.position;
                float r2 = r.sqrMagnitude;
                float a = 6.67430e-11f * other.mass / r2;
                acceleration += a * r.normalized;
            }
            planet.GetComponent<Rigidbody>().AddForce(acceleration, ForceMode.Acceleration);
        }
    }
}

```

```
using UnityEngine;

public class UniversallGravitation : MonoBehaviour
{
    CelestialBody[] planets;
    void Start()
    {
        planets = FindObjectsByType<CelestialBody>(FindObjectsSortMode.None);
        Debug.Log("Planets: " + planets.Length);
    }

    private void FixedUpdate()
    {
        CalcForce();
    }

    void CalcForce()
    {
        foreach (CelestialBody planet in planets)
        {
            if (planet.CompareTag("Sun"))
                continue;
            Vector3 acceleration = Vector3.zero;
            foreach (CelestialBody other in planets)
            {
                if (planet == other)
                    continue;
                Vector3 r = other.transform.position - planet.transform.position;
                float r2 = r.sqrMagnitude;
                float a = 6.67430e-11f * other.mass / r2;
                acceleration += a * r.normalized;
            }
            planet.GetComponent<Rigidbody>().AddForce(acceleration, ForceMode.Acceleration);
        }
    }
}

```

This code provides the first iteration of implementation to double check myself before I plunge into the darkness, it works perfectly fine, so I decide to then plunge into darkness!

As unity has a hard upper limit of rigidBody mass of 1.0e+9 units, it is not possible just to use the original kilogram mass of the planets. Instead, what I opted for is the unit for mass of the earth, referred to as M_E, as the base unit of mass. Similarly, I have changed the units for distance from meters to milli astronomical units (a thousandth of the distance from earth to sun).

As Newton's universal gravitation includes a constant G with units, it also has to be changed to match the changed units. This is simply done using dimensional analysis. The one step multiplication is trivial and is shown below.

$F = \frac{Gm_1m_2}{r^2} \Rightarrow ma = \frac{Gm_1m_2}{r^2} \Rightarrow [kg][m][s]^{-2} = G[kg]^2[m]^{-2} \Rightarrow G = [m]^3[kg]^{-1}[s]^{-2}$

Therefore, the conversion goes:

$$G_{new} = G * (\frac{(milli)au}{m})^3 * (\frac{M_E}{kg})^{-1} * (\frac{s}{s})^{-2} \Rightarrow G_{new} \cong \frac{6.6743015e-11 * 5.9722e24}{149597870.691^3}$$
$$= 1.190594655e-10\ (m)a.u.^3\ {M_E}^{-1}s^{-2}$$

As Unity allows customised frequency for fixed updates, it means I could change to each second updating 60 times, therefore each calculation that is taken to be one second long is 1/60 seconds long, therefore each second in real time corresponds to one minute (at its lowest frequency). I just thought it is a nice number; you could choose any other and it won't matter.

I could also now implement the Runge-Kutta method (RK4 to be precise). However, I quickly realised I don't understand how to implement it the despite some prior analysis on understanding how it works. The main issues are:

1. My lack of knowledge in ODEs
    a. I polished some basics with reading of core pure and further pure textbooks
2. I don't fully understand how state vectors work.
    a. I initially mis-assumed it as only doing it for acceleration but since RK4 only works for first order ODEs, it's necessary to split it up for both velocity and acceleration.
    b. What I had described earlier is an erroneous calculation where I use RK4 for acceleration but Euler for velocity
3. How small does h have to be to avoid chaotic behaviour?
    a. I misunderstood y'=f(x,y)
    b. h always must be less than abs(1) but not sure what unit does '1' must be in.

Hence, I dove a little deeper to understand RK4.

Here below is a simple and rather unsatisfactory derivation (with partial derivatives) for understanding how does RK2 work (2nd order rather than 4th order → which could be generalised to 4th order as the method is the same.) I won't delve into why beyond 4th order it stops being a linear relationship between the number of terms in RK4 and accuracy for truncation error. This is left as an exercise to the reader (if you derive RK4 and RK5 using the same method I used for RK2 you will see why)

Derivation of the RK2 Method from Taylor Series

Taylor series could be represented as follows:

$$x(t) = \sum_{k=0}^{\infty} x^{(k)}(a)\frac{(t-a)^k}{k!}$$

We know that the more terms we add, the more closely it will represent the values of the nearby region. Hence, we should try to match as many terms as possible without computation taking too heavy of a toll. Behind is an explanation which I will credit to a Stackexchange user as it helped me understand what it is and why it is.

Given that an ODE:

$$x'(t) = f\big(t, x(t)\big)$$

We can rewrite this as a Taylor expansion:

$$f(t+\Delta t, x+\Delta x) = f(t,x) + (\Delta t)f_t(t,x) + (\Delta x)f_x(t,x) + \text{higher order terms}$$

We can rewrite this also the ODE as:

$$x(t+h) = x(t) + \int_t^{t+h} f\big(\tau, x(\tau)\big)d\tau$$

However, as we can't find the integral analytically, we can approximate using:

$$x(t+h) \approx x(t) + h\sum_{i=1}^{N} \omega_i f\big(t+\nu_i h, x(t+\nu_i h)\big)$$

Let ($\nu_1 = 0$):

$$K_1 \coloneqq hf\big(t, x(t)\big)$$

Hence:

$$K_2 \coloneqq hf(t+\alpha h, x(t)+\beta K_1)$$

We can therefore write it as:

$$x(t+h) = x(t) + \omega_1 K_1 + \omega_2 K_2$$

To equate this with the Taylor expansion:

$$x(t) + hx'(t) + \frac{h^2}{2!}x''(t) + O(h^3) = x(t) + \omega_1 K_1 + \omega_2 K_2$$

Hence:

$$hf + \frac{h^2}{2}(f_t + ff_x) + O(h^3) = \omega_1 K_1 + \omega_2 K_2$$

Substituting in K values:

$$hf + \frac{h^2}{2}(f_t + ff_x) + O(h^3) = \omega_1 hf + \omega_2 hf(t+\alpha h, x+\beta K_1)$$

Taylor expand the right side:

$$hf + \frac{h^2}{2}(f_t + ff_x) + O(h^3) = \omega_1 hf + \omega_2(hf + \alpha h^2 f_t + \beta h^2 f f_x) + O(h^3)$$

This would give:

$$\omega_1 + \omega_2 = 1$$
$$\alpha\omega_2 = \frac{1}{2}$$
$$\beta\omega_2 = \frac{1}{2}$$

It is canonical to use ($\alpha = \beta = 1$) *and* ($\omega_1 = \omega_2 = \frac{1}{2}$), however there are infinite solutions.

Repeating this process for RK4 (using suitable partial differential with chain rule) would give the result of derivation for RK4 – the values chosen are also canonical as it's the 'nicest'
Further reading: rk_derivation_0.pdf

State Vectors

As mentioned previously, the method could only work for first order differential equations, as acceleration is a second derivative of position, we are solving for second order differential equations. The solution is to use state vectors which separates it into 2 first order equations that are updated with each other (as shown below).

Characteristic Time

Every system has its own characteristic time which determines how far in time (the variable) does it cause the system to undergo huge changes to its displacement and velocity. For the solar system, as it is a collection of near circular orbits, a good approximation can be derived from their time periods assuming they have circular orbits. As Mercury has the shortest time period in the solar system, if it's

stable, then the rest would also follow. With Mercury having a time period roughly ¼ of earth's, it means for an accurate simulation, the h value must be at least 2 orders of magnitude smaller. This means a good time step would be around 6/8 hours for a second in real life, maximising it while keeping it away from chaotic behaviours.

RK4 Pseudocode:

```
PROCEDURE RK4()
    // Initialize arrays for velocity and position coefficients
    DECLARE k1v[celestialBodyCount] as Vector3 array
    DECLARE k1r[celestialBodyCount] as Vector3 array
    DECLARE k2v[celestialBodyCount] as Vector3 array
    DECLARE k2r[celestialBodyCount] as Vector3 array
    DECLARE k3v[celestialBodyCount] as Vector3 array
    DECLARE k3r[celestialBodyCount] as Vector3 array
    DECLARE k4v[celestialBodyCount] as Vector3 array
    DECLARE k4r[celestialBodyCount] as Vector3 array

    // Arrays to store current positions and velocities
    DECLARE currentPositions[celestialBodyCount] as Vector3 array
    DECLARE currentVelocities[celestialBodyCount] as Vector3 array

    // Temporary arrays for intermediate calculations
    DECLARE tempPositions[celestialBodyCount] as Vector3 array
    DECLARE tempVelocities[celestialBodyCount] as Vector3 array

    // Store current positions and velocities
    FOR i = 0 TO celestialBodyCount - 1 DO
        currentPositions[i] = celestialBodies[i].position
        currentVelocities[i] = celestialBodies[i].velocity
    END FOR

    // Calculate k1 coefficients
    FOR i = 0 TO celestialBodyCount - 1 DO
        IF celestialBodies[i] is "Sun" THEN
            CONTINUE
        END IF
        k1v[i] = CalculateAcceleration(currentPositions, i) * timeStep
        k1r[i] = currentVelocities[i] * timeStep
    END FOR

    // Calculate intermediate positions and velocities for k2
    tempPositions = CalculateOffset(currentPositions, k1r, 0.5)
    tempVelocities = CalculateOffset(currentVelocities, k1v, 0.5)

    // Calculate k2 coefficients
    FOR i = 0 TO celestialBodyCount - 1 DO
        IF celestialBodies[i] is "Sun" THEN
            CONTINUE
        END IF
        k2v[i] = CalculateAcceleration(tempPositions, i) * timeStep
        k2r[i] = tempVelocities[i] * timeStep
    END FOR

    // Calculate intermediate positions and velocities for k3
    tempPositions = CalculateOffset(currentPositions, k2r, 0.5)
    tempVelocities = CalculateOffset(currentVelocities, k2v, 0.5)

    // Calculate k3 coefficients
    FOR i = 0 TO celestialBodyCount - 1 DO
        IF celestialBodies[i] is "Sun" THEN
            CONTINUE
        END IF
        k3v[i] = CalculateAcceleration(tempPositions, i) * timeStep
        k3r[i] = tempVelocities[i] * timeStep
    END FOR

    // Calculate intermediate positions and velocities for k4
    tempPositions = CalculateOffset(currentPositions, k3r, 1.0)
```

```
63.     tempVelocities = CalculateOffset(currentVelocities, k3v, 1.0)
64.
65.     // Calculate k4 coefficients
66.     FOR i = 0 TO celestialBodyCount - 1 DO
67.         IF celestialBodies[i] is "Sun" THEN
68.             CONTINUE
69.         END IF
70.         k4v[i] = CalculateAcceleration(tempPositions, i) * timeStep
71.         k4r[i] = tempVelocities[i] * timeStep
72.     END FOR
73.
74.     // Update velocities and positions using weighted average of coefficients
75.     FOR i = 0 TO celestialBodyCount - 1 DO
76.         celestialBodies[i].velocity += (k1v[i] + 2.0 * k2v[i] + 2.0 * k3v[i] + k4v[i]) / 6.0
77.         celestialBodies[i].position += (k1r[i] + 2.0 * k2r[i] + 2.0 * k3r[i] + k4r[i]) / 6.0
78.     END FOR
79. END PROCEDURE
80.
```

Mid-project Client Feedback:

As the project hits midway through development I asked Tommy to reflect on objectives and directions which it is heading. One decision which we decided to agree on is the removal of the extension in voxel technique, the main reason is that it doesn't align with the current vision of the game, as a realistic solar system simulation which the user could explore, terraforming planets just doesn't seem right in this scope!

Rocket Physics

Unlike aircrafts, travelling in space outside of atmosphere means there isn't air to provide lift or gliding, hence wings (and aerodynamic designs) are useless – bar the takeoff and landing (if the landing celestial body has atmosphere). The absence of an ever-present gravitational pull downwards meant 1. Direction doesn't matter – ie there's no set up or down. 2. Only controls are given by thrusters – critically due to Newtonian laws, this will exert an opposite but equal force (NIII) that causes proportional acceleration in the direction (NII) but if there's no force there would be no change in velocity, therefore staying at a constant velocity (NI).

Luckily that this rather easy to be implemented with Unity's default physics engine using the AddForce with ForceMode.Acceleration. Similarly with rotation, if a force is not applied onto its centre of mass, then there's a torque applied. If only rotational movement is desired, then you need to apply a couple. Luckily again, Unity offers Quaternions with method of AngleAxis, which avoids complicated calculations for forces in controlling rotation but achieving the same effect.

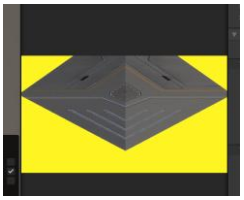
One option I had considered is for the player to directly control each of the engines to produce a force and calculate how much is needed for each to achieve the desired rotation + translation, but this made gameplay quite hard and rather annoying, so I decide to give a standard control – mouse movements for up, down, left and right rotation. Classic WASD for movements and the arrow keys for roll. However, it still abides by the Newtonian laws in vacuum and it's good enough for realism.

Spaceship View

Using 3rd person camera give the smallest possible linear scale to be 1e-4, beyond that it gives unrecognisable graphics that doesn't resemble the spaceship. This means that the spaceship in metric

units would be 10km long, which is far from realistic, hence realism in this regard could be scrapped. Instead, to avoid the player seeing the unrealistic scale, the camera is mounted inside the spaceship.

Normal shot vs

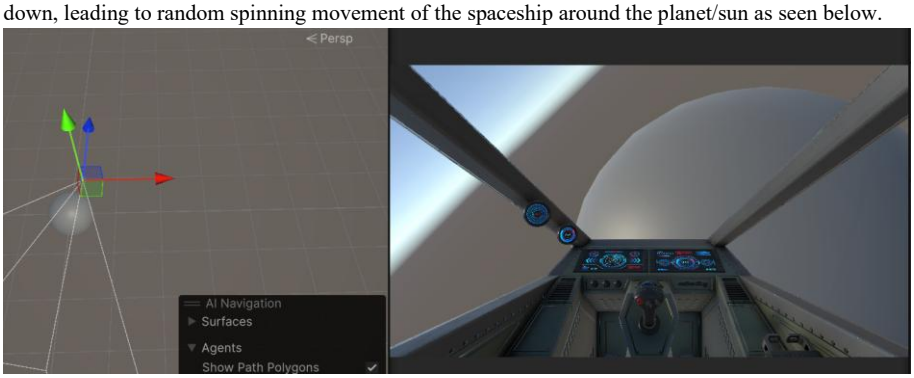


??? Completely distorted at the scale of 1e-5

Collision Handling

On paper, this should be trivial by using the built-in collision system for rigidBody in Unity. However, due to my usage of RK4 instead of using its default AddForce for calculating gravitational acceleration, it turns out to be much more annoying to be compatible with both systems. For this section, it makes most sense to document my design/coding journey to demonstrate iterative design and make the reasoning more clear in why I chose what I did at the end!

The pictures demonstrate two frames after a spaceship collides with a sphere that mimicks a celestial body. Initially it behaves fine (see above pic). But after a very short time the collision system breaks down, leading to random spinning movement of the spaceship around the planet/sun as seen below.



I initially thought it was caused by the minute acceleration of the gravitational attraction of other planets (which are moving in respect to the position of the planet I am on), causing a slight acceleration and velocity not pointing directly towards the centre.

This is probably then facilitated by the enormous acceleration the spaceship is experimenting that exaggerates the inaccuracies of RK4 method with the chosen time step as its characteristic time is likely much smaller, leading to chaos of random movements. However, decreasing the time step is not a solution as I needed the planets to orbit in the same (or similar rates).

To solve this issue, I first tried to implement a surface friction. Unfortunately, it failed, the physics material which I attached to the object has no effect on resolving any chaos.

To figure out the culprit to chaos, I 1. Changed the planet to just a sphere – it has a uniform distribution of mass to avoid any off-centre torque 2. lowered the mass of Sun temporarily to 300 units which reduces the acceleration and hence plays a 'slow-mo' effect on how the chaos occur.

Although my initial suspicion was correct (causing torque from off centred velocity), I realised it was only part of the issue. As I calculate gravitational attraction using RK4, I use ForceMode.VelocityChange for AddForce method (this is for greater accuracy – technically not the best method as I should directly update the position from the RK4 calculation, but I need to use AddForce instead of MovePosition – necessary due to two main issues: 1. collision detection issue with rb.MovePosition as discrete 2. it doesn't factor in the existence of thrusters and rollers and how they also exert a force (hence an acceleration) on the body.) Yet, Unity's default physics engine only automatic generation of normal force if my mode is on ForceMode.Acceleration and ForceMode.Force. This means that acceleration calculation with other bodies must be stopped once contact is mad. Hence, a stack instead of an array must be used for storing celestialBodies for the RK4

Floating Origin System

As Unity's transforms' positions uses Vector3, each value is a float and has its floating-point limitation – about 7 significant figures in precision. This means that anything above 10000 would start to experience precision error, causing 'glitches' in rendering or movement. This is a big issue for me as I need the spaceship to fly around the magnitude of those distances. One easy but effective method to bypass such issue is to use a floating origin. This tracks the spaceship's position, and if it exceeds a threshold, everything in the scene would be offsetted so the spaceship is back to the origin. Although this means planets far away would experience glitches, it doesn't matter as I can barely see them from the spaceship camera anyways. What is crucial is that the spaceship is near the centre so there's no visible defect as it is where the camera is at.

A related issue would be of depth buffer of the camera, which can also only go up to around 10000 (slightly higher but in this order of magnitude). This means the further away planets are completely not rendered. A solution to fix this is to use a log buffer, giving more even distribution to quality of render between close and far away object and its implementation is not too complicated. But on revaluation, since 1. I can barely see the planets far away anyways 2. Those planets are likely to be filled with visual defect artefacts, I have decided against this approach and instead stick with the normal z buffer, which has much more precision bits for closer objects, which is possibly more important for my game graphics.

Dynamic Scaling

With the planetary system running at a scale of one unit as one milli AU, if I was to maintain an accurate scale of celestial bodies, they would be minute, you can't see anything in the sky at all and the game is just trash. Instead, I've adopted a slightly enlarged scales of the planets which are still relative to each other scaled correctly, with the Earth approximated by a sphere of radius length of 5 game units. However, this is still way too small for a planet that at least feel realistic like flat ground on a planet rather than walking on a spherical object (main bit of Outer Wilds which I hate, the planets are way too small). Therefore, a suitable size would be where after touchdown on Earth, the horizon could be seen as horizontal with no significant curvature to the side. However, as previously explained, the floating-point limitation in Unity means I couldn't just have massive planets with realistic scales as they all must be within range of 10000 ish, instead, a dynamic scaling must be used.

I had initially thought of 2 options.

1. Consider three scales, one for inner solar system, one for outer solar system and one for planetary system. However, the game wouldn't be much fun if you can only see one planet (or no planets at all – distance between Uranus and Neptune is 10.8 AU, if my scale is any smaller than milli AU per unit length then the numbers would be well over 10000, meaning I can't even spot Neptune from Uranus due to floating-point error, a total disaster!).
2. Another is directly ignoring the scale issue, use the scale as it is then amplifying the scale of planets when closer to planets, adjusting the speed at the same time to balance it off. However, this means there is a strict limitation of its scale based on how far initially the distance of the split is. (As the planet is static). This means that it is likely not very useful as I cannot make the threshold large due to the inherent distance between planets in the system.

While both doesn't make much sense, what I realised is that combing them does make sense.

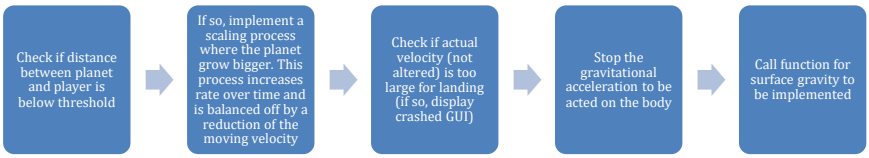
Consider the case where the whole system's scale change (from milli-AU to micro-AU for example, where everything would be enlarged by 1000), this would both have a smooth change in scale as well as avoiding the issue of the planet hitting the player (as discussed in 2nd point) if I need to make the scale larger.

Here, I think an initial split of threshold 100, then the scale is multiplied by 4 and the threshold multiplied by 2, with a cap of 2550, which is 10 times the size of one mesh. As each mesh has the dimension of 255*255. As I had set the Earth to have an initial scale of 10*10*10, it means effectively at the highest resolution I will have 100*100*100 meshes that will give the sweet horizontal horizon when viewed from the planet. The specific number of 4 in scale and 2 in threshold is that you always need to make half of the new scaled distance for a new split, just a nice number to use.

Surface Gravity

Due to the AddForce mode in Unity and its physics engine, it is needed to use the acceleration change to account for normal force when moving, hence an alternative acceleration must be used instead of calculating more accurately with the tweaking of velocities. The direction of such force could be easily found with a Vector subtraction of player's and planet's position, with the opposite being set to the player's orientation so it always faces 'up'. The magnitude of such calculation is then done by scaling the mass of the planet.

Here is a basic initial flowchart for the process of landing:



Player Movement

Trivial linear movement in two directions same as spaceship with wasd keys, with two speeds, walking or running. There are two rotational movements, one for camera and one for player. The Camera's restriction would be like how a normal person can turn their head in both axes. The player rotation would be limited to a 2D rotation alongside the tangential plane as I had fixed the tangential plane with the previous setting player's orientation as opposite to the direction of gravitational force.

Mesh Generation Optimisation + New Noise Sampling

With most of the gravitation and spaceship manoeuvring finished, I hopped back to my code for the mesh generation for each quadtree. Upon reading it, I had the very urge to throw up at how bad it was. Hence, I began my crusade to save my sanity with the Unity job+burst system to make it the logic 'logicable'.

The main issue with the noise sampling that I found was its redundancy. I practically did double the amount of work (doubling the times I need to calculate for each position of the vertices). It would be much smarter if I sample each of the vertex individually rather than sampling from a map in this case.

Here's a pseudocode improvement to clarify its logic:

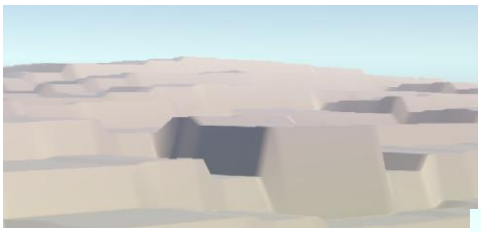
```
 1. FUNCTION ImprovedMeshGen(integer z)
 2.     // Initialize indices
 3.     SET vertexIndex = (resolution + 1) * z
 4.     SET triangleIndex = 2 * resolution * z
 5.     SET halfLength = resolution * settings.meshScale / 2
 6.
 7.     SET sampleZ = ((z / resolution - 0.5) * resolution * settings.meshScale / res) +
origin.y
 8.
 9.     FOR x = 0 TO resolution
10.         // Calculate positions
11.         SET sampleX = ((x / resolution - 0.5) * resolution * settings.meshScale / res) +
origin.x
12.         SET basePos = Vector3(sampleX, halfLength, sampleZ)
13.
14.         SET normalizedPos = normalize(basePos) * halfLength
15.         SET rotatedPos = multiply(meshRotation, normalizedPos)
16.
17.         // Calculate noise
18.         DECLARE derivatives as Vector2
19.         SET noiseHeight = NoiseGen.CalculateNoiseWithDerivatives(rotatedPos, settings,
OUTPUT derivatives)
20.
21.         SET inverseHeight = clamp((noiseHeight + settings.maxHeight * 0.7) / (2 *
settings.maxHeight * 0.7), 0, 1)
```

```
22.         SET finalPos = normalize(rotatedPos) * (127.5 * settings.meshScale +
settings.amplitude * noiseHeight)
23.
24.         // Create vertex
25.         CREATE vertex WITH
26.             position = finalPos,
27.             color = settings.SampleGradient(inverseHeight)
28.
29.         meshGen.SetVertex(vertexIndex, vertex)
30.
31.         // Generate triangles if not at edge
32.         IF z < resolution AND x < resolution THEN
33.             SET triangle1 = Vector3Int(
34.                 vertexIndex,
35.                 vertexIndex + resolution + 1,
36.                 vertexIndex + 1
37.             )
38.
39.             SET triangle2 = Vector3Int(
40.                 vertexIndex + 1,
41.                 vertexIndex + resolution + 1,
42.                 vertexIndex + resolution + 2
43.             )
44.
45.             meshGen.SetTriangle(triangleIndex, triangle1)
46.             INCREMENT triangleIndex
47.
48.             meshGen.SetTriangle(triangleIndex, triangle2)
49.             INCREMENT triangleIndex
50.         END IF
51.
52.         INCREMENT vertexIndex
53.     END FOR
54. END FUNCTION
55.
56. FUNCTION CalculateNoiseWithDerivatives(Vector3 position, NoiseSettingsData settings, OUTPUT
Vector2 derivatives)
57.     SET noiseHeight = 0
58.     SET amplitude = 1
59.     SET frequency = settings.frequency
60.     SET derivatives = Vector2(0, 0)
61.
62.     FOR i = 0 TO settings.octaveCount - 1
63.         SET samplePos = (position / settings.scale + settings.octaveOffsets[i]) * frequency
64.
65.         DECLARE gradient as Vector3
66.         SET noise = snoise(samplePos, OUTPUT gradient)
67.         ADD noise * amplitude TO noiseHeight
68.
69.         ADD Vector2(gradient.x, gradient.z) * amplitude * frequency / settings.scale TO
derivatives
70.
71.         MULTIPLY amplitude BY settings.persistance
72.         MULTIPLY frequency BY settings.lacunarity
73.     END FOR
74.
75.     RETURN noiseHeight
76. END FUNCTION
77.
```

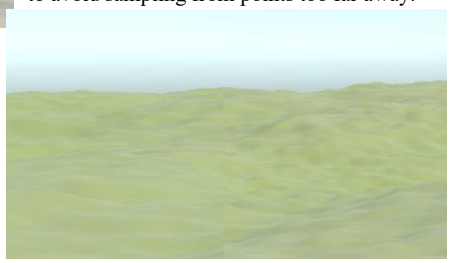
The Unity Burst system is possibly the greatest thing developers had developed, unlike their physics system. It auto-compile your code to the most optimal state with the tradeoff of not able to understand what went wrong if something did go wrong. It was designed to slot perfectly with the Job system which streamlines the process of scheduling and multithreading from manually setting up worker threads. I think it is self-explanatory why this will reduce the freezing happening when new meshes are generated (all are clogged in the main thread but now could be processed in parallel). Scheduling

makes sure the ones closer to the spaceship would process first using the jobHandle and scheduleParallel in jobs. With the previous explanation of the use of structs instead of class because of value type instead of reference type, the rest of the job is just about following the syntax for job systems, such as using Unity.mathematics instead of Mathf.

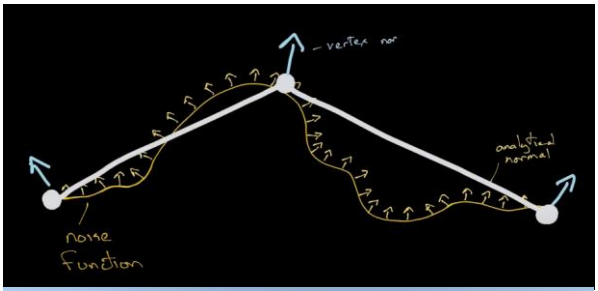
To further optimise, Unity offers an advanced mesh system that avoids the overheads with writing to the meshes by using native arrays for vertices and triangles, as well as colours that is optimised using the GPU. This method is also by default optimised with job+burst so it should give the best result.

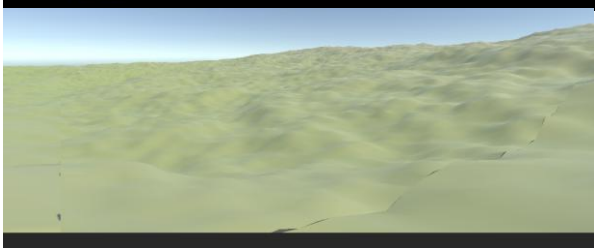


An issue that I bumped into is as I ramped up the details for the noise, it began to break down. These square-y artefacts are due to the floating point accuracy (again!) of the Unity.mathematics.noise library for snoise. To resolve it, it means the offset must be limited to avoid sampling from points too far away.



With the issue resolved, I realised some of the meshes does not match up, with the colouring being non-coherent between meshes. I realised this was due to the normals and tangents not constructing properly between neighboring meshes, and hence an analytical function is needed to calculate the normal so they could match up. (Simon Dev from YouTube has a good visual explanation which I borrow here below)

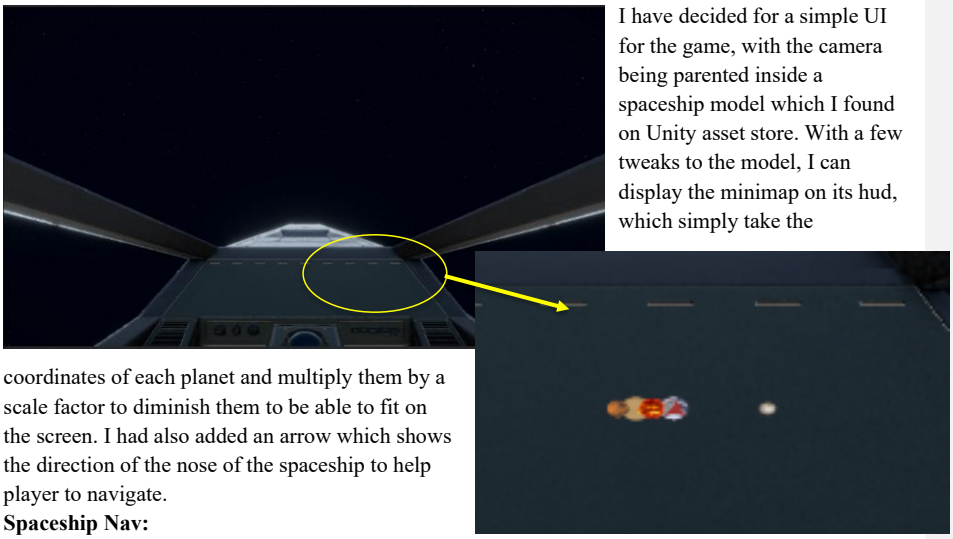




There is an extra issue of mesh seams between meshes of varying resolution, however this is insignificant as only the threshold of the split quad function of our quadtree ensures that surrounding the player would always be at least 4 meshes of the highest resolution (smallest size that I had set to be possible for the splitting), hence from the game perspective, there would be no chance in which these seams could be observed or affect a player's traversal.

UI/UX:

Spaceship HUD:



I have decided for a simple UI for the game, with the camera being parented inside a spaceship model which I found on Unity asset store. With a few tweaks to the model, I can display the minimap on its hud, which simply take the coordinates of each planet and multiply them by a scale factor to diminish them to be able to fit on the screen. I had also added an arrow which shows the direction of the nose of the spaceship to help player to navigate.

Spaceship Nav:

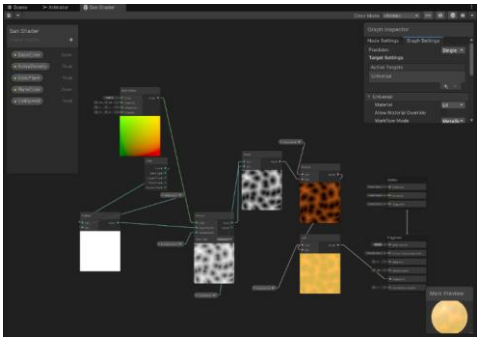
The navigation system is quite self-explantory with the usual keys WASD representing a thrust in the forward, left, back and right direction. The arrow keys are used for up/down movement as well as roll movements with left/right arrow keys. Other rotations (up/down/left/right) are controlled by the mouse movements. I have avoided implemented it as torque for rotation as I personally found it too hard to control the spaceship and instead it just moves to that rotation with smoothing applied. I have also decided for the key 'Z' to control time step so the player can increase/decrease the rate of simulation.

Planet Nav + UI:

When the player landed on a planet, they could switch to 'player mode' with the button 'B'. Here I parented a camera onto a cuboid which mimics a perspective from a person's eyes, making it a first-person view. The cuboid tracks the direction from it to the centre of the planet to ensure it is always 'upright' and disable rotation for rigidBody physics for smoother controls. The controls are common ones with spacebar as jump and WASD for movements. The rotation is the same as spaceships with no rolls, alongside with a constraint on up/down rotation so mimic a person's view.

Sun:

2 main purposes: generating correct lighting + accuracy in mimicking the actual sun. Here I follow a tutorial as I am not as familiar with shader graphs or shaders which is needed for generating post processing effect. So, this is MOSTLY NOT my work. However, it still begs for a brief explanation.



Using Voronoi noise that has its offset controlled by time mimics the glare movements. The shear applied further distorts the noise, which alongside a power function gives rise to an effect simulating the glaring magnetic fields of our sun. I have exaggerated the glare and lighting for better lighting in game.

Implementation

Specific Techniques

File	Starting Line	Technique	Description	Group
Quadtree.cs	38, 167	Queue/Scheduling	This queues the jobs which each generate one chunk of mesh through multithreading. The data types used are jobHandles and meshDataArray to be compatible with the job+burst system. The queue is locked everytime when one instance of a quadtree tries to gain access to it to ensure the ordering integrity is maintained. The queue is a stack type and the appending jobHandles are sorted linearly by closest distance for an ordered append. However, priority queue is not implemented here. The job system automatically schedules the jobHandles and once it is it's turn to dequeue, it calls complete.	A
UniversalGravitation.cs	12, 380	Stack	I've changed too many times between a stack and an array for this purpose (for an array resize). The sole purpose is to pop and push the spaceship back into celestialBody array for RK4 calculations when it landed on planet and when it is leaving the planet. As discussed in design, the reason here is due to the manual velocity update which will cause collision detection issues if it's not removed from RK4 calculations	A
Quadtree.cs	102,118	Pseudo-Recursion/ Self-implemented algo	As discussed in design, detect minimum threshold → break into more detailed quadtree (calculation of their centres) or vice versa (fall back to parent). Pseudo-recursive as it is not function calling itself, but rather a function generating a new instance of the same class which then calls that function of that class with a base case that is a static field. (Doesn't use a call stack like in traditional recursion). Reason is explained somewhere else	A
Quadtree.cs Main.cs	144 30	Dynamic OOP	Create quadtree children's instances from the main script which is attached to each individual planet. (Composition). Each planet also has a NoiseSetting componenet (ScriptableObject class). Each quadtree then	A

			will have a NoiseSettings struct which is itself an immutable 'instance' of a NoiseSetting.	
CelestialBody.cs rbBody.cs UniversalGravitation.cs ChunkGen.cs	N/A N/A 418 N/A		Each object is a celestialBody, inheriting from the base class of MonoBehaviour. Spaceship and player are 'rbBody's – celestial bodies with a rigidBody attached (inherited from celestialBody). Main reason is that I can access the rigidBody of a 'trackbody' which could either be player or spaceship depending on game state for both quadtree threshold calculations and floating origin calculations. The usage of job system uses inheritance from IJobFor, which is itself an interface.	
ChunkGen.cs AdvancedMeshGenSetUp.cs	Whole script Whole script	Multithreading	Job+Burst to multithread each mesh of quadtrees. For full detail please refer to other places	A
AdvancedMeshGenSetUp.cs AdvancedMeshGen.cs NoiseSettings.cs	11 Self-evident 78	Native array/ advanced mesh Unity operations (optimisations)	Native arrays work best with advanced mesh system as it has much quicker write speed with defined type and memory allocated for each. Native arrays are necessary to avoid accessing the same memory during multithreading.	A
UniversalGravitation.cs	251	RK4 manual physics	Uses double instead of float. Stores planets' velocities and positions as arrays and perform 4 step calcAcc on it before updating their positions and velocities.	A
NoiseGen.cs	Whole script	fBm	See explanation somewhere else	A
ChunkGen.cs	Self-evident	Sphere Generation	See explanation somewhere else	A
GeneralStruct.cs	Whole script	Self-defined structs (data clusters)	Data clarity. Restraining order in vertex data and triangle data to allow for 'unsafe' processing which speeds up the mesh generation. There's an implicit data type conversion for triangle to safe space (using short)	A
NoiseSettings.cs	78	Usage of immutable types (struct in Unity)	Convert all the stuff in NoiseSettings to a struct for multithreading	A
PlayerController.cs Spaceship.cs	92 118	Quaternion calculations	I sometimes forgot that quaternions are non-commutable and screwed up my rotations. Smoothing is also applied (slerp) which takes in a normalised quaternion. Most of the nitty gritty is dealt by Unity but the values, order of operation and smoothing is still controlled by me.	A
UniversalGravitation.cs	111	Scale shift algo	See explanation somewhere else	A
Spaceship.cs	223	Recursion	This is to disable the regeneration function for all parent quadtrees of the quadtree which the spaceship landed on to avoid null references	A

Coding Styles

- Each individual function is split into their respective subroutines.
- Each class is separated into its own file
- Limit each class (other than UniversalGravitation which acts as an overarching class) to maximum of 200 words and split the classes as much as possible to use composition for each object
- Each function is based upon functions that is only fully dependable on it
    - Partial or non-dependencies are delegated to static helper classes with each as a static function within the class
- Using constants to define a field like G to avoid accidental changes
- Use static fields to maintain data across all instances (such as quadtree)
- Use private and public fields (some public with setters and getters) to protect integrity
- Parameters of each class/function is clearly defined with return type being specified
- I avoid using var unless it's obvious what type the data is
- For cross script communication (non-static) I used events which functions in other scripts subscribe to rather than calling that function itself
    - Each call is checked for null before calling to avoid errors
- For objects involved in multithreading with unsafe low level, there's always a dispose function being called to avoid memory leaks
    - Implemented Try… Catch…finally loop in AdvancedMeshGen.cs for exception handling
- Deactivate the objects not in scene and disable to avoid memory leaks (the first completely ignores it – not rendering for example - while the second avoids calling updates only)
- Ensures the right method is called
    - Awake calls before OnEnable before Start
    - Update for normal interactions, FixedUpdate for physics calculations
        - I avoided using LateUpdate for floating origin as for some reasons it doesn't match with either Update or FixedUpdate timings and screwed up the scaling
- Variables names are named intuitively without repeating names
- Avoid null references in quadtrees by disabling all of the parents of the quadtree spaceship landed on

Code

For the code below, I have intentionally left in all the Debug statements in comments to demonstrate my iterative testing and problem solving during the coding process. I had removed all unnecessary comments (ie remarks I made to myself during the coding process about errors and ideas of what might be wrong as they heavily disrupt the flow).

Instead, I added comments to explain certain parts of the code which I find slightly unintuitive to help the reader understand better what exactly that part does. As it is in Unity, all the following are in in C#.

AdvancedMeshGen.cs

```
using Unity.Jobs;
using UnityEngine;
using Unity.Mathematics;

public static class AdvancedMeshGen
{
    public static (JobHandle, Mesh.MeshDataArray, NoiseSettingsData) GenerateHandle(string ID, Quaternion rotation, float res, NoiseSettings settings, JobHandle lastHandle)
    {
        var meshDataArray = Mesh.AllocateWritableMeshData(1); // using advanced mesh API to access and directly write mesh data to improve processing overheads
        var jobSettings = settings.CreateJobFriendlySettings();

        var meshData = meshDataArray[0]; // this is where we write our mesh data to

        string verticesID = ID.Substring(1);
        float scale = 0.5f;
        float2 origin = float2.zero;
        float quarterLength = 255 * settings.meshScale/4;

        for (int i = 0; i < verticesID.Length; i++)
        {
            scale = math.exp2(i);
            switch (verticesID[i])
            {
                case '0':
                    origin += new float2(1,1) * quarterLength / scale;
                    break;
                case '1':
                    origin += new float2(-1, 1) * quarterLength / scale;
                    break;
                case '2':
                    origin += new float2(-1,-1) * quarterLength / scale;
                    break;
                case '3':
                    origin += new float2(1, -1) * quarterLength / scale;
                    break;
            }
        }
        // calculation for getting the un-normalised origin of the mesh (position it would've been on the cube) for mesh generation

        JobHandle handle = ChunkGen.ScheduleParallel(
            meshData,
            jobSettings,
            origin,
            rotation,
            res,
            lastHandle
        );
        // this allows for multithreading (on CPU with jobs)
        return (handle, meshDataArray, jobSettings);
    }

    public static Mesh CompleteMeshGeneration(JobHandle handle, Mesh.MeshDataArray meshDataArray, NoiseSettingsData jobSettings)
    {
        Mesh mesh = new Mesh();
        try
        {
            handle.Complete();

            // Only apply the mesh data if we haven't hit an exception
            Mesh.ApplyAndDisposeWritableMeshData(meshDataArray, mesh);

            // Use the default Unity calculations - could be improved with my own algorithm but it is not too expensive
```

```
            mesh.RecalculateBounds();
            mesh.RecalculateNormals();
            mesh.RecalculateTangents();

            return mesh;
        }
        catch (System.Exception e)
        {
            Debug.LogError($"Error completing mesh generation: {e}");

            // If we created a mesh but failed, destroy it
            if (mesh != null)
            {
                if (Application.isEditor)
                {
                    Object.DestroyImmediate(mesh);
                }
                else
                {
                    Object.Destroy(mesh);
                }
            }

            // Dispose the mesh data without trying to apply it
            meshDataArray.Dispose();
            throw;
        }
        finally
        {
            // Safety to avoid memory leakds
            jobSettings.Dispose();
        }
    }
}

```

AdvancedMeshGenSetUp.cs

```
using UnityEngine;
using Unity.Mathematics;
using Unity.Collections;
using UnityEngine.Rendering;
using Unity.Collections.LowLevel.Unsafe;

public struct AdvancedMeshGenSetup
{
    // disbaled safety checks for performance
    [NativeDisableContainerSafetyRestriction]
    private NativeArray<Vertex> vertices;
    [NativeDisableContainerSafetyRestriction]
    private NativeArray<TriangleUInt16> triangles;

    public void Setup(Mesh.MeshData meshData, int vertexCount, int indexCount)
    {
        // Create vertex buffer that ChunkGen can write to
        NativeArray<VertexAttributeDescriptor> meshDataDescriptor = new
NativeArray<VertexAttributeDescriptor>(2, Allocator.Temp,
NativeArrayOptions.UninitializedMemory); // temporary array to hold vertex attributes for mesh
data creation - disposed after use pretty much immediately so no point in making it permenant
(safe memory and is quicker)
        meshDataDescriptor[0] = new VertexAttributeDescriptor(VertexAttribute.Position,
dimension: 3); // Vector3, hence dimension:3
        meshDataDescriptor[1] = new VertexAttributeDescriptor(VertexAttribute.Color,
VertexAttributeFormat.UNorm8,  dimension: 4); // UNorm8 is 0-1 range - uses Color32 struct hence
dimension 4 and 8 bits per channel

        meshData.SetVertexBufferParams(vertexCount, meshDataDescriptor);
```

```
        meshDataDescriptor.Dispose(); // dispose of the temporary array after setting up meshData

        meshData.SetIndexBufferParams(indexCount, IndexFormat.UInt16);

        meshData.subMeshCount = 1;
        meshData.SetSubMesh(0, new SubMeshDescriptor(0, indexCount) // means one submesh (submeshes in Unity are used usually for different textures - since I only use colours there's no point for having multiple submeshes)
        {
            vertexCount = vertexCount
        },
        MeshUpdateFlags.DontRecalculateBounds | MeshUpdateFlags.DontValidateIndices); // these MeshUpdateFlags disable checks to improve performance

        vertices = meshData.GetVertexData<Vertex>();
        triangles = meshData.GetIndexData<ushort>().Reinterpret<TriangleUInt16>(2); // short is 2 bytes instead of 4 to save memory, there's no need for 4 bytes for indices

        //Debug.Log($"Vertex buffer created with size: {vertices.Length}");
        //Debug.Log($"Vertex struct size: {System.Runtime.InteropServices.Marshal.SizeOf<Vertex>()}");

        //// Verify first few vertices have correct layout
        //unsafe
        //{
        //    var ptr = vertices.GetUnsafePtr();
        //    Debug.Log($"First vertex color offset: {(int)(&((Vertex*)ptr)->color) - (int)ptr}");
        //}
    }

    public void SetVertex(int index, Vertex vertex)
    {
        vertices[index] = new Vertex
        {
            position = vertex.position,
            color = vertex.color
        };
        //Debug.Log($"vertex color: {vertices[index].color}");
    }

    public void SetTriangle(int index, int3 triangle)
    {
        triangles[index] = triangle;
    }
}

```

CelestialBody.cs

```
using UnityEngine;
using Unity.Mathematics;

[ExecuteInEditMode]
public class CelestialBody : MonoBehaviour
{
    public double3 vel;
    public double3 pos;

    public float tiltAngle;
    public float rotationSpeed;

    public Vector3 initialLocalScale;

    public float mass;
    public float radius { get; set; } = 0.5f;

```

```
18.     public void Awake()
19.     {
20.         //Debug.Log("transfrom.position: " + transform.position);
21.         pos = new double3(transform.position.x, transform.position.y, transform.position.z);
22.         initialLocalScale = transform.localScale;
23.     }
24. }
25.
```

ChunkGen.cs

```
  1. using UnityEngine;
  2. using Unity.Jobs;
  3. using Unity.Collections;
  4. using Unity.Burst;
  5. using Unity.Mathematics;
  6. using static Unity.Mathematics.math;
  7.
  8. [BurstCompile(FloatPrecision.Standard, FloatMode.Fast, CompileSynchronously = true)]
  9. public struct ChunkGen : IJobFor
 10. {
 11.     [WriteOnly] AdvancedMeshGenSetup meshGen;
 12.     [ReadOnly] NoiseSettingsData settings;
 13.     // Adding these WriteOnly and ReadOnly tags improve performance by allowing the Burst
compiler to optimize the code (also safety)
 14.
 15.     public float2 origin;
 16.     public quaternion meshRotation;
 17.     public float res;
 18.
 19.     public int resolution;
 20.     //as resolution is the number of gaps between vertices on each row/column, the number
of vertices on each row/column is resolution + 1
 21.     public int vertexCount => (resolution + 1) * (resolution + 1);
 22.     public int indexCount => resolution * resolution * 6;
 23.     //need 6 indices per square (2 triangles per square, 3 indices per triangle) - this is
counted from the bottom left vertex of each square so no need for the final row/column
 24.     public int jobLength => resolution + 1;
 25.
 26.     public void Execute(int z)
 27.     {
 28.         // z is incremented by the job system - similar to a for loop but each loop is a
job and a seperate thread - it represents the row number (hence used z for depth)
 29.         int vertexIndex = (resolution + 1) * z;
 30.         int triangleIndex = 2 * resolution * z;
 31.         float halfLength = resolution * settings.meshScale /2f;
 32.
 33.         //Debug.Log($"z : {z}");
 34.
 35.         float sampleZ = ((float)z / resolution - 0.5f) * resolution *
settings.meshScale/res + origin.y;
 36.         // this places the sample point at the centre of the row of squares
 37.
 38.         //Debug.Log($"sampleZ : {sampleZ}");
 39.
 40.         for (float x = 0; x <= resolution; x++)
 41.         {
 42.             // the
 43.
 44.             float sampleX = (x / resolution - 0.5f) * resolution * settings.meshScale / res
+ origin.x;
 45.             // this places the sample point to the centre of each individual sqaure
 46.
 47.             float3 basePos = float3(sampleX, halfLength, sampleZ);
 48.             // this represent the point coordinate of the top side of the cube
 49.
 50.             //Debug.Log($"bassePos {basePos}");
 51.             float3 normalizedPos = normalize(basePos) * halfLength;
```

```
            // gets its sphere coordinates

            float3 rotatedPos = mul(meshRotation, normalizedPos);

            float noiseHeight = NoiseGen.CalculateNoiseWithDerivatives(rotatedPos,
settings);
            //Debug.Log($"noiseHeight : {noiseHeight}");
            //Debug.Log($"settings maxHeight : {settings.maxHeight}");
            float inverseHeight = clamp((noiseHeight +
settings.maxHeight*0.7f)/(2f*settings.maxHeight*0.7f),0f,1f);
            //Debug.Log($"Inverse height: {inverseHeight}");
            float3 finalPos = normalize(rotatedPos) * (127.5f*settings.meshScale +
settings.amplitude * noiseHeight);

            // Create vertex
            var vertex = new Vertex
            {
                position = finalPos,
                color = settings.SampleGradient(inverseHeight) // Store slope information
in color
            };

            meshGen.SetVertex(vertexIndex, vertex);

            // Generate triangles
            if (z < resolution && x < resolution)
            {
                int3 triangle1 = int3(
                    vertexIndex,
                    vertexIndex + resolution + 1,
                    vertexIndex + 1
                );

                int3 triangle2 = int3(
                    vertexIndex + 1,
                    vertexIndex + resolution + 1,
                    vertexIndex + resolution + 2
                );
                //Debug.Log($"triangle index 1 : {triangleIndex}");
                meshGen.SetTriangle(triangleIndex++, triangle1);
                //Debug.Log($"triangle index 2 : {triangleIndex}");
                meshGen.SetTriangle(triangleIndex++, triangle2);
            }

            vertexIndex++;
        }
    }

    public static JobHandle ScheduleParallel(
        Mesh.MeshData meshData,
        NoiseSettingsData settings,
        float2 origin,
        quaternion meshRotation,
        float res,
        JobHandle dependency,
        int resolution = 255)
    {
        var job = new ChunkGen
        {
            meshGen = new AdvancedMeshGenSetup(),
            settings = settings,
            origin = origin,
            meshRotation = meshRotation,
            res = res,
            resolution = resolution
        };

        job.meshGen.Setup(meshData, job.vertexCount, job.indexCount);

        var handle = job.ScheduleParallel(job.jobLength, 1, dependency);
```

```
118.
119.         return handle;
120.     }
121. }
122.
```

GeneralStruct.cs

```
 1. using System.Runtime.InteropServices;
 2. using Unity.Mathematics;
 3. using UnityEngine;
 4.
 5. [StructLayout(LayoutKind.Sequential)]
 6. public struct Vertex
 7. {
 8.     public float3 position;
 9.     public Color32 color;
10. }
11.
12. [StructLayout(LayoutKind.Sequential)]
13. public struct TriangleUInt16
14. {
15.     public ushort a, b, c;
16.     public static implicit operator TriangleUInt16(int3 t) => new TriangleUInt16
17.     {
18.         a = (ushort)t.x,
19.         b = (ushort)t.y,
20.         c = (ushort)t.z
21.     };
22.     // implicit operator to turn the supplied int3 struct into 3 short values as a struct to
save memory and improve performance for triangles
23. }
24.
```

Main.cs

```
 1. using Unity.Jobs;
 2. using UnityEngine;
 3.
 4. public class Main : MonoBehaviour
 5. {
 6.     public Material material;
 7.     public NoiseSettings settings;
 8.     private void Awake()
 9.     {
10.         Vector3[] rotations = new Vector3[6]
11.         {
12.             Vector3.zero,
13.             Vector3.forward,
14.             Vector3.forward*2, // the opposite side of the initial face of cube
15.             Vector3.back,
16.             Vector3.right,
17.             Vector3.left
18.         };
19.
20.         Vector3[] origins = new Vector3[6]
21.         {
22.             Vector3.up,
23.             Vector3.left,
24.             Vector3.down,
25.             Vector3.right,
26.             Vector3.forward,
27.             Vector3.back
28.         };
29.
30.         for (int i = 0; i < 6; i++)
31.         {
```

```
32.             GameObject childObj = new GameObject("Quadtree" + 0 + i.ToString()); // This
creates a new gameobject in the scene
33.             childObj.transform.parent = transform;
34.             childObj.transform.localScale = Vector3.one/2550; // so when landed it is at
localScale 1 to avoid artefacst
35.             childObj.transform.localPosition = Vector3.zero;
36.             Quadtree quadtree = childObj.AddComponent<Quadtree>();
37.             quadtree.enabled = false;
38.
39.             // setting the correct position for each directional quadtree
40.             Vector3 origin = origins[i] * 127.5f * settings.meshScale; // 127.5 is half of
255, the number of gaps between vertices on row of each mesh
41.             Quaternion rotation = Quaternion.Euler(rotations[i] * 90);
42.
43.             // generate their mesh, the reason why I don't generate mesh within the quadtree
class that belows to the chunk is for scheduling
44.             (JobHandle jobHandle, Mesh.MeshDataArray array, NoiseSettingsData noiseSettings)
= AdvancedMeshGen.GenerateHandle(i.ToString(), rotation, 1, settings, default);
45.             Mesh mesh = AdvancedMeshGen.CompleteMeshGeneration(jobHandle, array,
noiseSettings);
46.             quadtree.Initialise(1, rotation, i.ToString(), origin, material, mesh, settings,
null);
47.             quadtree.enabled = true;
48.         }
49.     }
50. }
51.
```

Minimap.cs

```
 1. using UnityEngine;
 2. using Unity.Mathematics;
 3. using static Unity.Mathematics.math;
 4.
 5. public class Minimap: MonoBehaviour
 6. {
 7.     public CelestialBody planet;
 8.     private float scalingFactor = 0.001f;
 9.     Spaceship spaceship;
10.
11.     private void Start()
12.     {
13.         spaceship = FindAnyObjectByType<Spaceship>();
14.         transform.localScale = Vector3.one * 0.3f;
15.         transform.rotation = Quaternion.Euler(90, 0, 0);
16.     }
17.
18.     void Update()
19.     {
20.         //Vector3 minimapPosition = spaceship.rb.rotation * float3(planet.pos)  *
scalingFactor;
21.         transform.position = float3(planet.pos) * scalingFactor;
22.     }
23. }
24.
```

MinimapPlayer.cs

```
 1. using UnityEngine;
 2.
 3. public class MinimapPlayer : MonoBehaviour
 4. {
 5.     Spaceship spaceship;
 6.     Quaternion initAngle = Quaternion.Euler(-90, 0, 0);
 7.
 8.     private void Start()
 9.     {
```

```
        spaceship = FindAnyObjectByType<Spaceship>();
        transform.localScale = Vector3.one * 0.2f;
        transform.rotation = initAngle;
        transform.position = Vector3.zero;
    }

    void Update()
    {
        transform.eulerAngles += Quaternion.AngleAxis(spaceship.yawInput,
Vector3.up).eulerAngles;
    }
}

```

NoiseGen.cs

```
using UnityEngine;
using Unity.Collections;
using Unity.Burst;
using Unity.Mathematics;
using static Unity.Mathematics.math;
using static Unity.Mathematics.noise;

public struct NoiseGen
{
    public static float CalculateNoiseWithDerivatives(float3 position, NoiseSettingsData
settings)
    {
        float noiseHeight = 0f;
        float amplitude = 1f;
        float frequency = settings.frequency;

        for (int i = 0; i < settings.octaveCount; i++)
        {
            float3 samplePos = (position/settings.scale + settings.octaveOffsets[i]) *
frequency; // control which bit of noise to sample from and how detailed it should be

            float noise = snoise(samplePos);
            noiseHeight += noise * amplitude;

            amplitude *= settings.persistance;
            frequency *= settings.lacunarity;
        }
        // classic fBm

        return noiseHeight;
    }
}
```

PlayerController.cs

```
using UnityEngine;

public class PlayerController : rbBody
{
    public float maxAcceleration;
    public float walkSpeed = 0.8f;
    public float runSpeed = 1.4f;
    public float angSpeed = 1.0f;

    public float maxVel = 25f;

    public Vector2 pitchMinMax = new Vector2(-40, 85);
    public Vector2 yawMinMax = new Vector2(-80, 80);

    float camYawVal;
    float yawVal;
```

```
17.     float pitchVal;
18.
19.     float smoothTime = 0.1f;
20.
21.     Vector3 targetVelocity;
22.     Vector3 smoothVelocity;
23.     Vector3 smoothVRef;
24.
25.     Quaternion camTargetAng;
26.     Quaternion yaw;
27.     Quaternion camSmoothedAng;
28.     Quaternion originalCamAng;
29.
30.     Camera cam;
31.     public Camera mainCam;
32.
33.     public Canvas canvas;
34.
35.     public GameObject planet;
36.
37.     Spaceship spaceship;
38.
39.     Vector3 gravityUp;
40.
41.     public static event System.Action OnSpaceship;
42.
43.     private void Awake()
44.     {
45.         rb = GetComponent<Rigidbody>();
46.         cam = GetComponentInChildren<Camera>();
47.         rb.interpolation = RigidbodyInterpolation.Interpolate; // More accurate collision
detection
48.         rb.collisionDetectionMode = CollisionDetectionMode.ContinuousDynamic; // More
accurate collision detection
49.         rb.useGravity = false;
50.         rb.isKinematic = false;
51.         base.Awake();
52.     }
53.
54.     private void Start()
55.     {
56.         if(Time.time<10f)
57.         gameObject.SetActive(false); // (Only) in the beginning of the game it should be
inactive, as Start() is called everytime the object is setActive
58.         yaw = transform.rotation;
59.         spaceship = FindAnyObjectByType<Spaceship>();
60.         originalCamAng = camTargetAng = cam.transform.rotation;
61.     }
62.
63.     private void Update()
64.     {
65.         LinearMove();
66.         Rotate();
67.         gravity();
68.         switchToSpaceship();
69.     }
70.
71.     void LinearMove()
72.     {
73.         Vector3 inputRight = Input.GetAxisRaw("Horizontal") * cam.transform.right;
74.         Vector3 inputForward = Input.GetAxisRaw("Vertical") * cam.transform.forward;
75.
76.         // It must be noted that the direction of the player cubioud doesn't matter for any
calculation but rather is the angle of the camera that dictates its movements
77.
78.         Vector3 input = inputRight + inputForward;
79.         float currentSpeed = Input.GetKey(KeyCode.LeftShift) ? runSpeed : walkSpeed;
80.         targetVelocity = (input.normalized) * currentSpeed;
81.
82.         // Smooth the velocity of the player to avoid jerky movements
```

```
        smoothVelocity = Vector3.SmoothDamp(smoothVelocity, targetVelocity, ref smoothVRef, smoothTime);

        // Avoid the player from moving too fast
        if (rb.linearVelocity.magnitude < maxVel)
        {
            rb.AddForce(smoothVelocity, ForceMode.VelocityChange);
        }
    }

    void Rotate()
    {
        float camYawInput = 0;
        camYawInput = Input.GetAxisRaw("Mouse X");
        camYawVal += camYawInput * angSpeed;
        pitchVal = Mathf.Clamp(pitchVal - Input.GetAxisRaw("Mouse Y") * angSpeed, pitchMinMax.x, pitchMinMax.y); // constraint for up/down rotation

        Quaternion camYaw = Quaternion.AngleAxis(camYawVal, transform.up); // rotate about the axis
        Quaternion pitch = Quaternion.AngleAxis(pitchVal, transform.right);

        camTargetAng = camYaw * pitch * rb.rotation;
        camSmoothedAng = Quaternion.Slerp(cam.transform.rotation, camTargetAng, Time.deltaTime * angSpeed); // smoothing applied, slerp splits the rotation into several frames over the time (Time.deltaTime * angSpeed)
        cam.transform.rotation = camSmoothedAng;
    }

    void gravity()
    {
        gravityUp = (rb.position - planet.transform.position).normalized;
        transform.up = gravityUp;
        //Debug.DrawLine(Vector3.zero, gravityUp*9999f, Color.red);
        //Debug.DrawLine(Vector3.zero, transform.up * 9999f, Color.blue);
        if (Input.GetKey(KeyCode.Space))
        {
            rb.AddForce(gravityUp * 2f, ForceMode.Acceleration);
        }
        else
        {
            rb.AddForce(-gravityUp * planet.GetComponent<CelestialBody>().mass, ForceMode.Acceleration);
        }
    }

    void switchToSpaceship()
    {
        if ((spaceship.rb.position - rb.position).magnitude < 10)
        {
            if (Input.GetKeyDown(KeyCode.V))
            {
                UniversalGravitation.trackBody = spaceship; // So floating origin system tracks the right object
                Quadtree.viewer = spaceship.transform; // regen and splitting tracks the right object
                spaceship.enabled = true;
                canvas.gameObject.SetActive(true); // This is the minimap
                mainCam.gameObject.SetActive(true);
                if (OnSpaceship != null)
                {
                    OnSpaceship(); // to allow the boost of velocity so the spaceship could fly away from the planet
                }
                gameObject.SetActive(false); // player cam is parented to player so it also deactivates itself
            }
        }
    }
}
```

```
143.
```

Quadtree.cs

```
  1. using UnityEngine;
  2. using System.Linq;
  3. using Unity.VisualScripting;
  4. using Unity.Jobs;
  5. using System.Collections.Generic;
  6. using static UnityEngine.Rendering.VirtualTexturing.Debugging;
  7. using UnityEngine.Rendering.Universal;
  8.
  9. [RequireComponent(typeof(MeshFilter), typeof(MeshRenderer), typeof(MeshCollider))]
 10. public class Quadtree: MonoBehaviour
 11. {
 12.     public const float minSize = 255f;
 13.
 14.     public static Transform viewer;
 15.     Mesh mesh;
 16.     Bounds bounds;
 17.     Quadtree[] children = new Quadtree[4];
 18.
 19.     Vector3 centre;
 20.     float sideLength;
 21.     float thresholdSqrdDistance;
 22.     float quarterLength;
 23.     float sqrDistance;
 24.
 25.     int res;
 26.     bool active;
 27.
 28.     MeshRenderer meshRenderer;
 29.     MeshCollider meshCollider;
 30.
 31.     string ID;
 32.     Quaternion rotation;
 33.
 34.     Material material;
 35.
 36.     // both of below are static as they need to be accessed by all quadtrees to determine
scheduling of mesh generation
 37.     private static readonly object jobHandlesLock = new object(); // placeholder object for
thread locking for thread safety in multithreading
 38.     private static Queue<JobHandle> jobHandles = new Queue<JobHandle>();
 39.
 40.     JobHandle[] childHandle = new JobHandle[4];
 41.     Mesh.MeshDataArray[] childArray = new Mesh.MeshDataArray[4];
 42.     NoiseSettingsData[] childSettings = new NoiseSettingsData[4];
 43.     string[] childID = new string[4];
 44.     int childRes;
 45.
 46.     public NoiseSettings settings;
 47.
 48.     public static PhysicsMaterial noBounce;
 49.
 50.     public bool genLock;
 51.     public Quadtree parent;
 52.
 53.     public void Initialise(int res, Quaternion rotation, string ID, Vector3 centre,
Material material, Mesh mesh, NoiseSettings settings, Quadtree parent)
 54.     {
 55.         GetComponent<MeshFilter>().sharedMesh = mesh;
 56.         meshCollider = GetComponent<MeshCollider>();
 57.         meshCollider.sharedMesh = mesh;
 58.         meshCollider.material = noBounce;
 59.         meshRenderer = GetComponent<MeshRenderer>();
 60.         meshRenderer.material = material;
```

```
61.         bounds = meshRenderer.bounds; //world space coordinates
62.         //Debug.Log($"bounds {bounds.size}"); -- doesn't work anymore with rotation
63.         //Debug.Log("Centre: " + centre);
64.         this.centre = centre;
65.         //Debug.Log("Position: " + position);
66.
67.         sideLength =  settings.meshScale * 255 / res; // meshScale is the max scale + res
is the current resolution (it goes in powers of 2 with 1 being the lowest res - ie the
directional quadtree generated from Main)
68.         //Debug.Log($"sidelength {sideLength}");
69.         thresholdSqrdDistance = sideLength * sideLength *1.4f /4f ;
70.         quarterLength = sideLength / 4;
71.
72.         this.res = res;
73.         this.ID = ID;
74.         active = true;
75.         this.rotation = rotation;
76.         this.material = material;
77.         childRes = res * 2;
78.         this.settings = settings;
79.         this.parent = parent;
80.     }
81.
82.     private void FixedUpdate()
83.     {
84.         bounds =  meshRenderer.bounds;
85.         //Debug.Log($"mesh {mesh}");
86.         //Debug.Log($"bounds {bounds}");
87.         sqrDistance = bounds.SqrDistance(viewer.position); // sqrDistance is much more
efficient than dist as sqaure roots are very demanding
88.         //Debug.Log($"sqaure dist {sqrDistance}");
89.         if (active)
90.         {
91.             splitQuad();
92.         }
93.         else
94.         {
95.             regenerate();
96.         }
97.     }
98.
99.
100.    // Update called to check for when to regenerate itself if distance is too far
101.    // called for disabling the children / setactieve(false) to save on performance
102.    void regenerate()
103.    {
104.        if (sqrDistance > thresholdSqrdDistance*UniversalGravitation.scaleForLanding *
UniversalGravitation.scaleForLanding/2550/2550*4 && genLock is false)
105.        {
106.            //Debug.Log("regen");
107.            meshRenderer.enabled = true;
108.            meshCollider.enabled = true;
109.            //Debug.Log($"mesh {mesh}");
110.            active = true;
111.            for (int i = 0; i < 4; i++)
112.            {
113.                children[i].gameObject.SetActive(false);
114.            }
115.        }
116.    }
117.
118.    void splitQuad()
119.    {
120.        if (sqrDistance < (thresholdSqrdDistance * UniversalGravitation.scaleForLanding *
UniversalGravitation.scaleForLanding /2550/2550 ) && sideLength > minSize)
121.        {
122.            //Debug.Log($"sqaure dist split {sqrDistance}");
123.            //Debug.Log("split");
124.            //Debug.Log($"Before split mesh {mesh}");
125.            enabled = false;
```

```
126.             //Debug.Log($"Disabled mesh {mesh}");
127.             if (children[0] is null)
128.             {
129.                 CreateChildren();
130.             }
131.             else
132.             {
133.                 GetChildren();
134.             }
135.             enabled = true;
136.             active = false;
137.             //Debug.Log($"Splitting mesh {mesh}");
138.             meshRenderer.enabled = false;
139.             meshCollider.enabled = false;
140.             //Debug.Log($"After split mesh {mesh}");
141.         }
142.     }
143.
144.     void CreateChildren()
145.     {
146.         // using standard quadrant labelling to avoid confusion
147.         Vector3[] centres = new Vector3[]
148.         {
149.             centre + rotation * new Vector3(quarterLength, 0, quarterLength),
150.             centre + rotation * new Vector3(-quarterLength, 0, quarterLength),
151.             centre + rotation * new Vector3(-quarterLength, 0, -quarterLength),
152.             centre + rotation * new Vector3(quarterLength, 0, -quarterLength),
153.         };
154.
155.         centres = centres.OrderBy(c => (c - viewer.position).magnitude).ToArray();
156.
157.         for (int i = 0; i < centres.Length; i++)
158.         {
159.             GameObject childObj = new GameObject("Quadtree" + res.ToString() +
i.ToString());
160.             Quadtree child = childObj.AddComponent<Quadtree>();
161.             child.enabled = false;
162.             child.transform.parent = transform.parent;
163.             childObj.transform.localPosition = Vector3.zero;
164.             childObj.transform.localScale = Vector3.one / 2550;
165.             children[i] = child;
166.             childID[i] = ID + i.ToString();
167.             lock (jobHandlesLock)
168.             {
169.                 JobHandle previousHandle = jobHandles.LastOrDefault();
170.                 (childHandle[i], childArray[i], childSettings[i]) =
AdvancedMeshGen.GenerateHandle(childID[i], rotation, childRes, settings, previousHandle);
171.                 jobHandles.Enqueue(childHandle[i]);
172.             }
173.         }
174.
175.         for (int i = 0; i < children.Length; i++)
176.         {
177.             Mesh thisMesh = AdvancedMeshGen.CompleteMeshGeneration(childHandle[i],
childArray[i], childSettings[i]);
178.             // this calls handle.Complete() -> this function would signify that it is
possible to generate and once the previous jobs are finished or there are spare threads this
could start
179.             children[i].Initialise(childRes, rotation, childID[i], centres[i], material,
thisMesh, settings, this);
180.             jobHandles.Dequeue();
181.             children[i].enabled = true;
182.         }
183.     }
184.
185.     void GetChildren()
186.     {
187.         for (int i = 0; i < 4; i++)
188.         {
189.             children[i].gameObject.SetActive(true);
```

```
        }
    }
}
```

rbBody.cs

```
using UnityEngine;

public class rbBody: CelestialBody
{
    public Rigidbody rb;
    private void Awake()
    {
        mass = 0.000001f;
        base.Awake();
    }
}

```

Spaceship.cs

```
using UnityEngine;
using Unity.Mathematics;

[RequireComponent(typeof(Rigidbody))]
public class Spaceship : rbBody
{
    KeyCode forward = KeyCode.W;
    KeyCode backward = KeyCode.S;
    KeyCode left = KeyCode.A;
    KeyCode right = KeyCode.D;

    KeyCode ascend = KeyCode.UpArrow;
    KeyCode descend = KeyCode.DownArrow;
    KeyCode rollCounter = KeyCode.LeftArrow;
    KeyCode rollClock = KeyCode.RightArrow;

    Vector3 thruster;
    float thrustStrength = 1f;

    float roller;

    float angSpeed = 1.0f;
    float rollSpeed = 10.0f;
    float smoothedAngSpeed = 1.0f;

    int timeTillPlayer = 60;

    Quaternion targetAng;
    Quaternion smoothedAng;

    int numCollisionTouches;

    public float yawInput;

    public Canvas canvas;

    public static event System.Action OnCollide;
    public static event System.Action OnExitCollide;
    public static event System.Action OnPlayer;
    // cross scripts communication (to UniversalGravitation and PlayerController)

    GameObject planet;

    Camera mainCam;

```

```
    bool isChangeSpeed = false;

    float scaleOnCollide;

    Transform collision;

    void Awake()
    {
        rb = GetComponent<Rigidbody>();
        rb.useGravity = false;
        rb.isKinematic = false;
        rb.collisionDetectionMode = CollisionDetectionMode.ContinuousDynamic;
        rb.interpolation = RigidbodyInterpolation.Interpolate;
        rb.centerOfMass = Vector3.zero;
        targetAng = transform.rotation;
        smoothedAng = transform.rotation;
        //Debug.Log("angle " + targetAng.eulerAngles);
        mass = rb.mass;
        mainCam = Camera.main;
        base.Awake();
    }

    void Update()
    {
        Move();
        //Debug.Log($"velocity {rb.linearVelocity}");
        //Debug.Log(rb.linearVelocity);
    }

    void FixedUpdate()
    {
        Vector3 thrustDir = transform.TransformVector(thruster); // from local space to
world space
        //Debug.Log($"thrustDir {thrustDir}");
        //Debug.Log($"velocity {rb.linearVelocity}");
        rb.AddForce(thrustDir * thrustStrength*UniversalGravitation.scaleForLanding,
ForceMode.Acceleration);

        if (numCollisionTouches == 0)
        {
            smoothedAng = Quaternion.Normalize(smoothedAng);
            rb.MoveRotation(smoothedAng);
        }

        if (numCollisionTouches == 1)
        {
            timeTillPlayer--;
            if (timeTillPlayer <= 0)
            {
                switchToPlayer();
            }
            //Debug.Log($"timeTillPlayer {timeTsillPlayer}");
        }
        else
        {
            timeTillPlayer = 60;
        }
        // avoid situations where the spaceship might collide and exit more than once after
impact
    }

    void Move()
    {
        thruster = new Vector3(
            Input.GetKey(right) ? 1 : Input.GetKey(left) ? -1 : 0,
            Input.GetKey(ascend) ? 1 : Input.GetKey(descend) ? -1 : 0,
            Input.GetKey(forward) ? 1 : Input.GetKey(backward) ? -1 : 0
        );

        if (numCollisionTouches == 0)
```

```
        {
            Rotate();
        }
    }

    void Rotate()
    {
        roller = Input.GetKey(rollClock) ? 1 : Input.GetKey(rollCounter) ? -1 : 0;
        yawInput = Input.GetAxisRaw("Mouse X") * angSpeed;
        float pitchInput = Input.GetAxisRaw("Mouse Y") * angSpeed;
        float rollInput = roller * rollSpeed * Time.deltaTime;

        Quaternion yaw = Quaternion.AngleAxis(yawInput, transform.up);
        Quaternion pitch = Quaternion.AngleAxis(-pitchInput, transform.right);
        Quaternion roll = Quaternion.AngleAxis(-rollInput, transform.forward);

        // reason can't use *= is due to non-commutative nature of quaternions
        targetAng = yaw * pitch * roll * targetAng;

        smoothedAng = Quaternion.Slerp(transform.rotation, targetAng, Time.deltaTime *
smoothedAngSpeed);
    }

    void OnCollisionEnter(Collision collision)
    {
        //numCollisionTouches = Mathf.Min(1, numCollisionTouches + 1);
        //Debug.Log(numCollisionTouches);
        //Debug.Log("Collision with " + collision.gameObject.name);
        //Debug.Log($"impact Velocity {rb.linearVelocity}");

        if (OnCollide != null)
        {
            OnCollide();
        }

        rb.linearVelocity = Vector3.zero;
        rb.angularVelocity = Vector3.zero;
        vel = double3.zero;
        if (collision.gameObject.GetComponent<PlayerController>() is null)
        {
            transform.SetParent(collision.transform);
            this.collision = collision.transform;
            Quadtree quadtree = collision.gameObject.GetComponent<Quadtree>();
            if (quadtree is not null)
            {
                QuadtreeLock(quadtree);
            }
            //Debug.Log($"collision transform {collision.transform.parent.name}");
            planet = collision.transform.parent.gameObject;
            //Debug.Log($"planet {planet}");
        }
        scaleOnCollide = UniversalGravitation.scaleForLanding;
    }

    private void OnCollisionExit(Collision collision)
    {
        //numCollisionTouches = Mathf.Max(0, numCollisionTouches - 1);
        //Debug.Log("Exit Collision with " + collision.gameObject.name);
        if (OnExitCollide != null)
        {
            OnExitCollide();
        }
        transform.SetParent(null);
        //Debug.Log(numCollisionTouches);
    }

    void switchToPlayer()
    {
        if (Input.GetKeyDown(KeyCode.B) && FindAnyObjectByType<PlayerController>() is null)
        {
```

```
            // rework direct call of function in player control
            PlayerController player =
FindAnyObjectByType<PlayerController>(FindObjectsInactive.Include);
            player.gameObject.SetActive(true);

            Vector3 gravityUp = (player.transform.position -
planet.transform.position).normalized;
            player.transform.position = transform.position - gravityUp*10;

            //Debug.Log($"player position {player.rb.position}, transform position
{player.transform.position}");
            player.planet = planet;
            player.transform.SetParent(planet.transform);

            //Debug.Log($"player parent {player.transform}");
            UniversalGravitation.trackBody = player;
            Quadtree.viewer = player.transform;

            //Debug.Log($"player position {player.rb.position}");
            mainCam.gameObject.SetActive(false);
            enabled = false;
            canvas.gameObject.SetActive(false);
        }
    }

    void thrusterUpdate()
    {
        thrustStrength *= 10;
        if (!isChangeSpeed)
        {
            InvokeRepeating("DropSpeed", 0, 0.1f); // chose this over coroutine due to its
more efficient processing (and it's good for purpose)
        }
    }

    void DropSpeed()
    {
        if (UniversalGravitation.scaleForLanding < scaleOnCollide)
        {
            thrustStrength /= 10;
            isChangeSpeed = true;
            CancelInvoke("DropSpeed");
        }
    }

    void QuadtreeLock(Quadtree quadtree)
    {
        if (quadtree.parent is not null)
        {
            quadtree.genLock = true;
            QuadtreeLock(quadtree.parent);
        }
    }
}

```

UniversalGravitation.cs

```
using System;
using System.Linq;
using UnityEngine;
using UnityEngine.InputSystem;
using UnityEngine.UIElements;
using Unity.Mathematics;
using System.Security.Cryptography;
using System.Collections.Generic;

public class UniversalGravitation : MonoBehaviour
```

```
{
    Stack<CelestialBody> celestialBodies;
    private float sunMass;
    int celestialBodyCount;

    private float timeStep = 8 * 60f;

    public const double G = 6.6743015e-11;
    public const double adjustedG = 1.190594655e-10; // as stated in design, calculated value for base unit change

    float distToLanding = 100f;
    bool landing = false;
    bool leaving = false;
    public static float scaleForLanding { get; set; } = 1f;
    public float reinstateDist;
    float distanceThreshold = 0.0001f;

    public GameObject sun;
    public Spaceship spaceship;
    PlayerController player;
    public static rbBody trackBody;
    public Transform viewer;

    Vector3 actualSpaceshipPos;

    public PhysicsMaterial noBounce;

    private void Awake()
    {
        celestialBodies = new Stack<CelestialBody>(FindObjectsByType<CelestialBody>(FindObjectsSortMode.None));
        celestialBodies = new Stack<CelestialBody>(celestialBodies.OrderBy(body => body is Spaceship ? 1 : 0)); // Spaceship to be last item ready to be popped
        celestialBodyCount = celestialBodies.Count;
        //Debug.Log($"celestialBodyCount{celestialBodyCount}");
        //foreach (CelestialBody body in celestialBodies)
        //{
        //    Debug.Log(body.name);
        //}

    }

    void Start()
    {
        Quadtree.noBounce = noBounce;
        trackBody = spaceship;

        if (sun is null)
        {
            //Debug.Log("sun is null");
            sun = GameObject.FindWithTag("Sun");
        }
        sunMass = sun.GetComponent<CelestialBody>().mass;
        foreach (CelestialBody planet in celestialBodies)
        {
            if (planet.CompareTag("Sun"))
                continue;
            if (math.all(planet.vel == double3.zero) && planet is not rbBody)
            {
                planet.vel = new double3(0, 0, CalculateInitialVel(planet));
            }
        }

        Time.fixedDeltaTime = 1 / 60f;

        Spaceship.OnCollide += removeBody;
        Spaceship.OnExitCollide += addBody;

        Quadtree.viewer = viewer;
```

```
78.     }
79.
80.     double CalculateInitialVel(CelestialBody planet)
81.     {
82.         double r = math.length(planet.pos);
83.         double v = Math.Sqrt(G * sunMass / r) * planet.pos.x/r;
84.         return v;
85.     }
86.
87.     private void Update()
88.     {
89.         if (Input.GetKeyDown(KeyCode.Z))
90.         {
91.             if (Input.GetKey(KeyCode.LeftShift))
92.             {
93.                 timeStep = Mathf.Max(1f, timeStep / 2);
94.             }
95.             else
96.             {
97.                 timeStep *= 2;
98.             }
99.         }
100.    }
101.
102.    private void FixedUpdate()
103.    {
104.        //Debug.Log($"update celestial body count {celestialBodyCount}");
105.        //foreach (CelestialBody body in celestialBodies)
106.        //{
107.        //    if (body.gameObject.activeSelf)
108.        //        Debug.Log(body.name);
109.        //}
110.        RK4();
111.        if (landing)
112.        {
113.            if (scaleForLanding < 2550)
114.            {
115.                if (scaleForLanding == 2048)
116.                {
117.                    scaleForLanding = 2550;
118.                    foreach (CelestialBody body in celestialBodies)
119.                    {
120.                        if (body is rbBody rigidBody)
121.                        {
122.                            rigidBody.rb.linearVelocity *= 1.245117f;
123.                        }
124.                    }
125.                }
126.                else
127.                {
128.                    float newScaleForLanding = Mathf.Min(scaleForLanding * 4f, 2048);
129.                    if (newScaleForLanding != scaleForLanding)
130.                    {
131.                        scaleForLanding = newScaleForLanding;
132.                        foreach (CelestialBody body in celestialBodies)
133.                        {
134.                            if (body is rbBody rigidBody)
135.                            {
136.                                rigidBody.rb.linearVelocity *= 4;
137.                            }
138.                        }
139.                    }
140.                }
141.            }
142.
143.            //Debug.Log($"landing scale {scaleForLanding}");
144.            distToLanding = Mathf.Min(distToLanding * 2f, 1275);
145.            //Debug.Log($"distance to landing {distToLanding}");
146.        }
147.        if (leaving)
```

```
148.         {
149.             if (scaleForLanding > 0)
150.             {
151.                 if (scaleForLanding == 2550)
152.                 {
153.                     scaleForLanding = 2048;
154.                     foreach (CelestialBody body in celestialBodies)
155.                     {
156.                         if (body is rbBody rigidBody)
157.                         {
158.                             rigidBody.rb.linearVelocity /= 1.245117f;
159.                         }
160.                     }
161.                 }
162.                 else
163.                 {
164.                     float newScaleForLanding = Mathf.Max(scaleForLanding / 4f, 1);
165.                     if (newScaleForLanding != scaleForLanding)
166.                     {
167.                         scaleForLanding = newScaleForLanding;
168.                         foreach (CelestialBody body in celestialBodies)
169.                         {
170.                             if (body is rbBody rigidBody)
171.                             {
172.                                 rigidBody.rb.linearVelocity /= 4f;
173.                             }
174.                         }
175.                     }
176.                 }
177.             }
178.             //Debug.Log($"landing scale {scaleForLanding}");
179.             Debug.Log($"leaving");
180.             distToLanding = Mathf.Max(100, distToLanding / 2f);
181.         }
182. 
183.         ChangeScale();
184.         //Debug.Log("landing 3" + landing);
185.         if (trackBody is not null)
186.         UpdateFloatingOrigin();
187.         //Debug.Log("scale" + scaleForLanding);
188.         //Debug.Log("fixed update");
189.     }
190. 
191.     //legacy method
192.     //public static Vector3 CalcAcc(Transform body)
193.     //{
194.     //    Vector3 acceleration = Vector3.zero;
195.     //    foreach (CelestialBody celBody in Instance.celestialBodies)
196.     //    {
197.     //        Vector3 r = celBody.transform.position - body.position;
198.     //        Debug.Log(r);
199.     //        float rSqauredMagnitude = r.sqrMagnitude;
200. 
201.     //        if (rSqauredMagnitude < 0.0001f) continue;
202. 
203.     //        acceleration += G * celBody.mass * r.normalized / rSqauredMagnitude;
204.     //        Debug.Log(acceleration);
205.     //    }
206.     //    return acceleration;
207.     //}
208. 
209.     double3 CalcAcc(double3[] positions, int bodyIndex, bool firstIteration = false)
210.     {
211.         double3 acceleration = double3.zero;
212.         double shortestDistance = double.MaxValue;
213. 
214.         bool isSpaceship = celestialBodies.ElementAt(bodyIndex) is Spaceship &&
firstIteration;
215.         for (int i = 0; i < celestialBodyCount; i++)
216.         {
```

```
            if (i == bodyIndex) continue;

            double3 r = positions[i] - positions[bodyIndex];
            double rSquaredMagnitude = math.lengthsq(r);
            double rMag = math.length(r);
            if (rMag < shortestDistance) shortestDistance = rMag;

            if (isSpaceship && shortestDistance == rMag)
            {
                //Debug.Log("rMag: " + rMag);
                if (rMag * scaleForLanding < distToLanding)
                {
                    landing = true;
                    //Debug.Log("landing true 1");
                }
                else landing = false;
                // only true if the shortest distance is more than that

                if (rMag * scaleForLanding > (2 * (distToLanding + 1)))
                {
                    //Debug.Log("landing false");
                    leaving = true;
                }
                else leaving = false;
            }

            if (rSquaredMagnitude < 0.0001f) continue; // Avoid division by zero

            acceleration += G * celestialBodies.ElementAt(i).mass * math.normalize(r) /
rSquaredMagnitude;
        }
        return acceleration;
    }

    // New Method
    void RK4()
    {
        double3[] k1v = new double3[celestialBodyCount];
        double3[] k1r = new double3[celestialBodyCount];
        double3[] k2v = new double3[celestialBodyCount];
        double3[] k2r = new double3[celestialBodyCount];
        double3[] k3v = new double3[celestialBodyCount];
        double3[] k3r = new double3[celestialBodyCount];
        double3[] k4v = new double3[celestialBodyCount];
        double3[] k4r = new double3[celestialBodyCount];

        double3[] currentPositions = new double3[celestialBodyCount];
        double3[] currentVelocities = new double3[celestialBodyCount];

        double3[] tempPositions = new double3[celestialBodyCount];
        double3[] tempVelocities = new double3[celestialBodyCount];

        for (int i = 0; i < celestialBodyCount; i++)
        {
            currentPositions[i] = celestialBodies.ElementAt(i).pos;
            currentVelocities[i] = celestialBodies.ElementAt(i).vel;
        }

        for (int i = 0; i < celestialBodyCount; i++)
        {
            if (celestialBodies.ElementAt(i).CompareTag("Sun"))
                continue;
            k1v[i] = CalcAcc(currentPositions, i, true) * timeStep;
            k1r[i] = currentVelocities[i] * timeStep;
        }

        tempPositions = tempOffset(currentPositions, k1r, 0.5f);
        tempVelocities = tempOffset(currentVelocities, k1v, 0.5f);

        for (int i = 0; i < celestialBodyCount; i++)
```

```
        {
            if (celestialBodies.ElementAt(i).CompareTag("Sun"))
                continue;
            k2v[i] = CalcAcc(tempPositions, i) * timeStep;
            k2r[i] = tempVelocities[i] * timeStep;
        }

        tempPositions = tempOffset(currentPositions, k2r, 0.5f);
        tempVelocities = tempOffset(currentVelocities, k2v, 0.5f);

        for (int i = 0; i < celestialBodyCount; i++)
        {
            if (celestialBodies.ElementAt(i).CompareTag("Sun"))
                continue;
            k3v[i] = CalcAcc(tempPositions, i) * timeStep;
            k3r[i] = tempVelocities[i] * timeStep;
        }

        tempPositions = tempOffset(currentPositions, k3r, 1f);
        tempVelocities = tempOffset(currentVelocities, k3v, 1f);

        for (int i = 0; i < celestialBodyCount; i++)
        {
            if (celestialBodies.ElementAt(i).CompareTag("Sun"))
                continue;
            k4v[i] = CalcAcc(tempPositions, i) * timeStep;
            k4r[i] = tempVelocities[i] * timeStep;
        }

        for (int i = 0; i < celestialBodyCount; i++)
        {
            double3 deltaVel = (k1v[i] + 2f * k2v[i] + 2f * k3v[i] + k4v[i]) / 6f;
            celestialBodies.ElementAt(i).vel += deltaVel;
            double3 deltaPos = (k1r[i] + 2f * k2r[i] + 2f * k3r[i] + k4r[i]) / 6f;

            // since spaceship is the last element, we can do the following without
worrying it will intersect with other planets
            if (celestialBodies.ElementAt(i) is Spaceship spaceship)
            {
                //Debug.Log("vel " + rbBody.rb.linearVelocity);
                //Debug.Log("deltaVel: " + deltaVel);
                //Debug.Log("original position " + rbBody.rb.position + "scale" +
scaleForLanding);
                //Debug.Log("Update delta vel: " + math.float3(deltaVel * scaleForLanding *
timeStep / Time.fixedDeltaTime));
                spaceship.rb.AddForce(math.float3(deltaVel * scaleForLanding * timeStep /
Time.fixedDeltaTime), ForceMode.VelocityChange);
                //Debug.Log("spaceship position "+ rbBody.rb.position + "scale" +
scaleForLanding);
                celestialBodies.ElementAt(i).pos = math.double3(spaceship.rb.position) /
scaleForLanding;
            }
            else
            {
                celestialBodies.ElementAt(i).pos += deltaPos;
                celestialBodies.ElementAt(i).transform.position =
math.float3(celestialBodies.ElementAt(i).pos * scaleForLanding);
                //CheckForCrash(celestialBodies.ElementAt(i));
            }
        }
    }

    void ChangeScale()
    {
        foreach (CelestialBody body in celestialBodies)
        {
            if (body is not rbBody)
            {
                body.transform.localScale = body.initialLocalScale * scaleForLanding;
            }
```

```
        }
    }

    double3[] tempOffset(double3[] currentState, double3[] calcPhase, float scale)
    {
        double3[] temp = new double3[celestialBodyCount];
        for (int i = 0; i < celestialBodyCount; i++)
        {
            temp[i] = currentState[i] + calcPhase[i] * scale;
        }
        return temp;
    }

    //I have just decided to implement calculation direction from this script so there's no
point of using this method anymore!
    //public static CelestialBody[] Bodies => Instance.celestialBodies;

    //No longer needed as I am calculating acceleration directly from here
    //static UniversalGravitation Instance
    //{
    //    get
    //    {
    //        if (instance == null)
    //        {
    //            instance = FindAnyObjectByType<UniversalGravitation>();
    //        }
    //        return instance;
    //    }
    //}


    //The main issue is that the collision detection could register more than once
occassionally? -- this is to avoid accidental pop/push of other celestial bodies
    void removeBody()
    {
        if (!celestialBodies.Any(body => body is Spaceship))
        {
            return;
        }
        else
        {
            celestialBodies.Pop();
            celestialBodyCount--;
        }
        Debug.Log(celestialBodies.Count);
        foreach (CelestialBody body in celestialBodies)
        {
            Debug.Log(body.name);
        }
    }
    // this function reworked for stack implementation

    void addBody()
    {
        if (celestialBodies.Any(body => body is Spaceship))
        {
            return;
        }
        else
        {
            celestialBodies.Push(FindAnyObjectByType<Spaceship>());
            celestialBodyCount++;
        }
        //Debug.Log(celestialBodies.Count);
        //foreach (CelestialBody body in celestialBodies)
        //{
        //    Debug.Log(body.name);
        //}
        //Debug.Log(celestialBodies.Count);
    }
```

```
417.
418.     private void UpdateFloatingOrigin()
419.     {
420.         float3 originOffset = trackBody.rb.position;
421.         //Debug.Log("origin: " + originOffset + "scale" + scaleForLanding);
422.         //Debug.Log("origin: " + originOffset + "scale" + scaleForLanding);
423.         double dstFromOrigin = math.length(originOffset);
424.         if (trackBody is Spaceship)
425.         {
426.             actualSpaceshipPos += (Vector3)originOffset;
427.         }
428.
429.         if (dstFromOrigin > distanceThreshold)
430.         {
431.             //Debug.Log("distance from origin: " + dstFromOrigin);
432.             foreach (CelestialBody body in celestialBodies)
433.             {
434.                 body.pos -= originOffset / scaleForLanding;
435.                 //Debug.Log("body pos: " + body.pos + "name: " + body.name);
436.                 //Debug.Log("body pos: " + body.pos);
437.                 if (body is rbBody rbBod)
438.                 {
439.                     rbBod.rb.position = math.float3(rbBod.pos * scaleForLanding);
440.                 }
441.                 else
442.                 {
443.                     body.transform.position = math.float3(body.pos * scaleForLanding);
444.                 }
445.             }
446.             if (trackBody is PlayerController)
447.             {
448.                 trackBody.rb.position -= (Vector3)originOffset;
449.             }
450.         }
451.     }
452.
453.     void reActivateShip()
454.     {
455.         if (trackBody is not Spaceship)
456.         {
457.             if ((trackBody.rb.position - spaceship.rb.position).magnitude < reinstateDist)
458.             {
459.                 trackBody.gameObject.SetActive(false);
460.                 trackBody = spaceship;
461.             }
462.         }
463.     }
464. }
465.
```

NoiseSettings.cs

```
 1. using System;
 2. using Unity.Collections;
 3. using Unity.Mathematics;
 4. using UnityEngine;
 5.
 6. [CreateAssetMenu(fileName = "NoiseSettings", menuName = "Scriptable
Objects/NoiseSettings")]
 7. public class NoiseSettings : ScriptableObject
 8. {
 9.     public int seed;
10.     public float scale = 50f;
11.     public int octave = 6;
12.     public float persistance = 0.6f;
13.     public float lacunarity = 2f;
14.     public float frequency = 1f;
15.     public float amplitude = 50f;
```

```
    public int meshScale = 1000;
    public Vector3 offset;
    public AnimationCurve heightCurve;
    public Gradient gradient;

    public NoiseSettingsData CreateJobFriendlySettings(int samplePoints = 256)
    {
        // Generate octave offsets
        var prng = new System.Random(seed);
        var octaveOffsets = new NativeArray<float3>(octave, Allocator.Persistent);

        float maxPossibleHeight = 0;
        float testAmplitude = 1f;

        for (int i = 0; i < octave; i++)
        {
            float offsetX = prng.Next(-100000, 100000) + offset.x;
            float offsetY = prng.Next(-100000, 100000) + offset.y;
            float offsetZ = prng.Next(-100000, 100000) + offset.z;
            offsetX = offsetY = offsetZ = 0;
            octaveOffsets[i] = new float3(offsetX, offsetY, offsetZ);

            maxPossibleHeight += testAmplitude;
            testAmplitude *= persistance;
        }

        // Sample height curve into array
        var heightCurvePoints = new NativeArray<float>(samplePoints, Allocator.Persistent);
        for (int i = 0; i < samplePoints; i++)
        {
            float vertex = i / (float)(samplePoints - 1);
            heightCurvePoints[i] = heightCurve.Evaluate(vertex);
        }

        // Convert gradient to color array
        var gradientColors = new NativeArray<Color32>(samplePoints, Allocator.Persistent);
        for (int i = 0; i < samplePoints; i++)
        {
            float vertex = i / (float)(samplePoints - 1);
            gradientColors[i] = gradient.Evaluate(vertex);
            //Debug.Log($"Color: {color}");
        }

        return new NoiseSettingsData
        {
            scale = scale,
            octaveCount = octave,
            persistance = persistance,
            lacunarity = lacunarity,
            frequency = frequency,
            amplitude = amplitude,
            maxHeight = maxPossibleHeight,
            meshScale = meshScale,
            octaveOffsets = octaveOffsets,
            heightCurvePoints = heightCurvePoints,
            gradientColors = gradientColors,
            heightCurveSampleCount = samplePoints
        };
    }
}

// Job-compatible struct
public struct NoiseSettingsData : IDisposable
{
    public float scale;
    public int octaveCount;
    public float persistance;
    public float lacunarity;
    public float frequency;
    public float amplitude;
```

```
86.     public float maxHeight;
87.     public float meshScale;
88.     public int heightCurveSampleCount;
89.
90.     [ReadOnly] public NativeArray<float3> octaveOffsets; // improves clarify + performance
91.     [ReadOnly] public NativeArray<float> heightCurvePoints;
92.     [ReadOnly] public NativeArray<Color32> gradientColors;
93.
94.     // Helper method to sample height curve
95.     public float SampleHeightCurve(float vertexNo)
96.     {
97.         float index = vertexNo * (heightCurveSampleCount - 1);
98.         int lowIndex = (int)math.floor(index);
99.         int highIndex = (int)math.ceil(index);
100.        float t = index - lowIndex;
101.
102.        // Clamp indices
103.        lowIndex = math.clamp(lowIndex, 0, heightCurveSampleCount - 1);
104.        highIndex = math.clamp(highIndex, 0, heightCurveSampleCount - 1);
105.
106.        return math.lerp(heightCurvePoints[lowIndex], heightCurvePoints[highIndex], t);
107.    }
108.
109.    public Color32 SampleGradient(float inverseHeight)
110.    {
111.        float index = inverseHeight * (heightCurveSampleCount - 1);
112.        int lowIndex = (int)math.floor(index);
113.        int highIndex = (int)math.ceil(index);
114.        float t = index - lowIndex;
115.
116.        // Clamp indices
117.        lowIndex = math.clamp(lowIndex, 0, heightCurveSampleCount - 1);
118.        highIndex = math.clamp(highIndex, 0, heightCurveSampleCount - 1);
119.
120.        Color32 lerpedColor = Color32.Lerp(gradientColors[lowIndex],
gradientColors[highIndex], t);
121.        //Debug.Log(lerpedColor);
122.
123.        return Color32.Lerp(gradientColors[lowIndex], gradientColors[highIndex], t);
124.    }
125.
126.    public void Dispose()
127.    {
128.        try
129.        {
130.            if (octaveOffsets.IsCreated) octaveOffsets.Dispose();
131.            if (heightCurvePoints.IsCreated) heightCurvePoints.Dispose();
132.            if (gradientColors.IsCreated) gradientColors.Dispose();
133.        }
134.        catch (System.Exception e)
135.        {
136.            Debug.LogError($"Error disposing NoiseSettingsData: {e}");
137.        }
138.    }
139. }
140.
```

Design/Code Explanation (pt 2)

Final Interesting Data Structures Used

Field/Structure Name	Type/Structure	Explanation and Rationale
Vertex	struct (GeneralStructs.cs)	Contains float3 position and Color32 color. Represents a vertex in a mesh.

		Chosen to minimize memory usage by omitting normals/tangents (unused in shaders) while encoding height data via color for GPU-driven rendering.
TriangleUInt16	struct (GeneralStructs.cs)	Stores triangle indices (ushort a, b, c). Uses 16-bit integers to reduce memory overhead for large meshes (critical for quadtree LOD with thousands of triangles).
NoiseSettingsData	struct (NoiseSettings.cs)	Holds noise parameters (octaveOffsets, gradientColors, etc.). Uses NativeArray for Burst compatibility. Precomputes gradients/offsets to avoid runtime allocations during terrain generation.
NativeArray<Vertex>	NativeArray<T>	Unmanaged array storing mesh vertices. Chosen for thread-safe access in Jobs, avoiding garbage collection and enabling parallel mesh generation via IJobFor.
double3	Unity.Mathematics.double3	Stores celestial body positions in double-precision (e.g., UniversalGravitation.cs). Mitigates floating-point errors at astronomical scales while allowing conversion to float3 for Unity's Transform system.
quaternion	Unity.Mathematics.quaternion	Represents rotations in ChunkGen.cs and Spaceship.cs. Avoids gimbal lock during spacecraft maneuvers and ensures smooth interpolation between orientations. Compatibility with the job+burst instead of the usual Mathf.Quaternion. Quaternions are 4D representation (check out Terence Tao's post on explaining how it works – in fact it was first proposed by Hamilton but rightfully despised on due to its non-intuitive usages). However, it is very useful for game dev as it avoids 'gimble locks' which happens for 3D rotation directly changing the rotation of the orthogonal axes. It is also nice that it

		follows multiplicative rules though is non-commutative.
JobHandle	Unity.Jobs.JobHandle	Manages asynchronous jobs (e.g., ChunkGen). Enables parallelling and scheduling mesh generation without blocking the main thread, critical for maintaining frame rate during quadtree subdivision.
Mesh.MeshDataArray	UnityEngine.Mesh.MeshDataArray	Writes mesh data directly to GPU buffers. Used in AdvancedMeshGen.cs to bypass main-thread Mesh API limitations, enabling thread-safe LOD updates.
NativeArray<float3>	NativeArray<float3>	Stores precomputed noise octave offsets (NoiseSettings.cs). Optimized for Burst-compiled jobs, ensuring deterministic memory access and SIMD optimization.
Rigidbody	UnityEngine.Rigidbody	Manages physics interactions in Spaceship.cs and PlayerController.cs. Chosen for built-in collision detection and force-based movement, simplifying integration with Unity's physics engine.
Bounds	UnityEngine.Bounds	Defines quadtree chunk boundaries in Quadtree.cs. Used for frustum culling and LOD distance checks, minimizing rendering overhead by culling invisible chunks.
Stack<CelestialBody>	System.Collections.Generic.Stack	Tracks active celestial bodies in UniversalGravitation.cs. Ensures efficient insertion/removal during collisions (e.g., spacecraft landing) without array resizing.
float2/float3	Unity.Mathematics.float2/3	Represents 2/3D coordinates (e.g., origin in ChunkGen.cs). Used for planar noise sampling and quadtree spatial partitioning to be compatible with the mathematics library for job+burst systems.
int3	Unity.Mathematics.int3	Defines triangle vertex indices in ChunkGen.cs. Enables efficient batch assignment of mesh triangles using Burst-optimized loops.
AnimationCurve	UnityEngine.AnimationCurve	Configures height-to-color mapping in NoiseSettings.cs. Allows me to

		tweak terrain elevation profiles without code changes.
Gradient	UnityEngine.Gradient	Maps terrain height to vertex colors in NoiseSettings.cs. Prebaked into NativeArray<Color32> for GPU efficiency, avoiding runtime gradient evaluation.
Queue<Quadtree>	Systems.Collection.Generic.Queue	Used for scheduling the meshes generation so that the closest mesh near to the player/spaceship would be generated first to mitigate the freezing screens if the object is moving too fast and activating many generations at once.

A Synopsis of Important/Interesting Implementation Used

The design centres on three interdependent systems: **procedural terrain generation**, **celestial mechanics**, and **adaptive rendering**. Each system is tailored to address Unity's architectural constraints while maintaining computational efficiency and realism. Below, I elaborate on the critical algorithms and design philosophies that underpin the project, focusing on their technical implementation and rationale.

Procedural Terrain Generation

Core Algorithm: Fractional Brownian Motion (fBm)

The terrain generation system employs a multi-octave Simplex noise algorithm to simulate planetary surfaces. The algorithm iteratively layers noise at increasing frequencies and decreasing amplitudes—a technique known as fractional Brownian motion (fBm). Each octave introduces finer details, such as mountains or craters, while preserving the macro-level structure of the terrain. For example, the first octave defines continental shapes, while subsequent octaves add ridges and valleys. This approach balances computational cost with visual fidelity, as higher octaves contribute diminishing returns to the overall heightmap.

In NoiseGen.cs, the CalculateNoiseWithDerivatives method computes both the noise value and its spatial derivatives at a given position. The derivatives are critical for calculating surface normals, which are used to shade the terrain realistically. Analytic normals are crucial for binding between neighboring meshes as their normals recalculated would not match up as it uses numerical approximation by default in Unity, leaving to some shadow mismatch as shown earlier on in the 'iterative design' section. Tangents are not as visible, but it is still nice for it to be analytical. By sampling noise at a scaled position (position / settings.scale), the system ensures that terrain features remain consistent across different levels of detail (LOD). The final height value is clamped and mapped to a gradient (settings.SampleGradient) to encode elevation data into vertex colors, enabling GPU-driven texturing without CPU overhead.

Mesh Construction and Quadtree LOD

The terrain is divided into chunks managed by a quadtree structure (Quadtree.cs). Each chunk generates a mesh using Unity's Job System, which parallelizes vertex and triangle calculations across CPU threads. The ChunkGen job processes a grid of vertices, positioning them on a spherical surface by normalizing their 3D coordinates. This normalization ensures that terrain chunks seamlessly tile around a planet's surface, avoiding visible seams.

The quadtree dynamically subdivides chunks as the viewer approaches, increasing the vertex density (resolution) from 256 to 512, 1024, and beyond. (For those confused, the counting start at index 0, hence 0-255 represents 256 states – matching the max number of vertices allowed for one mesh default in Unity). This subdivision is governed by a distance threshold: if the viewer's squared distance to a chunk's bounds falls below a calculated value (thresholdSqrdDistance), the chunk splits into four children. Conversely, distant chunks merge back into their parent to reduce rendering load. To prevent frame spikes during subdivision, mesh generation jobs are queued and executed asynchronously, with completed meshes cached for reuse. It is queued with distance in mind but not implemented as a priority queue, just within the 4 subchunks the closest one would go first and vice versa.

Unity-Specific Optimization

- **NativeArray and Burst Compiler**: Vertex and triangle data are stored in unmanaged NativeArray buffers to avoid garbage collection stalls. The Burst Compiler optimizes the ChunkGen job into SIMD-enabled native code, reducing vertex processing time by ~40% compared to managed C#. These effects compound up and the result is an almost instantaneous render (still a bit of delay) compared to the old design. This can be seen in the following testing section for further comparison.
- **Precision + Scaling**: At large spatial scales, Unity's 32-bit floating-point precision causes vertex jitter. To mitigate this, all positions are stored as double3 in UniversalGravitation.cs and scaled down during rendering using a dynamic scaleForLanding factor (e.g., 1:1000). This dual-scale approach ensures smooth camera movement even when traversing interplanetary distances.

Celestial Physics Simulation

Gravitational Integration via RK4

The n-body gravitational system uses fourth-order Runge-Kutta (RK4) integration to compute trajectories, simulating orbits over long run while not boring the user to death. There's a design for the user to be able to change the time step as desired. In UniversalGravitation.cs, the RK4 method iterates over all celestial bodies, calculating mutual gravitational accelerations. For each body, the acceleration is calculated using Newton's law of gravitation. These calculations are performed in double-precision (double3) to minimize rounding errors before rounding it to float3 then Vector3 for display as that's the data type supported for rendering transforms' positions.

Floating Origin Adjustment

To prevent floating-point precision errors, the UpdateFloatingOrigin method shifts all celestial bodies when the viewer exceeds a distance threshold (e.g., 1,000 km). This adjustment is invisible to the player, as the spacecraft's position is recalculated relative to the new origin.

Collision Handling

Collisions between the spacecraft and terrain are detected using the default Unity rigidBody and its ContinuousDynamic collision mode, which performs iterative raycasting between frames to prevent tunneling. Upon collision, the spacecraft's velocity is zeroed, and control shifts to a planetary exploration mode (PlayerController.cs). Here, movement is constrained to the planetary surface using a gravity-aligned coordinate system. The player's orientation is continuously updated via Quaternion.FromToRotation, ensuring the "up" direction always points toward the planet's center. I have decided to remove the system for crashes as it's really hard to control the spaceship and it is very frustrating (for me) to keep crashing.

Adaptive Rendering and Memory Management

Quadtree LOD System

The quadtree manages terrain resolution by evaluating the viewer's proximity to each chunk. When a chunk's bounds intersect the viewer's frustum, it generates higher-detail meshes using AdvancedMeshGen.cs. Each mesh is constructed from a NativeArray of vertices and triangles, which are uploaded to the GPU via Mesh.MeshDataArray. This approach bypasses Unity's main thread, allowing parallel mesh updates without blocking rendering.

Mesh Pooling

To minimize heap allocations, the system recycles mesh data from inactive chunks. When a chunk is culled, its mesh is stored in a pool and reassigned to new chunks during subdivision. This reduces the frequency of Mesh instantiation, which is very costly.

User Interaction and Controls

Spaceship Flight Model

The spacecraft (Spaceship.cs) uses quaternion rotations to avoid gimbal lock during complex maneuvers. Thrust is applied using Rigidbody.AddForce in ForceMode.Acceleration. The keyboard inputs are smoothed to ensure a nicer user experience. The speed is also adjusted based on the current scaling of the system to maintain the same relative speed.

Planetary Exploration Mode

In planetary mode (PlayerController.cs), movement is constrained to a spherical coordinate system through applying a centripetal acceleration that is relative to the planet's mass (serving as gravity) that is perpendicular to the player velocity (which is changed by the usual wasd inputs), with a cap on velocity to match realism. The camera's orientation is coupled to the player's rotation to maintain a first-person perspective.

Minimap System

The minimap (Minimap.cs) renders celestial bodies as 2D icons on a UI overlay. Positions are scaled by a fixed factor (e.g., 1:10,000) to fit within the minimap's bounds. This abstraction simplifies navigation, at the trade-off of in non-appearance of distant planets.

Testing

The game has 4 major functionalities - planet generation, gravitational simulation, spaceship control, player control. Hence, it would be most sensible to split the testing into these areas, alongside a test for the UI/UX and an extra test for speed through profiling on the CPU and GPU.

Testing Plan

1. Procedural Planets & Terrain

This is tested both in game and separately for choosing the values for the terrain generation for the planets. The reason for this is due to that in game the planets are not supposed to be changed but instead preset with values that match as closely to each planet's appearance.

Test ID	Requirement Reference	Description	Method	Expected Outcome	Actual Outcome
1a	Procedural Planets	Verify Perlin/Simplex noise generates unique terrain.	Generate 5 planets with different seeds. Inspect planets	Each planet has distinct mountains, valleys, and craters.	Normal
1b	Noise Function (Fractal Brownian Motion)	Test multi-octave noise layering.	Generate terrain and inspect planets	Terrain shows fine details overlaid on large-scale features.	Normal
1c	LOD System (Quadtree)	Check LOD transitions as player approaches.	Generate planet, inspect it through moving a 'viewer' that mimicks the player. Observe mesh resolution.	Mesh subdivides smoothly; no visible seams or popping.	Normal
1d	Planets in game	Verify planet matches the expected gradient	Fly by planets to check the gradient	Matches colours of the planets in real life that is set in the colour gradient	Normal
1e	Colour Gradients (Biomes)	Validate elevation-based colour mapping.	Walk from a valley to a mountain peak	Colours transition from matching each planet's gradient	Normal
1f	Sun	Verify the sun's colour and glare movement	Fly by the sun and observe it	It should be glaring red/orange with the glare in random movement	Normal
1g	Size of landed planet	Verify sizes of planets from player POV after landed	Observe the curvature of the horizons for each planet	Larger planets should have straight line horizon whereas smaller planets' horizon could be observed	Normal

2. Visuals, Screen when loaded & General UI/UX

The main test here is how good it looks in game, so it is most sensible to test this in the built version of game. The main aim here is not about the details of each individual celestial body but focused on

the visual aspect from the player's perspective, especially the initial scene once the player loaded up as this gives the first impression.

Test ID	Requirement Reference	Description	Method	Expected Outcome	Actual Outcome
2a	Minimap	Test minimap display area and planet sprite position displays	Inspect the minimap when the game loads	The minimap shows up at the correct place and angle in the hud. The planet sprites match the positions of the actual planets in the right ratio	Normal
2b	Spaceship Interior	Test the visuals of the interior in game where the camera is placed inside the model	Look at it and critique	The spaceship interior should be rendered in high quality.	Normal
2c	Light Reflection	Test lighting	Check the lighting onto the spaceship from different angles.	Light should reflect the brightest in the direction of the sun	Normal
2d	Spawn Position	Verify spawn position	Move/rotate a bit to check	Should be 90 degrees off Earth (Left) /Mars (right) to either side	Normal
2e	Planet Spawn Positions	Verify planet spawn positions	Rotate towards Earth then wait for a bit, rotate towards Mars and wait for a bit	The planets should align initially more or less but then the planets should start orbiting and I should be able to see horizontal movements of planets from the spaceship POV	Normal

3. Gravitational Physics

The Following Test in done both in a built version and in the Unity Editor so I could also have a view of the whole game in the scene mode to verify whether physics interaction between various bodies are correct as it's hard to be observed from my spaceship

Test ID	Requirement Reference	Description	Method	Expected Outcome	Actual Outcome
3a	Newtonian Gravity (Planets)	Validate inverse-square law for planetary orbits.	Simulate Mercury's orbit around the Sun for 88 days in game time	Mercury follows a roughly circular orbit	Normal
3b	Spaceship Gravity	Test gravitational pull-on spaceship various planets	Set the initial position of the planets to be a short set distance away from each planet and time how long it takes until it lands on the planet	The time for heavier planets would be smaller for the spaceship to land	Normal
3c	Gravity during scaling changes	Test whether the gravity system works	Do a fly by the planets (not landing but close	The spaceship would demonstrate 'gravity' assist and the position	Normal

Test ID	Requirement Reference	Description	Method	Expected Outcome	Actual Outcome
		as expected when the scaling changes	enough for the scale to change)	of the planets before and after should match how it would be without the scaling in place	
3d	Floating Origin System	Check cases when the spaceship or player is the main character whether both cases are tracked properly (the 'trackbody' to be reset to the centre after the minimal threshold was met)	Check the position of the transforms of the spaceship and player in the Unity Editor as well as the resolution of the screen in game	There's no jitter on screen + the transforms of the player + spaceship stay relatively close to 0.	Normal
3e	Time control	Test whether time control works	Press **Z** and left shift **Z** Time the planets' orbits	The planets orbit at higher speeds	Normal

4. Spaceship Mechanics

Test ID	Requirement Reference	Description	Method	Expected Outcome	Actual Outcome
4a	WASD Controls (Thrusters)	Test linear movement in zero-G.	Press **W** → forward thrust. **A** → left thrust. **S** → reverse. **D** → right thrust.	Spaceship accelerates smoothly in the pressed direction.	Normal
4b	Mouse Rotation (Pitch/Yaw)	Validate camera-relative rotation.	Move mouse up/down (pitch), left/right (yaw).	Spaceship rotates precisely with mouse movement with smoothing applied	Normal
4c	Arrow Keys	Test arrow controls.	Press → (roll right) and ← (roll left). Up arrow and down arrows also give acceleration in their respective directions	Spaceship rolls/accelerates up or down with smoothing apply	Normal
4d	Camera	Test Camera stability	Do any movement and test whether the camera wobbles	The camera should be perfectly still compared to the spaceship, no wobbliness should be detected	Normal
4e	LOD System	Test when spaceship is	Drive spaceship into the planets	We should see a gradual increase in	Normal

Test ID	Requirement Reference	Description	Method	Expected Outcome	Actual Outcome
		approaching the planets		detail as we head closer	
4f	Mesh Collision	Test when spaceship collides (lands on the planet)	Drive the spaceship into the direction of the planet until it collides	Lands on it (maybe with a few bounces) but doesn't go through the mesh	Normal
4g	Exit the ship	Test transition from spaceship mode to player mode	Press **B** once it landed and is still. There should be a slight time delay before the activation is possible to mimick landing procedures	Slight delay before the button works and the view switches to the player mode	There is an issue with spaceship ejected into outer space that causes a drop in scale for planet and the mesh goes weird. Solved!
4h	Re-enter the ship	Test transition back from player to ship mode	Press **V** when the player is close to the spaceship	Pops back to spaceship mode	Normal
4i	Escape Velocity	Test controls after landing	Press keys to move the spaceship away from the planet	Initially would give higher acceleration but then acceleration would drop once it is slightly further away	Normal

5. Player Interaction (Planetary Surface)

Test ID	Requirement Reference	Description	Method	Expected Outcome	Actual Outcome
5a	Surface Gravity	Test gravity on low-mass vs. high-mass planets.	Jump on Earth vs. Mars	Jump height on Mars is larger than jump height on Earth. Gravity works properly	Normal
5b	Movement Speed	Validate walking/running on terrain.	Press **W** to walk forward; **Shift+W** to sprint.	Player sprints at higher speed than	Normal
5c	Jump Mechanics	Test jump height under varying gravity.	Press **Space**	Player jumps and lands with normal mesh collision	Normal
5d	Camera (First-Person)	Ensure camera follows player orientation.	Rotate player left/right; tilt head up/down.	Camera matches player's head movement without latency. Vertical movement is also	Normal

Test ID	Requirement Reference	Description	Method	Expected Outcome	Actual Outcome
				constrained around 80 degrees up and down	
5e	Movement	Test direction of movement	Change rotation and start walking on the planet	Walking direction should align with the camera direction	Normal
5f	LOD	Test whether trackbody is changed to player	Start moving and observe the quadtrees in the editor	Like spaceship, when the player moves around the planet the quadtree level updates	Normal
5g	Spaceship Chunk Maintained	Test whether spaceship got flung off/ some error message for object reference	Walk quiet far away from the spaceship and observe	Nothing should happen	Normal

6. Performance & Optimization

This is the test on how bad/good my optimisations are. As I had previously (in design) mentioned, the main bottleneck comes from the generation of quadtree. Another goal is testing the other bottlenecks, whether it's a cpu or a gpu issue for evaluation in future improvements. This could be tested by simply comparing the time (between the generation using multithreading and scheduling vs not), as well as using the profiler.

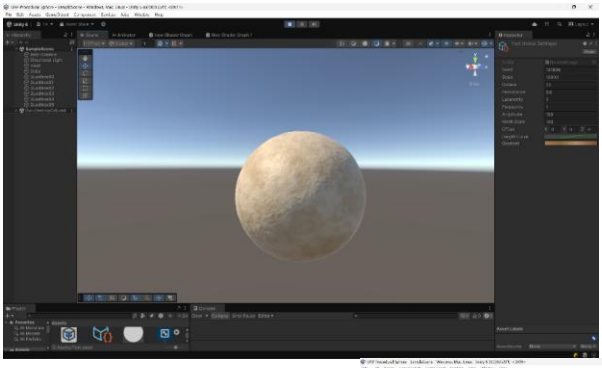
Test ID	Requirement Reference	Description	Method	Expected Outcome	Actual Outcome
6a	Normal Frame Rate	Check average CPU/GPU usage	Use profiler	Maintain 60fps ish	Normal
6b	Frame Rate During Landing on Planet (In game)	Check CPU/GPU load during LOD updates.	Monitor CPU/ GPU usage while flying toward a planet.	I honestly don't know what to expect	
6c	Quadtree Gen	Test speed of quadtree generation using multithreading or without	Time both using a moving viewer for speed of generation	The multithreading should be a lot faster than the counterpart	Normal

Recorded Tests:

Test 1:

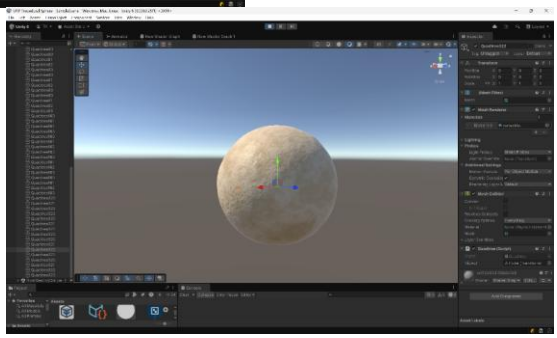
Part A Objectives: (1a-1c + 6f) – Test Planet Generation Process

As stated previously, this test is conducted on a separate Unity project that contains only the scripts for planet generation. This is to for the mapping of the noise settings to be directly tested. There is an argument that this is not in game and therefore the test is invalid, but since the scripts are the same for planet generation, isolating through modular testing demonstrate the modular design of my code and clearer test.



Here I generated a planet using this specific scriptable object. Although I can't mathematically prove it that it is equivalent to the parameters based on the image. The image's gradient, amplitude, etc matches the expected as shown above. The quadtrees are also generated from the main.cs file as expected.

The LOD system is working as expected, more details are shown in the video below that demonstrates both generating quadtrees when the viewer moves closer to a chunk (with the parent mesh not deactivated but instead disabled – confirm with the tick on meshRenderer and collider removed) and deactivating (greyed out) as the view moves further away.



Compare this video to the one in design for test 6c. It is quite clear that the generation here is much quicker! (100ms delay vs 5s delay)

Commented [TK1]: Provide a video of direct comparison as well

Part B Objectives: (1d + 3c) – Fly by Planets

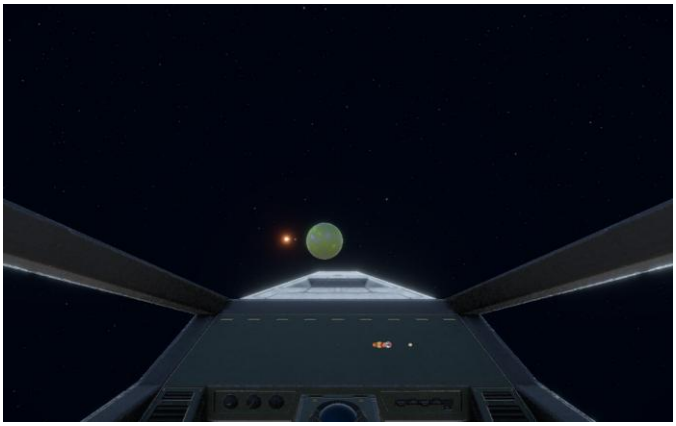
The pictures (from left to right) show the progression of a spaceship conducting a fly by on Earth. The enlargement in scale and disappearance of sun suggest scaling works and that it resumes in normal speed suggest the gravitational attraction is working as expected as well!

Test 2:

Objectives: (2a-2e + 1f) – Scene After Immediate Loading

Scene immediate after loading:

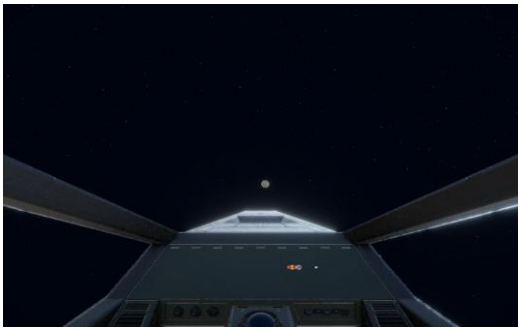
Background stars are present from the skybox with the colour matching (black). Spaceship interior is rendered without much visible artefact, with the hud displaying the minimap of the planets nearby of the spaceship – represented by an arrow pointing in the direction that the nose of its spaceship is facing. The positions of the planets of the minimap seem to accurately reflect their actual positions. The exterior reflections are accurate as seen by the glow.



As I rotate the ship towards the Earth, the arrow moves corresponding to the direction. The glare of the sun is observed (1f), and the Earth's colours and shape is accurate. From the video, you can also clearly see the Earth's movement in orbit, meaning the initial velocity calculation is at least responding with an output.

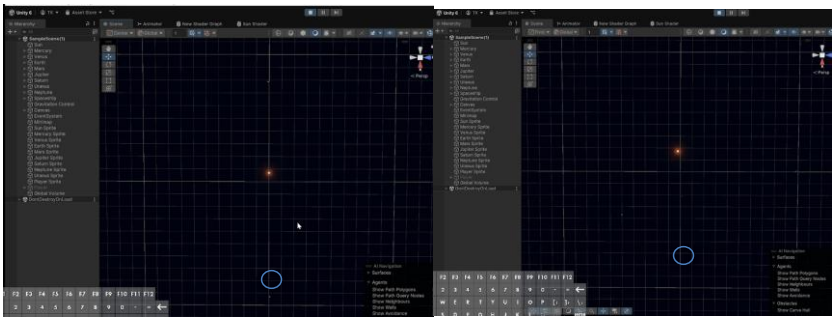
P.S. You can also see the other planets in the background emerging out, so it seemed that they are in the right place as well. + the background star arrangement has changed, check video for continuous movement

The other side (which is Jupiter) is also rendering as expected.



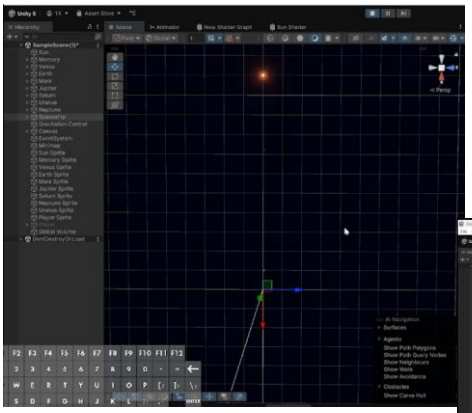
Test 3:

Objectives (3a,3b,3d,3e) – Celestial Body Movements

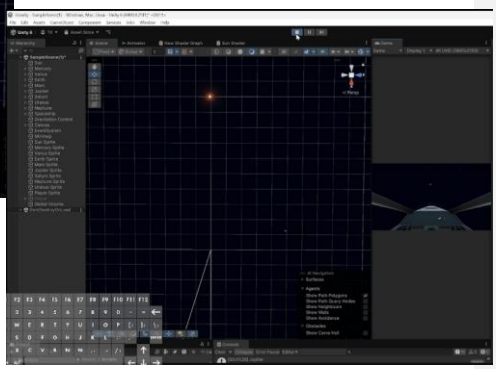


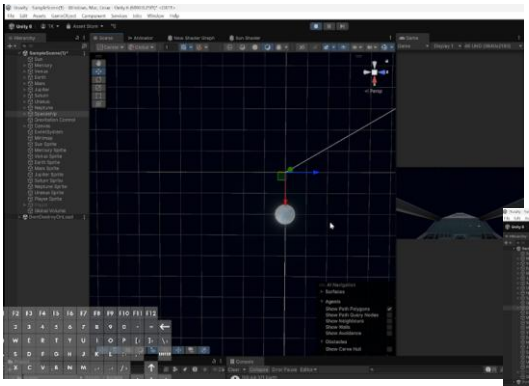
Zooming in on the circle will spot Mercury completing one orbit. Based on the other planets' positions it is not unreasonable to conclude that the planets do move in circular orbit. (The reason why the sun has 'moved' is due to the floating origin system and since the spaceship is continuously moving due to gravitational attraction, the sun's position is also updated). For the complete orbital movement (which I also used the time control to speed up orbit) check the first bit in the video above.

The following 2 scenarios demonstrate the gravitational attraction of a spaceship not exerting any force (only force experienced is gravitational force from other bodies). I've placed the spaceship to be 101 units away from planets and test their trajectories.

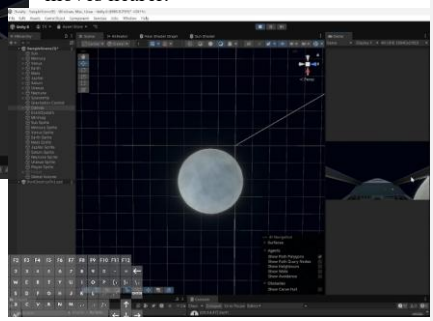


The first scenario is done with Earth. (Picture shows progression from left to right). The earth starts off straight above the spaceship and below the Sun. As time progress, spaceship doesn't get attracted into Earth (ie collision) – expected as Earth has such a low mass and force is proportional to it + Earth is orbiting at high velocity itself. I have also doubled checked with Venus which gives the same result (as expected due to same reasons).





I have then tested with a planet of heavier mass (ie Saturn). This time the spaceship is pulled onto Saturn – verifying that the gravitational system is indeed working as expected. This also indirectly tested the scaling change of the planet as spaceship moves nearer.

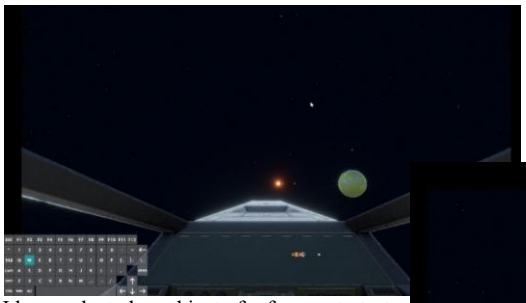


Test 4:

Part A Objectives (4a-4d) – Spaceship Controls

Here I used a plugin 'Input Relay' with OBS studio to track my keys and mouse to demonstrate the player POV. This is not part of the in-game footage.

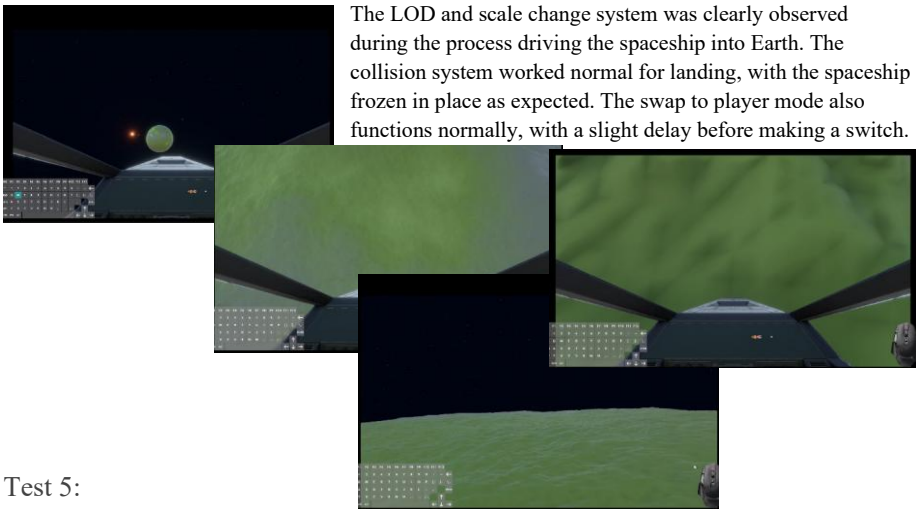
This test tested the rotation – controlled by the mouse movement, linear movements controlled by WASD + up/down arrow as well as the rolls through left and right arrow keys.



I have selected an object of reference to test against, in this case the Earth (as motion is relative). While I haven't explicitly stated about camera stability, it is evident through the video that it is the case.

Part B Objectives (4e-4g) – Landing on Planet

https://www.youtube.com/watch?v=i_y4Wh4sTZs

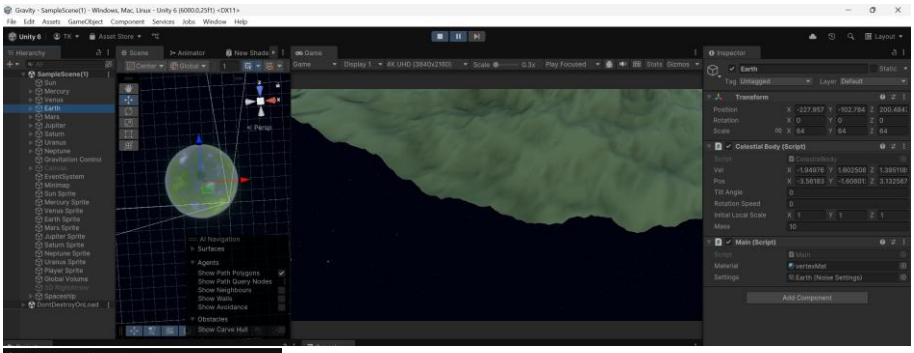


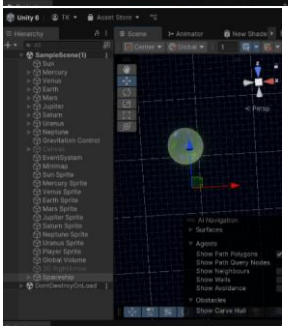
The LOD and scale change system was clearly observed during the process driving the spaceship into Earth. The collision system worked normal for landing, with the spaceship frozen in place as expected. The swap to player mode also functions normally, with a slight delay before making a switch.

Test 5:

Objectives: (5a-5f, 1e, 1g, 4h, 4i) – on Planet Movement

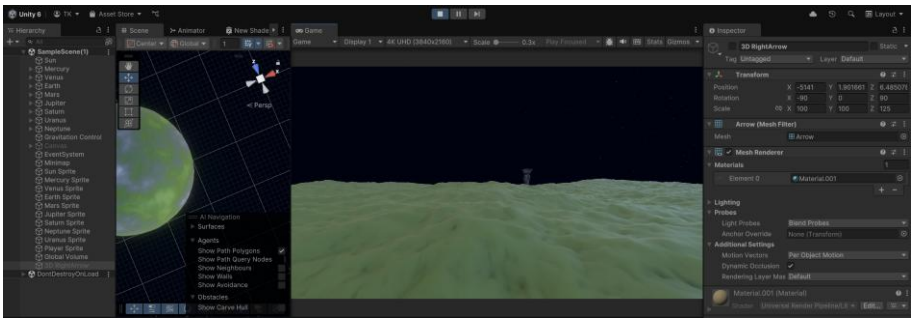
Issue





There is an issue found with planetary scale dropping one level (/4 in linear scale) after a certain time due to the spaceship being ejected so far away that it causes a ‘leaving’ operation to be called and hence a drop in scale. This is due to the shortcut that I used for initiating the player’s position (placed exactly same as the spaceship instead of slightly further away causing a strong collision). However, when I first coded it there were no issues as the spaceship was only ejected a certain distance away then fall back down to the planet relatively quickly. Again, speculating, I think this is due to some changes I made to the gravitational algorithm afterwards that caused issues.

Solution

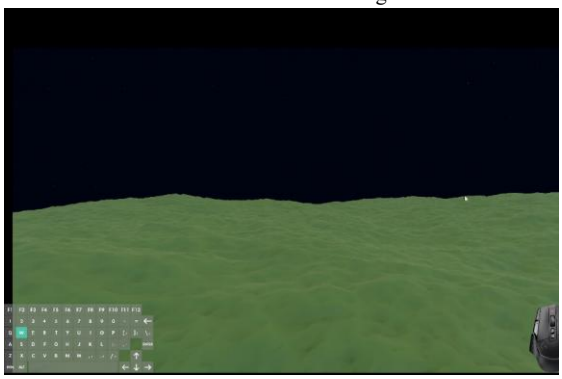


This is fixed easier through changing the initial position of the player to be placed vertically higher, now the spaceship is there without being ejected away.

```
1. Vector3 gravityUp = (player.transform.position - planet.transform.position).normalized;
2. player.transform.position = transform.position - gravityUp*10;
```

Once the player has landed and exit the spaceship, he lands on the terrain with the spaceship locked in the position of landing (in this case nosedive). The player can manoeuvre freely on the terrain with WASD and spacebar (jump), with the commands responsive to the direction which the player is facing. The direction can also be rotated using mouse like spaceship but unlike its counterpart there's a constraint of up/down movement as designed.

The colour gradient sample is also evident (clearly in clip) – when I walked from a high terrain (landing strip) to a lower terrain (pic to right), the colour gradually shifted from pale green/ white to grass green. The video also shows the LOD system updating the chunks as I move along. The spaceship is still clearly there unchanged after I took a long scroll so the recursive algorithm to disable mesh regeneration worked!



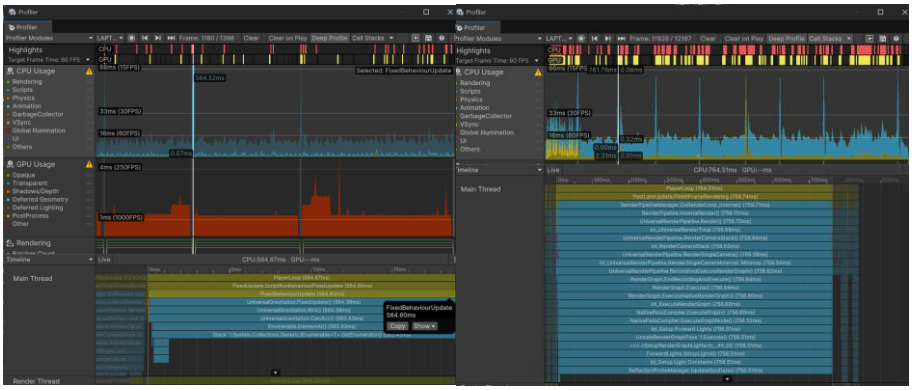
The re-entering to spaceship was pictured on the left. The bottom displays the spaceship leaving the planet, the velocity is increased as expected for the initial takeoff then resumed back into normal. This extra clip tests the left shift system

for running as I forgot in the previous full test.

Test 6:

Objectives (6a-6b) – Performance

CPU/GPU performance in game:



Unfortunately, Unity profiling doesn't support URP yet so for GPU usage it is all labelled as 'other', but through the video, we can still infer some information on its causes. Another aspect to keep in mind is that when the profiler is attached, the render jobs are not applied, which would slow down the rendering (although I'm not sure how great its impact would be)

We can clearly see spikes in two places.

1. During normal in space gameplay, the spike comes across for update function calls, this is normal and not much is there to do → the fps pretty much maintained at 60 so it's good enough (left side)
2. During landing phase, the main issue of lag comes from the rendering overhead in CPU. However, other than when the rendering is called (where mesh is received from the jobs), the fps does maintain between 30-60 which is acceptable. As I don't understand how to rewrite the rendering calls to space them out, it is an issue which I can't improve on immediately.

Evaluation

Completed Objectives

Here I will compare the initial objectives to the final product. Some points are quite simple so I would just list a simple yes/no and other points I will further elaborate if the point is not direct.

1. Procedural generation of noise mimicking planets of solar system
    1.1. Use technique of Fractional Brownian Motion to generate 3D terrain
    Yes, I used scriptable objects to control fbm to achieve a unique look for each planet's terrain
        1.1.1. Add upon the built-in Unity.Mathematics.noise.snoise function for noise variants
        I didn't particularly add to the noise function as they would require rewriting the function from scratch. I had considered using doubles instead of floats to avoid the floating-point precision issue that I had shown in design, but the drawbacks overshadow the benefits. Mainly due to the optimisation that the mathematics library has innate and the sampling efficiency – as there's already issues with the speed of quadtree generation it doesn't make sense to make it even worse. This does mean I sacrifice the noise offset function but honestly doesn't make too much of a difference in its appearance.
        1.1.2. Use seed to generate the base layer of noise to ensure that it remains constant every time it is being generated (pseudo-random)
        Done – obviously different for each planet pseudo-random system make sure the game every time when restarting will always give the same looks
        1.1.3. Tweak values for each of the 8 planets in solar system to match its looks
        This is done by tweaking the scriptable objects for each planet. The main thing is the gradient of the planets which I tweaked
        1.1.4. Use offset to ensure that each layer of noise comes from a different part of the simplex noise function for distinctiveness
        This is the tradeoff which I somewhat sacrificed. However, each planet still samples from a different part but is constrained to avoid floating point precision.
        1.1.5. Implement a gradient through lerp of colours for various heights to generate height-based colours reflective of real terrain of planets
        As said before, done.
2. Mesh generation for procedural real sized planets
    I generated mesh for procedural planets, though might not be 'real-sized'.
    2.1. Generate by the sphere with technique of spherised cube
    I went with normalising at the end rather than the more even formulae. The main reason is that it is good enough for purpose while avoiding inefficient calculations that the more even formulae entail
    2.2. Model with real data (ie their velocity, position, mass, etc all scaled down)
    I got data from NASA website so I assume it should be right. The ratio is maintained with the unit as mentioned in design tweaked to balance between the floating-point limitation in Unity with a changing scale system. The velocity is calculated assuming circular orbit, although in reality the orbits are elliptical. However, the difference is

too small to be observed from the angles available in game, so I resort to an easier calculation.

2.2.1. Each planet has a unique mesh to reflect off real life structures

As mentioned above, done.

2.3. Implement surface gravity on the bodies which is varied proportionally to the mass of the planets

Done. However, a slightly unrealistic adjustment is needed for jump I made the acceleration constant regardless of surface gravity (physically incorrect but needed to you can navigate properly on every planet).

2.3.1. Bounciness of Player On Terrain

Automatically done with Unity Physics system.

2.4. Friction on planets to avoid players slipping around due to the planet's rotation (if implemented) and movement

I haven't implemented rotation at the end. I had experimented with friction in Unity with physics material. However, in physics material friction is only accounted for linear friction, meaning that I must manually implement rotational mechanics that is not supported with the physics system in rigidbody for planetary rotation. The main issue with this is that this clashes with my input movement as I had used a linear movement using AddForce (mimicking a tangential force with a small delta like in that of numerical solutions). To resolve it I must not use the physics system and instead directly calculating its torque for angular acceleration that must be updated every frame. This is very inefficient and doesn't add too much so I decide to leave it out.

3. Implement continuous Level of Detail system using quadtrees

Used a non-standard recursive method to get quadtree generation and deactivation depending on a threshold distance from planet, recycling the already generated mesh to avoid extra generation time. It tracks the spaceship and splits the planets meshes into higher details when it's closer.

3.1. Implement chunked terrain that allows for endless non-repeating terrain reflective of movement on a spherical planet

Implemented a floating origin system tracks the player to make sure it is always at the origin to avoid floating point precision issue. The player is also tracked so that the closest mesh could split to give maximum details.

3.2. Allows maximum detail of mesh without a significant drop in fps

The mesh by default has a max number of vertices in 256*256, hence the max size is 255*255. Therefore, it makes most sense to maximize each mesh, once the mesh dimension drops below 255, I won't split anymore as the extra detail is non-distinguishable.

4. [Extension] Optimise the generation of meshes with multithreading and scheduling

This is the extension which I focused on as it is severely affecting my gameplay. The offloading from main thread in mesh generation saved a lot of time. However, I am unsure whether scheduling has further optimised the system. It theoretically should generate the closest mesh first as it operates as a queue but since there are so many meshes, and each timing is quite small with batches of 4 being sent at the same time due to quadtree structure, it doesn't have significant impact from my tests.

4.1. Use Unity's job and burst system for best performance possible

This perhaps took the longest as I was previously unfamiliar with the Unity Job system. The main thing here is that I need to translate my scriptable object which is a

class to a struct as parallel processing can't access the same piece of memory or override issues might occur.

5. Create a spaceship to fly around in, this would also be affected by gravitational forces of other bodies.
   Normal stuff chucked it in the universal gravitation
   - 5.1. Spaceship model could be taken from unity asset store
     Yes, I did indeed
   - 5.2. When landed players can leave the spaceship and transfer into surface gravity and walk around the planet reflecting realistic physics interactions
     Pretty much did what it said. I added buffer time to avoid any issues with landing bugs which happens sometimes due to Unity's collision system registering more than 1 collision and cause oscillation before it settles down in certain occasions.
6. Realistic gravitational attraction between bodies
   As realistic as it could go!
   - 6.1. Implement gravity systems between celestial bodies with Newtonian law of universal gravitation.
     Yup, with a twist in changing scale to fit planets into the screen. After all I don't have a screen of 1 AU wide.
   - 6.2. Find a way to balance out realism but fitting the constraint of Unity, its physics and rigidBody usages
     I just didn't really use Unity physics as it has too many constraints that makes it useless for large scale simulations. Instead, I wrote my own system that directly updates the planets' transforms. One of the main reasons' so that I could use double precision instead of float (single) precision.
   - 6.3. Use RK4 for greater accuracy over varying time steps
     Yup.
     - 6.3.1. Allow the user to control how fast or slow the simulation should be running
       Yup!
7. Create a 2D map for the solar system that you can view when flying
   YUP
   - 7.1. Have markers around planets to indicate the spaceship's velocity and distance away from planet
     Didn't do this. Mainly due to the scale that I used at the end makes giving a unit to speed redundant. Also, I don't like that there are velocities and distance around the planet I quite like the organic view of just a space exploration game.
   - 7.2. [Optional] Embed the minimap into the UI of the spaceship giving it a realistic look
     Made it fit into the UI, albeit with some limitations as space is limited on the hud
8. Have a camera which follows the player/spaceship that stays away from them in a fixed distance.
   I put the camera inside the spaceship and camera outside of a makeshift rectangular cuboid for a representative player. No coding is needed here.
   - 8.1. Rendering of planets using Camera as the origin for optimisation such as occlusion culling to reduce vertices and triangles rendered
     Done with Unity setting, I didn't do any work for this.
   - 8.2. Allows swapping between the cams of player and spaceship seamlessly
     Done!

EXTRA: Changing scale system. To transition between celestial scale to landing on planet scale for larger planets that feels like actual planets rather than a large sphere I set thresholds where like the

LOD quadtree system, once exceeded, will go to the next scale. However, this scale scales everything up, the size of planets and their distance from each other. There are many thresholds which as explained in design scaled twice with the distance scaled up 4 times to keep the threshold always at half distance of the spaceship to planet distance. This allows for a large planet to be seen whether landed while accompanying many planets from far away, allowing for best in both worlds.

A Short Critique (User Feedback Interview)

After evaluating myself, it is time to let Tommy have a dig at the project

Below are some comments and constructive feedback from him summarised in bullet points

- The game run mostly on good fps except for landing on planets – there are no crashes or bugs experienced (Tommy only complained heavily on the landing procedure and was rather happy on the rest)
    - However, he did mention quite a bit of times that the landing procedure was very laggy and clearly wasn't the happiest guy during the landing phase
- The game reflects of real-life physics in the movement of spaceship and planetary movement (Tommy said the game feels very real unlike sci fi – slow moving compared to ridiculously movement speed although he did say it feels a bit weird initially as he's not used to this format)
    - Tommy also likes that the planet ratios and speed are accurate
    - However, he also mentioned that the navigation is quite hard, and I agree that it would benefit from an easier automatic navigation option
- It's nice how when landed the terrain has magnified and it has realism in representing an actual planet, feels very real rather than a massive sphere (Tommy's a happy guy here)
- The sun's glare is good (Tommy was quite amazed at it)
- The lighting is nice (Tommy's impressed by the changing lighting and slight glow around the planets and the spaceship)
- Larger screen/interface (Tommy complained about the miniscule size of planets on minimap)
    - However, Tommy quite liked the spaceship model/interior which I picked
- Put something on the planet (Tommy is rather disappointed that the planet manoeuvring is empty, although he liked the freedom in roaming around the terrain)
    - Place objectives like that in Outer Wilds with hints dropped across different places on planets to guide the user to figure out some physical fact (for example orbit calculation – Kepler's) which can lead to achievements
- Maybe add interfaces for o2 levels etc especially in the spaceship to make the game more time-tight for the user to calculate an efficient trajectory travel to next planet (Tommy's a bit bored after a while and can be seen yawning by the end of the interview/test-play)
- Would be great to add a tutorial (Tommy was quite confused initially of each key's functionalities and the spaceship navigation system)

Finally, I must conclude with the question:
What would you point to as the most important feature to improve?
I think most elements are there, I can very much see the resemblance to No Man's Sky but for our solar system. I think the one thing that really trips it up right now is the lag during landing. Once that is fixed, I can see great potential from this game.

Extension, Efficiency and Enrichment

The process of landing onto the planet is still quite inefficient, as demonstrated by the lag experienced in the testing. This is taken into account that I am running a nvidia 4070 gpu yet it is running on 99%, meaning there's still a long journey of optimisation that I could do. Yet, the solution that I have finalised with is still for the most part quite efficient (especially in comparing with the first iterations which I had). The usage of multithreading, native arrays alongside scheduling with closer mesh generating first, as well as the improved algorithm reduce the generating time by 50 times!

However, further improvements could be made to the landing process through

1. **Vectorising** – instead of having float3 values I could use float3x4 for 4 vertices data which could be simultaneously processed effectively reducing the overheads and maximise multithreading capabilities – the noise function by writing the noise myself – could use double instead of floats to allow a larger sample for greater flexibility in offset controls (This approach works especially well with the job and burst system considering that one of the main objectives for burst is to vectorise code for multithreading)
2. Change the **threshold calculation** so that the earlier less detailed mesh could be generated at earlier times. (I had tried this before, but this would require rewriting my code for both the scale change and the quadtree splits which deterred me. This is due to that I want to when I landed on planet, only the closest 4 quads are to be highest resolution to save memory which is quite intensive due to the sheer amount of meshes the planet has on max detail. This in turn would require an extra function to calculate the adjustive threshold distance from the distance and I haven't worked out an algorithm that is satisfactory for such purpose.)
3. **Compute Shaders** – this frees up the freezing CPU during rendering and instead directly call the rendering from GPU. This means the only calculation on CPU would be the vertex positions and triangulation, whereas the rendering calculations (such as colours/textures) would be done on directly on the GPU. Compute shaders are also much more efficient at multithreading as it has many more cores and therefore threads than the CPU. Offloading the rendering task there would help with performance a lot.

The overall aim in creating a playable game emulating the solar system is generally successful. The player could travel from planet to planet, as well as observe the planetary rotation around the sun. The main 'breakthrough' implemented is taking the idea of No Man's Sky landing procedure and LOD system for a massive planet that represents more accurately of the scales of planets with a flat horizon rather than 'walking on a sphere'. Obviously, the terrain is still empty, and the game is really boring with nothing to do realistically, but at least for each planet there's a unique terrain that is generated by noise with matching colours that mimic its looks. Again, with no storyline or objectives there's still a lot of extensions available in the future to make it a game.

Unfortunate Ending

It is a shame that projects must end as time is limited, it is a sombre moment as there are so much more to do, but perhaps this limitation is what prompt us as humans to experience life. I doubt I will pick this back up in the future, but in case I do, I will do a brief review on the suitability and difficulty for the unfinished extension objectives that I had.

In my initial end user requirements, there are 5 main extension categories which I haven't achieved: voxel, a larger universe, atmosphere, storyline, and autopilot. Out of these, with the current direction

of the game, voxel and larger universe makes no sense for this project. 1. Voxel simply breaks the aim of the game as a realistic exploration, whereas 2. larger universe makes the already difficult navigation even harder and will involve a substantial change in scale system. An incorporation of either would break the realism which I tried so hard to install.

For 3. atmosphere, I am pretty much there but I haven't figured out how to use render features and render graph to display the atmosphere. Hence, this would be the first extension for future development.

I had experimented with glsl on shadertoy using noise offset with further noise. (Inspiration thanks to IQ, although he did it for different purpose). This creates a cloudy simulation as shown in the following video. The code is as follows:


capture.webm

```
// Simplex Noise implementation by Inigo Quilez (https://www.shadertoy.com/view/Msf3WH)
vec2 hash(vec2 p) {
    p = vec2(dot(p,vec2(127.1,311.7)), dot(p,vec2(269.5,183.3)));
    return -1.0 + 2.0*fract(sin(p)*43758.5453123);
}

float noise(vec2 p) {
    const float K1 = 0.366025404;
    const float K2 = 0.211324865;

    vec2 i = floor(p + (p.x+p.y)*K1);
    vec2 a = p - i + (i.x+i.y)*K2;
    float m = step(a.y,a.x);
    vec2 o = vec2(m,1.0-m);
    vec2 b = a - o + K2;
    vec2 c = a - 1.0 + 2.0*K2;
    vec3 h = max(0.5-vec3(dot(a,a), dot(b,b), dot(c,c)), 0.0);
    vec3 n = h*h*h*h*vec3(dot(a,hash(i+0.0)), dot(b,hash(i+o)), dot(c,hash(i+1.0)));
    return dot(n, vec3(70.0));
}

float fractalNoise(vec2 uv) {
    float f = 0.0;
    float amplitude = 0.5;
    float frequency = 1.0;

    for(int i = 0; i < 6; i++) {
        f += amplitude * noise(uv * frequency);
        amplitude *= 0.5;
        frequency *= 2.0;
    }

    return f * 0.5 + 0.5;
}

void mainImage(out vec4 fragColor, in vec2 fragCoord) {
    vec2 uv = fragCoord.xy / iResolution.xy;
uv.x *= iResolution.x / iResolution.y;

    vec2 cloudUV1 = uv * 1.5 + iTime * vec2(0.03, 0.02);
float cloudBase = fractalNoise(cloudUV1);

    vec2 cloudUV2 = uv * 4.0 + iTime * vec2(-0.02, 0.03) + cloudBase * 0.5;
float cloudDetail = fractalNoise(cloudUV2);

    float clouds = smoothstep(0.3, 0.7, cloudBase * 0.7 + cloudDetail * 0.3);

    vec3 skyColor = mix(
        vec3(0.05, 0.15, 0.3),
        vec3(0.3, 0.6, 1.0),
        pow(uv.y, 0.5)
```

```
    );

    vec3 cloudColor = mix(
        vec3(0.7, 0.75, 0.8),
        vec3(1.0, 0.98, 0.95),
        smoothstep(0.4, 0.9, cloudDetail)
);

vec3 color = mix(skyColor, cloudColor, clouds * 0.9);
    color += vec3(0.4, 0.6, 1.0) * pow(1.0 - abs(uv.y - 0.5) * 2.0, 4.0) * 0.2;

    fragColor = vec4(color, 1.0);
}

```

Obviously, Unity use hlsl so I did translate into hlsl however I haven't been able to test it as I am still unfamiliar with its render system as I need this as a post processing shader.

After sorting the atmosphere, I would love to create a 4. storyline. The game is very empty right now more in its sandbox stage without any real objectives. Having a storyline would fulfil the ultimate objective in attracting younger teenagers to learn physics from a game, where I could put in objectives and achievements based on observation of such phenomenon in game and link to an exterior website (which I am currently working on alongside others) that explains such concepts.

Finally, the autopilot would help with the navigation more which is one of the issues that the user feedback highlights. The main difficulty is due to the impossibility of an analytic solution to gravitational attraction, unless I treat it the orbits using a simpler deterministic model (which I am hesitant towards) it would require me to simulate ahead (time depending how far the planet is away). This in turn exponentially increase the time complexity which is not great considering the already average current performance. Hence, this is ranked as the least pressing extension.

It must therefore be concluded that this project is an overall success, but the windows of exploration are still plenty, of which maybe in the future will be completed in a grand finale.

Appendix:

Main.cs

```
using ProceduralMeshes;
using ProceduralMeshes.Generators;
using ProceduralMeshes.Streams;
using UnityEngine;
using UnityEngine.Rendering;

[RequireComponent(typeof(MeshFilter), typeof(MeshRenderer))]
public class ProceduralMesh : MonoBehaviour
{
    [SerializeField, Range(1, 10)]
    private int resolution = 1;

    private Mesh mesh;


    private void Awake()
    {
        mesh = new Mesh
        {
            name = "Procedural Mesh"
        };
        GetComponent<MeshFilter>().mesh = mesh;
        GenerateMesh();
    }

    private void OnValidate()
    {
        GenerateMesh();
    }

    private void GenerateMesh()
    {
        Mesh.MeshDataArray meshDataArray = Mesh.AllocateWritableMeshData(1);
        Mesh.MeshData meshData = meshDataArray[0];

        MeshJob<SeamlessCubeSphere, PlanetStream>.ScheduleParallel(mesh, meshData,
resolution, default).Complete();

        Mesh.ApplyAndDisposeWritableMeshData(meshDataArray, mesh);
    }
}

```

SeamlessCubeSphere.cs

```
using Unity.Mathematics;
using UnityEngine;
using static Unity.Mathematics.math;
using static Noise;

namespace ProceduralMeshes.Generators
{
    public struct SeamlessCubeSphere : IMeshGenerator
    {
        struct Side
        {
            public int id;
            public float3 uvOrigin, uVector, vVector;
            public int seamStep;
            public bool TouchesMinimumPole => (id & 1) == 0; // bitwise and operation to
check if it is even
        }
```

```
17.         static Side GetSide(int id) => id switch
18.         {
19.             0 => new Side
20.             {
21.                 id = id,
22.                 uvOrigin = -1f,
23.                 uVector = 2f * right(),
24.                 vVector = 2f * up(),
25.                 seamStep = 4
26.             },
27.             1 => new Side
28.             {
29.                 id = id,
30.                 uvOrigin = float3(1f, -1f, -1f),
31.                 uVector = 2f * forward(),
32.                 vVector = 2f * up(),
33.                 seamStep = 4
34.             },
35.             2 => new Side
36.             {
37.                 id = id,
38.                 uvOrigin = -1f,
39.                 uVector = 2f * forward(),
40.                 vVector = 2f * right(),
41.                 seamStep = -2 // same as 4mod6 for next grid
42.             },
43.             3 => new Side
44.             {
45.                 id = id,
46.                 uvOrigin = float3(-1f, -1f, 1f),
47.                 uVector = 2f * up(),
48.                 vVector = 2f * right(),
49.                 seamStep = -2
50.             },
51.             4 => new Side
52.             {
53.                 id = id,
54.                 uvOrigin = -1f,
55.                 uVector = 2f * up(),
56.                 vVector = 2f * forward(),
57.                 seamStep = -2
58.             },
59.             _ => new Side
60.             {
61.                 id = id,
62.                 uvOrigin = float3(-1f, 1f, -1f),
63.                 uVector = 2f * right(),
64.                 vVector = 2f * forward(),
65.                 seamStep = -2
66.             }
67.
68.         };
69.         public int Resolution { get; set; }
70.         public int VertexCount => 6 * Resolution * Resolution + 2;
71.         public int IndexCount => 6 * 6 * Resolution * Resolution;
72.         public int JobLength => 6 * Resolution;
73.         public void Execute<S>(int i, S streams) where S : struct, IMeshStreams
74.         {
75.             // u for row number and v for column number
76.             int u = i / 6;
77.             Side side = GetSide(i - 6 * u);
78.             int vi = Resolution * (Resolution * side.id + u) + 2;
79.             int ti = 2 * Resolution * (Resolution * side.id + u);
80.             bool firstColumn = u == 0;
81.
82.             u++;
83.
84.             float3 pStart = side.uvOrigin + side.uVector * u / Resolution;
85.
86.             var vertex = new Vertex();
```

```
            if (i == 0)
            {
                vertex.position = -sqrt(1f / 3f);
                streams.SetPosition(0, vertex);
                vertex.position = sqrt(1f / 3f);
                streams.SetPosition(1, vertex);
            }

            vertex.position = CubeToSphere(pStart);
            streams.SetPosition(vi, vertex);

            /* This is the expanded version of the following code, which helps to reduce the numbers of calling of SetTriangle to 1
            if (firstColumn)
            {
                if (side.TouchesMinimumPole)
                {
                    streams.SetTriangle(ti, int3(vi, 0, vi + side.seamStep * Resolution * Resolution));
                    // first value describes the min vertex of the grid, the second the minimum and the third the previous grid's vertex as it forms the top for the triangle
                }
                else
                {

                    streams.SetTriangle(ti, vi + int3(0, -Resolution, Resolution == 1 ? side.seamStep : -Resolution + 1));
                }
            }
            else
            {
                streams.SetTriangle(ti, vi + int3(0, -Resolution, -Resolution + 1));
            }
            */

            var triangle = int3
                (vi,
                firstColumn && side.TouchesMinimumPole ? 0 : vi - Resolution,
                vi + (firstColumn ?
                side.TouchesMinimumPole ?
                side.seamStep * Resolution * Resolution
                : Resolution == 1 ? side.seamStep : -Resolution + 1
                : -Resolution + 1)
                );

            streams.SetTriangle(ti, triangle);

            vi+=1;
            ti+=1;

            //logic for triangle index incrementation, pulled out of loop as it is invariant!
            int zAdd = firstColumn && side.TouchesMinimumPole ? Resolution : 1;
            int zAddLast = firstColumn && side.TouchesMinimumPole ?
                Resolution :
                !firstColumn && !side.TouchesMinimumPole ?
                    Resolution * ((side.seamStep + 1) * Resolution - u) + u :
                    (side.seamStep + 1) * Resolution * Resolution - Resolution + 1;

            for (int v = 1; v < Resolution; v++, vi ++, ti += 2)
            {
                vertex.position = CubeToSphere(pStart + side.vVector * v / Resolution);
                streams.SetPosition(vi, vertex);

                // logic for how to increment triangle index based on the unfolding of our sphere
                /*
                if (firstColumn && side.TouchesMinimumPole)
                {
                    triangle.z += Resolution;
```

```
                }
                else if (!firstColumn && !side.TouchesMinimumPole)
                {
                    triangle.z +=
                        Resolution * ((side.seamStep + 1) * Resolution - u) + u;
                }
                else
                {
                    triangle.z +=
                        (side.seamStep + 1) * Resolution * Resolution -
                        Resolution + 1;
                }
                */
                triangle.x += 1;
                triangle.y = triangle.z;
                triangle.z += v == Resolution - 1 ? zAddLast : zAdd;

                streams.SetTriangle(ti, int3(triangle.x - 1, triangle.y, triangle.x));
                streams.SetTriangle(ti + 1, triangle);

            }
            streams.SetTriangle(ti, int3(
                triangle.x,
                triangle.z,
                side.TouchesMinimumPole ?
                    triangle.z + Resolution :
                    u == Resolution ? 1 : triangle.z + 1
            ));
        }
        public Bounds Bounds => new Bounds(Vector3.zero, new Vector3(2f, 2f, 2f));
        static float3 CubeToSphere(float3 p) => p * sqrt(1f - ((p * p).yxx + (p * p).zzy) /
2f + (p * p).yxx * (p * p).zzy / 3f);
    }
}

```

PlanetStream.cs

```
using System.Runtime.CompilerServices;
using System.Runtime.InteropServices;
using Unity.Collections;
using Unity.Mathematics;
using UnityEngine;
using UnityEngine.Rendering;
using Unity.Collections.LowLevel.Unsafe;
using UnityEngine.UIElements;
using static Noise;

namespace ProceduralMeshes.Streams
{
    // Struct needed compile with Burst
    public struct PlanetStream : IMeshStreams
    {
        [StructLayout(LayoutKind.Sequential)]
        public struct Stream0
        {
            public float3 position;
            public float3 normal;
            public float4 tangent;
            public float2 texCoord0;
        }
        [NativeDisableContainerSafetyRestriction]
        private NativeArray<Stream0> stream0;
        [NativeDisableContainerSafetyRestriction]
        private NativeArray<TriangleUInt16> triangles;

        public void Setup(Mesh.MeshData meshData, Bounds bounds, int vertexCount, int
indexCount)
        {
```

```
31.             NativeArray<VertexAttributeDescriptor> meshDataDescriptor = new
NativeArray<VertexAttributeDescriptor>(4, Allocator.Temp,
NativeArrayOptions.UninitializedMemory);
32.             meshDataDescriptor[0] = new VertexAttributeDescriptor(VertexAttribute.Position,
dimension: 3);
33.             meshDataDescriptor[1] = new VertexAttributeDescriptor(VertexAttribute.Normal,
dimension: 3);
34.             meshDataDescriptor[2] = new VertexAttributeDescriptor(VertexAttribute.Tangent,
dimension: 4);
35.             meshDataDescriptor[3] = new VertexAttributeDescriptor(VertexAttribute.TexCoord0,
dimension: 2);
36.
37.             meshData.SetVertexBufferParams(vertexCount, meshDataDescriptor);
38.             meshDataDescriptor.Dispose();
39.
40.             meshData.SetIndexBufferParams(indexCount, IndexFormat.UInt16);
41.
42.             meshData.subMeshCount = 1;
43.             meshData.SetSubMesh(0, new SubMeshDescriptor(0, indexCount)
44.             {
45.                 bounds = bounds,
46.                 vertexCount = vertexCount
47.             },
48.             MeshUpdateFlags.DontRecalculateBounds | MeshUpdateFlags.DontValidateIndices);
49.
50.             stream0 = meshData.GetVertexData<Stream0>();
51.             triangles = meshData.GetIndexData<ushort>().Reinterpret<TriangleUInt16>(2);
52.         }
53.
54.         [MethodImpl(MethodImplOptions.AggressiveInlining)]
55.         public void SetPosition(int index, Vertex vertex)
56.         {
57.             stream0[index] = new Stream0
58.             {
59.                 position = vertex.position,
60.                 normal = vertex.normal,
61.                 tangent = vertex.tangent,
62.                 texCoord0 = vertex.texCoord0
63.             };
64.         }
65.
66.         [MethodImpl(MethodImplOptions.AggressiveInlining)]
67.         public void SetPosition(int index, float3 position)
68.         {
69.             stream0[index] = new Stream0
70.             {
71.                 position = position
72.             };
73.         }
74.
75.         public void SetTriangle(int index, int3 triangle)
76.         {
77.             triangles[index] = triangle;
78.         }
79.     }
80. }
81.
```